IFC4x3 Alignment Geometry Implementation Guide

Richard Brice, PE

Forward

`IfcAlignment`, and the related geometric concepts, are new to the IFC4x3 specification. By virtue of expanding the IFC specification into the infrastructure domain, an entirely new audience is summoned into the buildingSmart community. I am one of those new arrivals.

The IFC specification is very technical and is written with an expectation that the reader has a high level of knowledge and experience with IFC. Being a technical specification, there is no hand-holding for the novice. Industry guidance documents are generally not available to assist those new to the infrastructure and alignment aspects of IFC4x3.

I set out to add alignment geometry support to the IfcOpenShell Toolkit. Being a bridge engineer, I am well versed in highway alignment geometry. I was completely overwhelmed and befuddled with alignment representations in the IFC specifications. Through a long and persistent struggle, I was able to figure out most all of what a developer needs to implement IFC alignment geometry in a software application.

This implementation guide captures what I have learned about IFC4x3 alignment geometry and provides guidance as to how a software developer may go about creating an implementation. I hope that other developers find this guide useful, because in the end, IFC is about interoperability. We all need a common understanding of the infrastructure alignment concepts and geometry to successfully exchange information.

Table of Contents

1.0 Introduction
2.0 Horizontal
3.0 Vertical
4.0 Cant
5.0 Linear Placement
6.0 Offset Curves
7.0 Referents and Stationing
8.0 Specialized Alignment-based Infrastructure Geometry
9.0 LandXML to IFC
10.0 Precision and Tolerance
11.0 Examples and Validation Data

Section 1.0 - Introduction

IFC 4.3 ADD 2 finalized the adoption of alignments for infrastructure works. Since the concept of alignment is relatively new to IFC, few details and examples are provided in the available literature. This guide aims to document the relevant concepts and mathematics a software developer might need to implement alignment-based geometry.

If you are unfamilar with infrastructure alignment geometry, information is available in a multitude of highway engineering, rail engineering, and surveying texts. A concise primer is available in the US Federal Highway Adminstration (FHWA), Bridge Geometry Manual, Chapters 1 - 5 and Appendix B.

The IFC specification identifies concept templates as the mechanism for mapping semantic alignment representations to their geometric counterparts. The concept templates — covering aggregation to project, alignment layout nesting, and the four alignment geometry variants (horizontal; horizontal+vertical; horizontal+vertical+cant; segments) — provide the minimum information required to properly structure alignment semantic and geometric entities an IFC model. The *partial* concept templates that cover individual curve segment geometry types, however, are sparse: they show class relationships but provide no mathematical equations or parameter mappings. This guide fills that gap. It assumes the reader has a basic working knowledge of IFC — familiarity with the schema structure, entity relationships, and how IFC files are organized — and focuses specifically on the alignment geometry that IFC 4.3 ADD 2 introduced.

1.1 Horizontal Alignment

A horizontal alignment typically consists of straight lines called tangents (or tangent runs) that are connected by curves. The curves are usually circular arcs. Easement or transition curves in the form of spirals can be used to provide gradual transitions between tangents and circular curves. The horizontal alignment is a plan view curvilinear path in a Cartesian coordinate system aligned with East and North. Positions along an alignment, measured along the plan view projection of the 3D alignment curve, are typically denoted with stations.

1.2 Vertical Alignment

A vertical alignment (often referred to as the vertical profile by civil engineers) consists of straight sections of grade lines connected by vertical curves. The vertical curves are typically parabolas, though sometimes circular arcs or clothoid curves are used. A vertical alignment is defined along the curvilinear path of a horizontal alignment in a 2D "Distance Along, Elevation" coordinate system. Combined, the horizontal and vertical alignments define a 3D curve. This combination of two 2D curves is sometimes referred to as a 2.5D or 2D+1D geometry. The IFC specification notes that contemporary alignment design almost always implements this 2.5D approach, with precision varying based on management priorities, design era, and available software tools.

1.3 Cant Alignment

When a rail vehicle traverses a horizontal curve, centrifugal force acts laterally on passengers and cargo. Tilting the track cross-section — raising the outer (high) rail above the inner (low) rail — converts part of that lateral force into a downward component, improving ride comfort and permitting higher operating speeds through curves. This tilt is called **cant** (or *superelevation* in road engineering, though the two are geometrically analogous).

Cant is measured as the height difference between the two rail heads, perpendicular to the track. The low (inner-curve) rail is the conventional pivot for rotation; the high (outer-curve) rail rises by the cant amount.

A cant alignment is structured similarly to the horizontal and vertical alignments: **constant-cant zones** — corresponding to the circular arc sections of the horizontal alignment, where cant is held at a fixed value — are connected by **cant transitions** that ramp over the length of the horizontal transition spiral. Transitions may be linear or may follow the same non-linear curve families used for horizontal spirals.

Like the vertical alignment, the cant alignment is defined in a two-dimensional profile: distance along the horizontal alignment on one axis, cant value on the other. The cant profile encodes not only a deviating elevation but also the cross slope of the rail head, fully defining the rotated cross-section. This profile is then layered onto the combined horizontal and vertical alignment to produce a full 3D alignment curve incorporating cross-sectional banking.

1.4 Semantic Definition (Business Logic)

The semantic definition of alignment allows for the alignment to be described as close as possible to the terminology and concepts used in a business context. Examples include horizontal, vertical and cant layouts, stationing, and anchor points for domain specific properties such as design speed, cant deficiency, superelevation transitions, and widenings.

Specialized applications can analyze alignments using the semantic information. Alignments can be evaluated against design criteria such as speed requirements, sight distances, and maximum gradient, to name a few.

Importantly, the semantic and geometric representations are independent and can be used and exchanged separately. A proper IFC file may contain only the semantic definition without any geometric representation, which is common in early design stages when geometry has not yet been computed.

An `IfcAlignment` may be defined with a horizontal layout only, a horizontal and vertical layout, or a full horizontal, vertical, and cant layout. Each layout type is composed through an `IfcRelNests` relationship: `IfcAlignmentHorizontal`, `IfcAlignmentVertical`, and `IfcAlignmentCant` are nested within the `IfcAlignment` as applicable. Each layout is in turn composed of one or more `IfcAlignmentSegment` entities, also through `IfcRelNests`. The `IfcAlignmentSegment` models the segment design parameters in a subtype of `IfcAlignmentParameterSegment`: `IfcAlignmentHorizontalSegment`, `IfcAlignmentVerticalSegment`, or `IfcAlignmentCantSegment`, depending on the layout. The semantic definition model is shown in Figure 1.4-1.

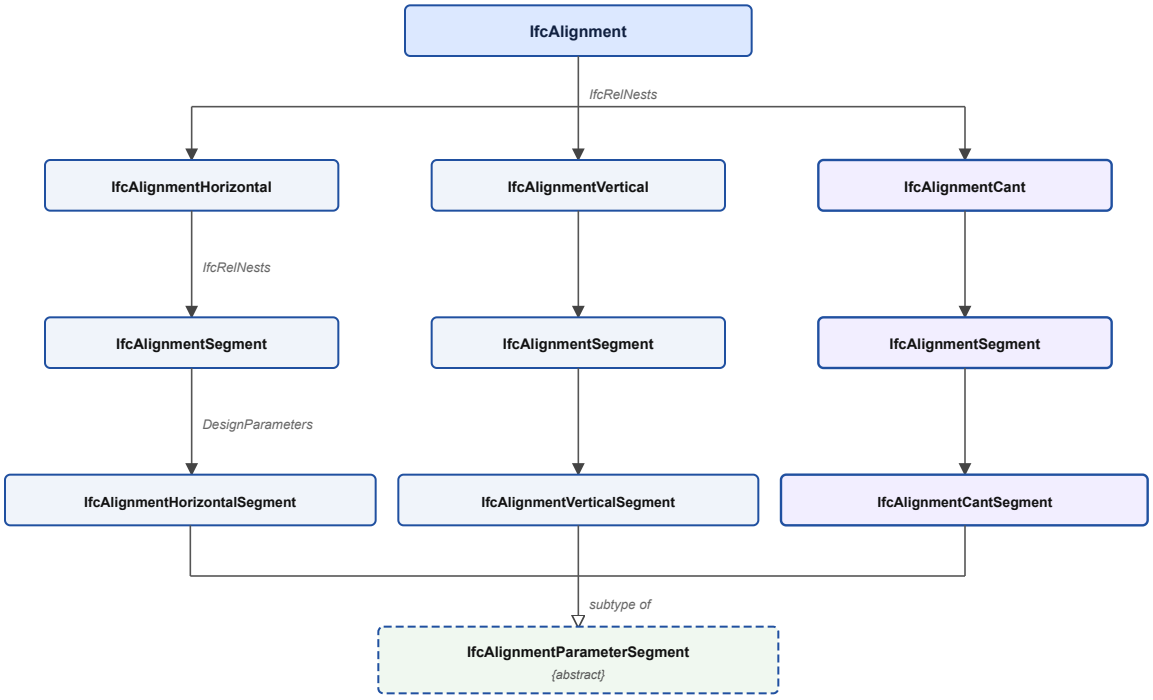

Figure 1.4-1 Semantic alignment model

`IfcRelNests.RelatedObjects` is an ordered collection. That means there is a specific order for the alignment layouts and the alignment segments. The IFC specification doesn't explicitly define a ranking for ordering. The ordering of the alignment layouts can be infered from the images in the `IfcAlignment` documentation 5.4.3.1.5 as horizontal preceeds vertical, which preceeds cant. The ordering of `IfcAlignmentSegment` in an alignment layout is equally nebulus. For `IfcAlignmentHorizontalSegment` the only logical ordering is start to end. For `IfcAlignmentVerticalSegment` and `IfcAlignmentCantSegment` ordering in `IfcRelNests.RelatedObjects` could be anything because positional ordering can be derived from the `StartDistAlong` attribute. Such an approach is unnecessarily difficult. Segments should always be ordered in `IfcRelNests.RelatedObjects` in a start to end sequence.

An interesting caveat of the alignment layout entities (`IfcAlignmentHorizontal`, `IfcAlignmentVertical`, and `IfcAlignmentCant`) is that the last `IfcAlignmentSegment.DesignParameters` must be zero length. If a geometric representation is provided, then its corresponding last segment must also be zero length. For a continuous composition of segments, the end of one segment is at the same location as the start of the next segment. The zero-length segment is intended to provide the end point of the last segment.

Aggregation to project. `IfcAlignment` is a mandatory participant in the IFC spatial structure. Every alignment must be aggregated to `IfcProject` — directly, or indirectly through a parent alignment — via an `IfcRelAggregates` relationship. An alignment not connected to the project is invalid.

Referencing to site. Because an alignment typically traverses multiple administrative or physical sites, `IfcAlignment` associates with `IfcSite` using `IfcRelReferencedInSpatialStructure` rather than `IfcRelContainedInSpatialStructure`. The referenced-in relationship is non-exclusive: a single alignment may be referenced by every `IfcSite` it crosses. This is the key distinction between *containment* (one element, one spatial zone) and *referencing* (one element, many zones).

1.5 Geometric Representation

1.5.1 Overview

Section 4.1.7.1.1 of the IFC 4x3 specification indicates that the valid representations of `IfcAlignment` are:

- `IfcCompositeCurve` as a 2D horizontal alignment (represented by its horizontal alignment segments), without a vertical layout.
- `IfcGradientCurve` as a 3D horizontal and vertical alignment (represented by their alignment segments), without a cant layout.
- `IfcSegmentedReferenceCurve` as a 3D curve defined relative to an IfcGradientCurve to incorporate the application of cant.
- `IfcOffsetCurveByDistances` as a 2D or 3D curve defined relative to an `IfcGradientCurve` or another `IfcOffsetCurveByDistances`.
- `IfcPolyline` or `IfcIndexedPolyCurve` as a 3D alignment by a 3D polyline representation (such as coming from a survey).
- `IfcPolyline` or `IfcIndexedPolyCurve` as a 2D horizontal alignment by a 2D polyline representation (such as in very early planning phases or as a map representation).

The `RepresentationIdentifier` and `RepresentationType` values required by the IFC concept templates for each alignment geometry variant are listed in Table 1.5.1-1.

Alignment variant	Curve entity	`RepresentationIdentifier`	`RepresentationType`
Horizontal only	`IfcCompositeCurve`	`'Axis'`	`'Curve2D'`
Horizontal + Vertical	`IfcCompositeCurve` (plan view)	`'FootPrint'`	`'Curve2D'`
Horizontal + Vertical	`IfcGradientCurve` (3D)	`'Axis'`	`'Curve3D'`
Horizontal + Vertical + Cant	`IfcCompositeCurve` (plan view)	`'FootPrint'`	`'Curve2D'`
Horizontal + Vertical + Cant	`IfcSegmentedReferenceCurve` (3D)	`'Axis'`	`'Curve3D'`

Table 1.5.1-1 — Required RepresentationIdentifier and RepresentationType for each alignment geometry variant

`IfcGradientCurve` and `IfcSegmentReferenceCurve` inherit from `IfcCompositeCurve`. These curves consist of a sequence of segments defined by `IfcCurveSegment`. The mathematical computations for `IfcCurveSegment` geometry are the primary focus of this document.

`IfcPolyline` and `IfcIndexedPolyCurve` are trivial mathematically and are not discussed in this document.

`IfcOffsetCurveByDistances` is treated in a dedication section.

1.5.2 Understanding Geometric Representation of Alignment

The semantic definition of an alignment and its geometric representation carry overlapping information — both describe segment types, lengths, and radii — but they serve different consumers. The semantic representation encodes design intent in domain vocabulary: a horizontal segment typed as `CLOTHOID` with a `StartRadius` of infinity, an `EndRadius` of 300 m, and a `SegmentLength` of 100 m tells a design application *what* the engineer specified and enables evaluation against standards for minimum radius, design speed, and sight distance. The geometric representation encodes the computed mathematical form: the exact start coordinates, the tangent bearing at each point, and parametric equations that yield position and direction at any arc-length along the curve — the form that a rendering engine, clash-detection tool, or quantity-takeoff calculation requires. Because the two representations serve different consumers, an IFC file may legally contain only the semantic definition, only the geometric representation, or both.

The geometric representation of `IfcAlignment` consists of one or more `IfcShapeRepresentation` instances: a plan-view 2D curve and, where vertical or cant geometry is present, a 3D curve. These are illustrated in Figure 1.5.2-1.

Geometrically, the horizontal alignment is defined by an `IfcCompositeCurve` and the vertical alignment is defined by an `IfcGradientCurve`. The horizontal alignment is the basis curve for the vertical alignment.

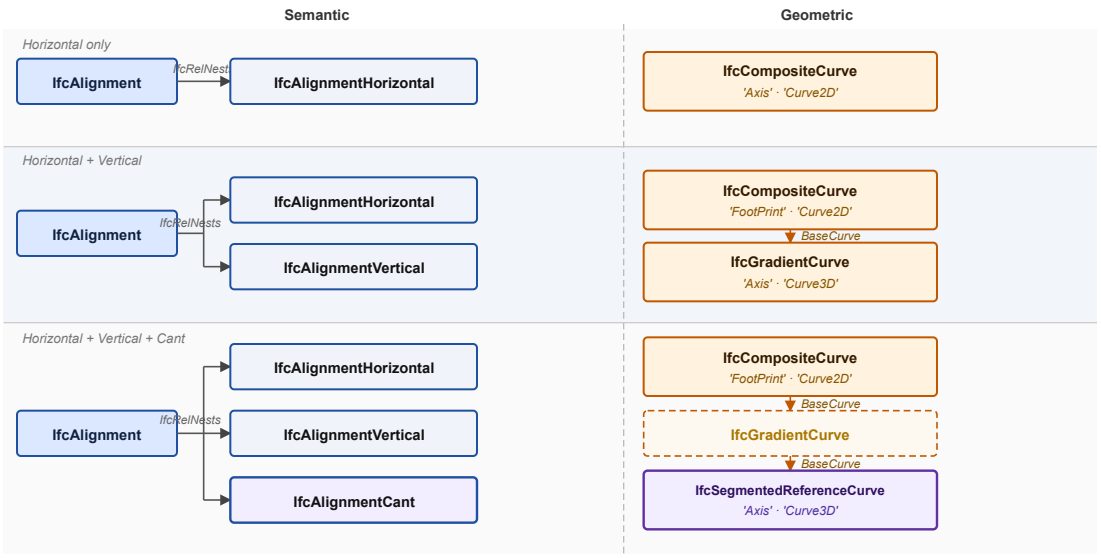

Figure 1.5.2-1 Geometric representation variants for the three alignment constructs (H, H+V, H+V+C). The dashed `IfcGradientCurve` box in the H+V+C row appears as a `BaseCurve` intermediate only — it is not itself a shape representation.

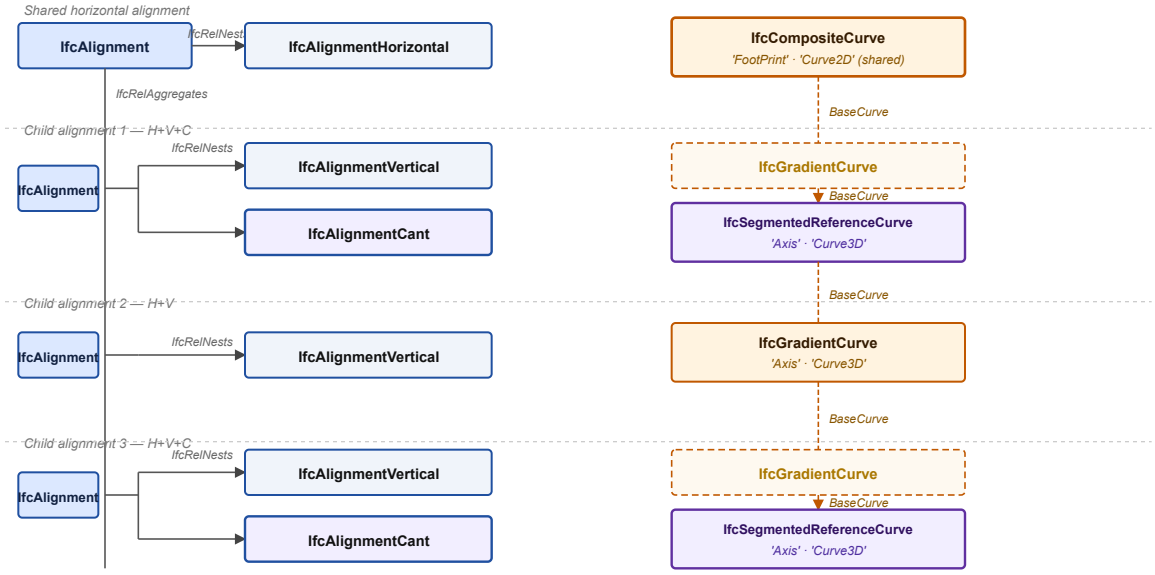

Figure 1.5.2-2 Reuse of a shared `IfcCompositeCurve` across multiple child alignments. Each child `IfcAlignment` nests only its own vertical and cant layouts; the horizontal `IfcCompositeCurve` is referenced as `BaseCurve` by each child's gradient or segmented reference curve.

The geometric elements are modeled with `IfcCurveSegment`. `IfcCurveSegment` cuts a segment from a parent curve and places it in the alignment coordinate system. A horizontal circular arc is modeled with an `IfcCurveSegment` that cuts an arc from an `IfcCircle`. Figure 1.5.2-3 shows an alignment consisting of a tangent run (red) and a horizontal curve (green). The line and curve segments are cut from their `IfcLine` and `IfcCircle` parent curves, respectively. The process of defining a parent curve, cutting a segment from that curve, and placing the cut segment into the alignment

is repeated to define the entire horizontal alignment. All the horizontal alignment `IfcCurveSegment` objects are then combined to create an `IfcCompositeCurve` to complete the plan view geometric representation of the horizontal alignment.

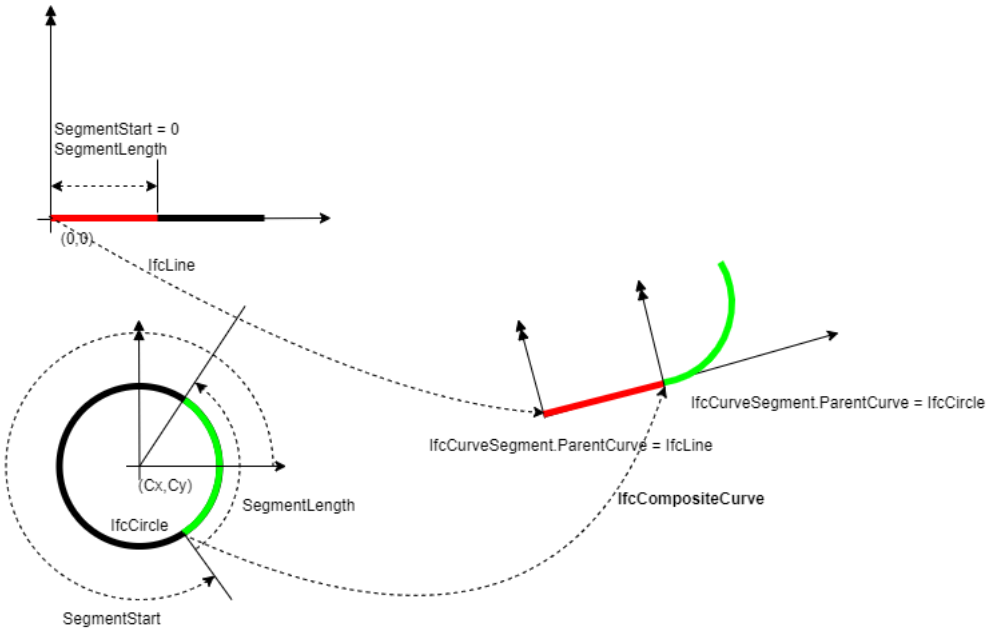

Figure 1.5.2-3 Illustration of parent curves and their placement with `IfcCurveSegment.Placement`

A similar process is used to create the vertical alignment. `IfcCurveSegment` trim geometry from parent curves and combine end to start to create an `IfcGradientCurve`. `IfcGradientCurve` is a sub-type of `IfcCompositeCurve` with the additional attribute `BaseCurve`. The segments of the gradient curve are positions in a 2-dimensional plane in a "distance along the horizontal alignment, elevation" coordinate system. The `IfcGradientCurve` uses the horizontal alignment `IfcCompositeCurve` as its base curve.

`IfcSegmentedReferenceCurve` is also a sub-type of `IfcCompositeCurve`. It defines a deviating elevation and rail head cross slope along the base curve. The base curve is typically an `IfcGradientCurve`, though the IFC specification would permit it to be any `IfcCompositeCurve`. The "distance along" coordinate is along the gradient curve's base curve, which is the horizontal alignment curve.

1.5.2.1 Curve segment geometry

When defining curve segment geometry with an `IfcCurveSegment`, the location and orientation of the parent curve are not particularly important. The `IfcCurveSegment.Placement` defines the location of the start point of the trimmed curve as well as the orientation of the tangent at the start of the trimmed curve. In the example above, the `IfcLine` parent curve is starts at (0,0) and progresses in horizontal direction, but the tangent segment of the alignment starts an specific (X,Y) position and is oriented in a North-East direction. The `IfcCurveSegment.Placement` attribute establishes this position and orientation.

1.5.2.2 Vertical and Cant Curve Extents

For `IfcGradientCurve` and `IfcSegmentedReferenceCurve` the segments do not need to start or end at the same location of their base curve. These curves can be shorter or longer than the horizontal alignment. This is illustrated in Figure 1.5.2.2-1 where the blue curve is the full horizontal curve and the orange curve is the portion of the horizontal curve coinciding with the vertical curve. The `IfcGradientCurve` (#770) has the `IfcCompositeCurve` (#657) as its base curve. The placement of the first `IfcCurveSegment` (#771) in the `IfcGradientCurve` is defined by `IfcAxis2Placement2D` (#777) with Location as `IfcCartesianPoint` (#778). The first segment is located at a distance of 250.0036 along the horizontal `IfcCompositeCurve` at an elevation of 145.3129 m. The IFC specification does not provide guidance on how to define complete 3D geometry in the regions of the horizontal alignment that do not overlap with the vertical or cant curves.

```
#657=IFCCOMPOSITECURVE((#658,#671,#684,#697,#710,#723,#736,#749,#762),.F.)
#770=IFCGRADIENTCURVE((#771,#784,#796,#809,#821,#834,#846,#859,#871),.F.,#657,#887)
#771 = IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE., #777, FCNONNEGATIVELENGTHMEASURE(0.),
IFCNONNEGATIVELENGTHMEASURE(40.0002408172751), #780);
#777 = IFCAXIS2PLACEMENT2D(#778, #779);
#778 = IFCCARTESIANPOINT((250.003634293681, 145.312908277025));
```

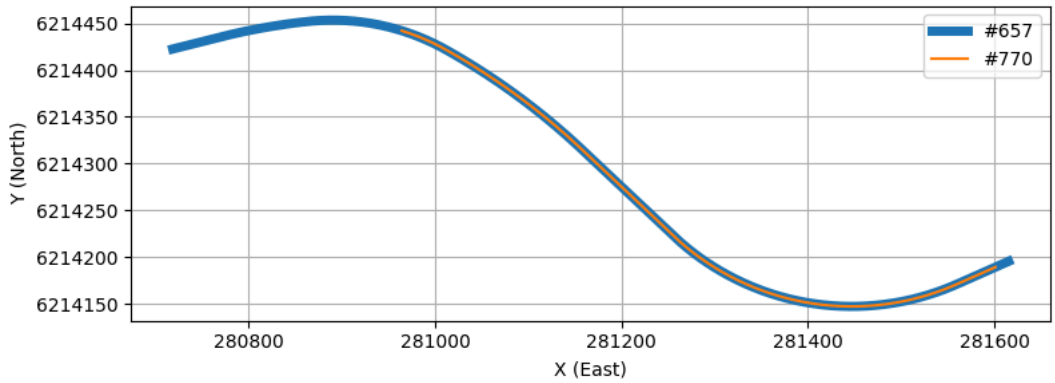

Figure 1.5.2.2-1 — Example of `IfcGradientCurve` with a start location and length different than the `IfcCompositeCurve` BaseCurve.

1.6 Reference Implementatio ans Segment Mapping:

The sections that follow cover the mapping of the semantic definition of alignment segments (`IfcAlignmentSegment` subtypes) to their corresponding geometric representation (`IfcCurveSegment.ParentCurve`). In general, the mapping formulas are given without derivation and have been developed from the reference implementation, EnrichIFC4x3, published at IFC-Rail-Unit-Test-Reference-Code/EnrichIFC4x3/EnrichIFC4x3/business2geometry at master · bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code (github.com).

The cant segment mappings in the EnrichIfc4x3 reference implementation are incorrect. These errors are documented in Section 4, which presents corrected formulas and explains why the reference implementation is incorrect.

1.7 Units of Measure

Unless otherwise specified, the following unit conventions apply throughout this guide:

Quantity	Unit
Length, distance, elevation, offset, cant	meter (m)
Spiral and polynomial coefficients (A, A_0, A_1, ...)	meter (m)
Curvature ($\kappa = 1/R$)	reciprocal meter (m^{-1})
Angles (bearings, grade angles, cross-slope angles)	radian (rad)
Gradient (vertical slope)	dimensionless ratio (m/m)

Table 1.7-1 — Unit conventions used throughout this guide

Gradient is expressed as a decimal rise-over-run value — 0.02 represents a 2% grade — not as a percentage or angle.

todo:

- Add figure for IfcCircle showing local and global axes, along with trimmed portion and start trim tangent
- Add figure for IfcClothoid showing both the X < 0 and X > 0 sides. Show trimmed portion highlighting that SegmentStart is a negative value and SegmentLength is a positive value. Add a discussion about the sign of start and length, being clear that it is true for all IfcCurveSegment trimming, not just clothoid.
- Review all subsections in this section for consistency

Section 2.0 - Horizontal Alignment

The geometric representation of a horizontal alignment is accomplished with an `IfcCompositeCurve`. The composite curve consists of a sequence of `IfcCurveSegment` entities whose geometry is defined by a parent curve. This section defines the mathematical relationships and equations for each parent curve type and the algorithm for evaluating points on those curves.

Table 2.0-1 maps each `IfcAlignmentHorizontal.PredefinedType` to its corresponding parent curve type.

Business Logic (`IfcAlignmentHorizontal.PredefinedType`)	**Geometric Representation (`IfcCurveSegment.ParentCurve`)**
LINE	`IfcLine`
CIRCULARARC	`IfcCircle`
CLOTHOID	`IfcClothoid`
CUBIC	`IfcPolynomialCurve`
HELMERTCURVE	`IfcSecondOrderPolynomialSpiral`
BLOSSCURVE	`IfcThirdOrderPolynomialSpiral`
COSINECURVE	`IfcCosineSpiral`
SINECURVE	`IfcSineSpiral`
VIENNESEBEND	`IfcSeventhOrderPolynomialSpiral`

Table 2.0-1 — Mapping of business logic to geometric representation for horizontal alignment

2.1 General

The following parameters are common to all horizontal alignment curve types. Each curve is parameterized by arc-length s, where $s = 0$ at the start of the parent curve. The start and end radii R_s and R_e are taken from `IfcAlignmentHorizontalSegment.StartRadius` and `EndRadius`; a value of zero indicates infinite radius (zero curvature, i.e. a straight line). The segment length L is `IfcAlignmentHorizontalSegment.SegmentLength`. The tangent angle $\theta(s)$ is measured from the positive x-axis; its cosine and sine form the `RefDirection` of the curve at s.

Parameter	**Equation**
Start Curvature	$\kappa_s = \frac{1}{R_s}$
End Curvature	$\kappa_e = \frac{1}{R_e}$
Cumulative change in curvature	$f = \frac{L}{R_e} - \frac{L}{R_s} = L\left(\frac{1}{R_e} - \frac{1}{R_s}\right)$
Angle of tangent line, RefDirection = $\cos(\theta(s)), \sin(\theta(s))$	$\theta(s) = \int \kappa(s) ds$
X-ordinate as a function of curve length s	$x(s) = \int \cos(\theta(s)) ds$
Y-ordinate as a function of curve length s	$y(s) = \int \sin(\theta(s))\, ds$

Table 2.1-1 — Common horizontal alignment parameters

2.2 Curve Segment Evaluation Algorithm

This algorithm evaluates the 2D position and tangent direction of a point on an `IfcCurveSegment` in the alignment coordinate system. Let s_0 = `SegmentStart` and s = the arc-length parameter of the point to evaluate, where $s_0 \leq s < s_0 + \texttt{SegmentLength}$.

Step 1 — Evaluate the parent curve at the trim start

Compute the position (x_0, y_0) and tangent angle θ_0 of the parent curve at s_0. This establishes the frame of the parent curve at the point where trimming begins.

Step 2 — Form the normalization matrix M_N

M_N simultaneously translates the trim-start point to the origin and rotates so that the tangent at s_0 aligns with the positive x-direction.

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -x_0\cos\theta_0 - y_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & -x_0\sin\theta_0 - y_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

M_{CSP} places the trimmed segment into the alignment coordinate system. It is constructed directly from `IfcCurveSegment.Placement`, where (x_p, y_p) is the `Location` and θ_p is the bearing of the `RefDirection`.

$$M_{CSP} = \begin{bmatrix} \cos\theta_p & -\sin\theta_p & 0 & x_p \\ \sin\theta_p & \cos\theta_p & 0 & y_p \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Evaluate and map each point

For the point at arc-length s, compute the parent curve position $(x(s), y(s))$ and tangent angle $\theta(s)$ and form:

$$M_{PC} = \begin{bmatrix} \cos(\theta(s)) & -\sin(\theta(s)) & 0 & x(s) \\ \sin(\theta(s)) & \cos(\theta(s)) & 0 & y(s) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and placement in sequence:

$$M_h = M_{CSP} M_N M_{PC}$$

The position of the point in the alignment coordinate system is the fourth column of M_h, and its tangent direction is the first column. Numerical examples of this algorithm are given in Sections 2.3 through 2.11.

2.3 Line

A tangent run is geometrically represented with a segment trimmed from an `IfcLine` parent curve.

2.3.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(t) = \theta$$

$$\kappa(t) = 0$$

The point on line equation can be parameterized with a **unit parameterization** where the parameter u ranges from 0 to 1 over the full extent of the curve, independent of its physical length.

$$\lambda(u) = C + L\left(\int_0^{u=1} \cos\left(\theta(t)\right)dt\ x,\ \int_0^{u=1} \sin\left(\theta(t)\right)dt\ y\right)$$

It can also be parameterized with an **arc length parameterization** where the parameter u equals the distance along the line in units of length, so $u = s$.

$$\lambda(u) = C + \left(\int_0^{u=L} \cos(\theta(t))dt\ x,\ \int_0^{u=L} \sin(\theta(t))dt\ y \right)$$

Other parent curve types can also be parameterized by unit value or arc length. However, the polynomial spirals do not have a well defined unit value parameterization. For this reason, the IFC specification mandates that all parameterization for alignment geometry be by arc length. For `IfcCurveSegment`, the `SegmentStart` and `SegmentLength` attributes **must** be of type `IfcLengthMeasure`. See Section 8.9.3.28 IfcCurveSegment, Informal Proposition 1.

This matters because `IfcCurveSegment.SegmentStart` and `IfcCurveSegment.SegmentLength` are of type `IfcCurveMeasureSelect`, which can be either `IfcParameterValue` (a dimensionless scalar based on unit value) or `IfcLengthMeasure` (a physical length).

2.3.2 Semantic Definition to Geometry Mapping

Mapping of the semantic definition of the linear segment to the geometric definition is described with the following example.

Given a horizontal alignment segment is a line segment starting at point (500,2500), bearing in the direction S 57 E (5.70829654085293 radian) and has a length of 1956.785654.

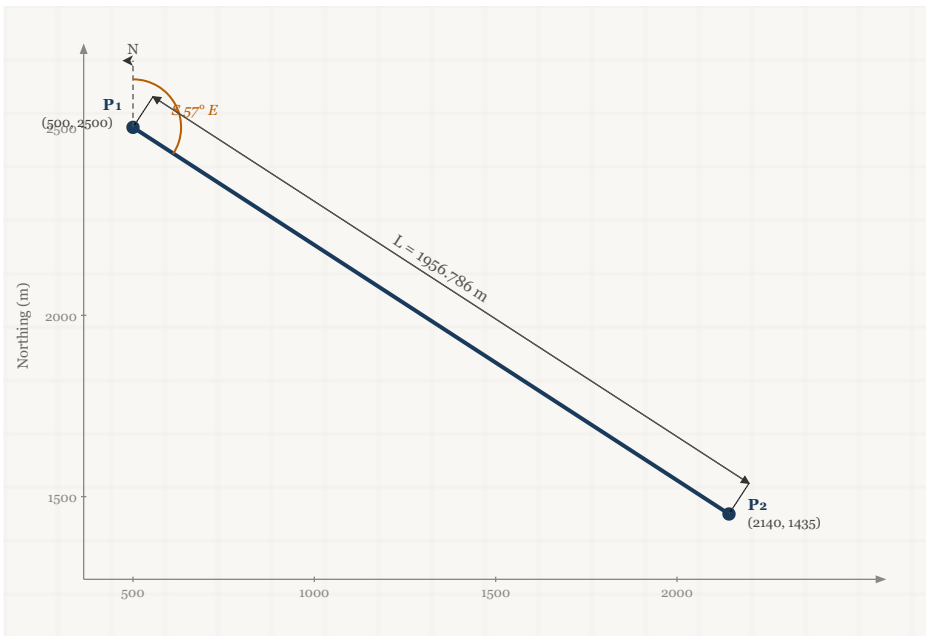

Figure 2.3.2-1 Tangent segment

This semantic definition of this segment is:

```
#31=IFCCARTESIANPOINT((500.,2500.));
#32=IFCALIGNMENTHORIZONTALSEGMENT($,$,#31,5.70829654085293,0.,0.,1956.785654,$,.LINE.);
```

The geometric representation is an `IfcLine`. The line can be defined in different ways and is generally not important, as will be shown in the example below. `IfcLine` is an infinitely long line that passes through a specified point at some direction in the X-Y plane. A valid, but unnecessisarly complex parent curve is show in Figure 2.3.2.2-1. The `IfcLine` passes through poin t(1000,-800) and is direct at 135° from the x-axis. The `IfcCurveSegment` trim starts 10 m from (1000,-800) and the trimmed segment has a lenght of 1956.796 m.

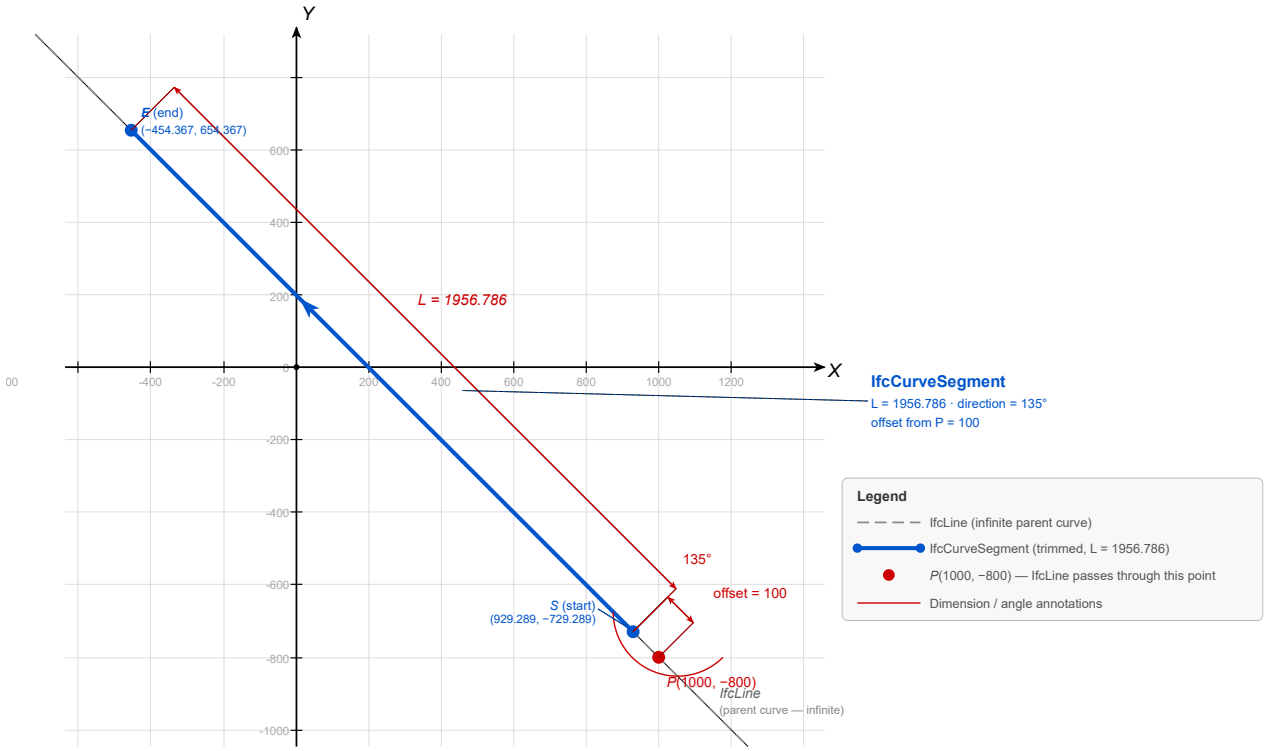

Figure: IfcLine with Trimmed IfcCurveSegment — Arbitrary Placement

Figure 2.3.2-2 IfcLine at arbitrary placement

Figure 2.3.2-1 represents a valid parent curve defiition and implementations must be prepared to properly interpret this geometry. This guide defines general algorithms to accomodate this geometry.

In practice, it is far easier to define the parent curve passing through point (0,0) and in the direction of the X-axis. The trimming can begin at the origin as shown in Figure 2.3.2-3.

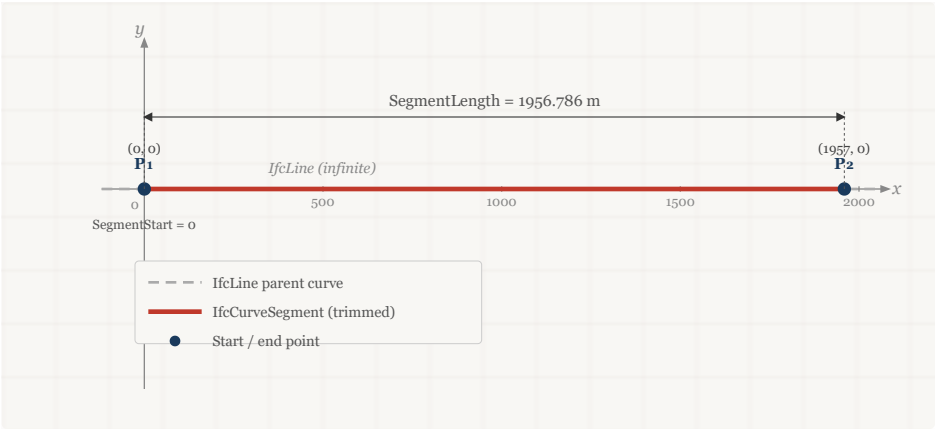

Figure 2.3.2-3 IfcLine parent curve at origin

The parent curve definition is:

```
#51=IFCCARTESIANPOINT((0.,0.));
#52=IFCDIRECTION((1.,0.));
#53=IFCVECTOR(#52,1.);
#54=IFCLINE(#51,#53);
```

`IfcCurveSegment` defines the portion of the line used and how it is placed and oriented in the X-Y plane.

`IfcCurveSegment.SegmentStart` defines where the parent curve trimming begins. It can be anywhere along the line. It is easiest to begin the trimming at the origin but could be anywhere. The `SegmentStart` attribute is 0.0 in this case.

`IfcCurveSegment.SegmentLength` defines where the parent curve trimming ends relative to `SegmentStart`. This is the length of the curve we want to trim. The `SegmentLength` attribute is 1956.785654 for this example.

The `SegmentStart` and `SegmentLength` attributes trims a portion of the `IfcLine` parent curve, which is oriented in the direction (1,0) with origin at (0,0).

The trimmed portion of the curve needs to be placed and oriented into the horizontal alignment. The tangent segment begins at (500,2500) and runs in the direction 5.70829654085293 radian.

From the segment direction,

$$dy = \sin(5.70829654085293) = -0.54374144087698$$

$$dx = \cos(5.70829654085293) = 0.839252789970355$$

The segment placement is

```
#31=IFCCARTESIANPOINT((500.,2500.));
#49=IFCDIRECTION((0.839252789970355,-0.54374144087698));
#50=IFCAXIS2PLACEMENT2D(#31,#49);
```

The `IfcCurveSegment` is be defined as:

```
#55=IFCCURVESEGMENT(.CONTSAMEGRADIENT.,#50,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(1956.785654),#54);
```

2.3.3 Compute Point on Curve

Compute the position matrix for a point 1500 m from the start of the curve segment.

Step 1 - Evaluate the parent curve at the trim start

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Step 2 - Form the normalization matrix M_N

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = I$.

Step 3 - Form the curve segment placement matrix M_{CSP}

From the `IfcCurveSegment.Placement`:

$$x = 500, y = 2500$$

$$dx = 0.839252789970355,\ dy = -0.54374144087698$$

$$M_{CSP} = \begin{bmatrix} 0.839252789970355 & 0.54374144087698 & 0 & 500 \\ -0.54374144087698 & 0.839252789970355 & 0 & 2500 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 - Evaluate and map each point

Evaluate the parent curve at $u = 100$

$$\lambda(u) = C + \left(\int_0^{u=L} \cos(\theta(t))dt\ x,\ \int_0^{u=L} \sin(\theta(t))dt\ y \right)$$

$$\theta(100) = \arctan\left(\frac{0}{1}\right) = 0$$

$$x = 0 + \cos(0) \int_0^{100} dt = 0 + 1(1500\ m - 0\ m) = 1500\ m$$

$$y = 0 + \sin(0)\int_0^{100} dt = 0 + 0(1500\ m - 0\ m) = 0\ m$$

Though it is much easier to use the following calculation

$$x(u) = p_x + u(dx)$$

$$y(u) = p_y + u(dy)$$

$$x = 0 + 1(1500) = 1500$$

$$y = 0 - 0(1500) = 0$$

$$M_{PC} = \begin{bmatrix} 1 & 0 & 0 & 1500 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The resulting position matrix is

$$M_h = M_{CSP} M_N M_{PC}$$

$$M_h = \begin{bmatrix} 0.839252789970355 & 0.54374144087698 & 0 & 500 \\ -0.54374144087698 & 0.839252789970355 & 0 & 2500 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} [I] \begin{bmatrix} 1 & 0 & 0 & 1500 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.83925279 & 0.54374114 & 0 & 1758.879185 \\ -0.54374114 & 0.83925279 & 0 & 1684.3878387 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.4 Circular Arc

Circular alignment curves are geometrically represented by an arc trimmed from an `IfcCircle` parent curve.

2.4.1 Parent Curve Parametric Equations

The curve tangent angle, curvature, and point on the curve are given by the following equations

$$\theta(s) = \frac{s}{R}$$

$$\kappa(s) = \frac{1}{R}$$

$$x(s) = \int \cos(\theta(s)) ds$$

$$y(s) = \int \sin(\theta(s))\ ds$$

2.4.2 Semantic Definition to Geometry Mapping

`IfcCircle` is defined by its center (`IfcCircle.Position`) and radius (`IfcCircle.Radius`).

Consider a horizontal curve segment that starts at (0,0), has a radius of 300 m, and an arc length of 100 m. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 300., 300., 100., $, .CIRCULARARC.);
```

The parent curve can be defined as follows:

```
#45 = IFCCIRCLE(#46, 300.);
#46 = IFCAXIS2PLACEMENT2D(#47, #48);
#47 = IFCCARTESIANPOINT((0., 300.));
#48 = IFCDIRECTION((0., -1.));
```

This is a circle centered at point (0,300) with the local X-axis aligned with the negative global Y-axis.

[todo: add a figure of the circle along with the local and global coordinate systems.]

The parent curve is placed such that a trim starting at 0.0 is at (0,0) and the tangent is in the direction (1,0).

The curve segment is defined by its placement at a segment trimmed from the parent curve starting at 0.0 for a length of 100.0 along the curve.

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
```

2.4.3 Compute Point on Curve

Compute the placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Evaluate the parent curve at the trim start

The trim begins where the local x-axis of the circle intersects the circumfrance. The circle is centered at (0,300) with radius = 300 and the local x-axis is in the direction of the global y-axis. This puts the trim start point at $x_0 = 0, y_0 = 0$ and the tangent direction (1,0) with $\theta_0 = 0$

Step 2 — Form the normalization matrix M_N

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = I$.

Step 3 — Form the curve segment placement matrix M_{CSP}

$M_{CSP} = I$

Step 4 — Evaluate and map each point

Compute point on parent curve at $u = 50$

$$\theta(s) = \frac{s}{R} = \frac{50}{300}$$

$$d_x = \cos(\theta(s)) = \cos(\frac{50}{300}) = 0.98614323$$

$$d_y = \sin(\theta(s)) = \sin(\frac{50}{300}) = 0.16589613$$

$$x(s) = \int \cos\left(\theta(s)\right) ds = \int_0^{50} \cos\left(\frac{s}{300}\right) ds = 300 \sin\left(\frac{50}{300}\right) - 300 \sin\left(\frac{0}{300}\right) = 49.76883981$$

$$y(s) = \int \sin\left(\theta(s)\right) ds = \int_0^{50} \sin\left(\frac{s}{300}\right) ds = -300 \cos\left(\frac{50}{300}\right) - \left(-300 \cos\left(\frac{0}{300}\right)\right) = 4.1570305$$

$$M_{PC} = \begin{bmatrix} 0.98614323 & -0.16589613 & 0 & 49.76883981 \\ 0.16589613 & 0.98614323 & 0 & 4.1570305 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The resulting position matrix is

$$M_h = M_{CSP} M_N M_{PC}$$

$$M_h = [I]\,[I] \begin{bmatrix} 0.98614323 & -0.16589613 & 0 & 49.76883981 \\ 0.16589613 & 0.98614323 & 0 & 4.1570305 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.98614323 & -0.16589613 & 0 & 49.76883981 \\ 0.16589613 & 0.98614323 & 0 & 4.1570305 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.5 Clothoid

Spiral transition curves are geometrically represented by a segment trimmed from one of the many spiral types in IFC. `IfcClothoid` geometry is used for clothoid curves.

2.5.1 Parent Curve Parametric Equations

The curve tangent angle, curvature, and point on the curve are given by the following equations

$$\theta(s) = \int \kappa(s) ds = \frac{\pi}{2} \frac{A}{|A|} s^2$$

$$\kappa(s) = \frac{A}{|A^3|} s$$

$$x(u) = A\sqrt{\pi} \int_0^u \cos\left(\frac{\pi}{2} \frac{A}{|A|} t^2\right) dt$$

$$y(u) = A\sqrt{\pi} \int_0^u \sin\left(\frac{\pi}{2} \frac{A}{|A|} t^2\right) dt$$

IFC provides a unit parametrization of clothoid curve.

$$u = \frac{s}{|A\sqrt{\pi}|}$$

with $-\infty < u < \infty$. When $\theta = \pi/2$, $u = 1.0$.

Care must be taken when evaluating clothoids because `IfcCurveSegment.SegmentStart` and `IfcCurveSegment.SegmentLength` are defined with arc length. The details are illustreated in the example calculations.

2.5.2 Semantic Definition to Geometry Mapping

Consider a horizontal clothoid segment that starts at (0,0) with a start direction of (1.0,0.0). The radii at the start and end of the segment are 300 and 1000, respectively. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 300., 1000., 100., $, .CLOTHOID.);
```

From the semantic definition, compute the clothoid constant, A

$$R_s = 300,\ R_e = 1000,\ L = 100$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{1000} - \frac{100}{300} = -0.23333$$

$$A = \frac{L}{\sqrt{|f|}} \frac{f}{|f|} = \frac{100}{\sqrt{|-0.23333|}} \frac{-0.23333}{|-0.23333|} = -207.0196678$$

The curve segment starts where the radius is 300. The distance from the origin where this radius occurs can be computed from the curvature equation. The curvature at the start is 1/300 and the clothoid constant is -207.0196678. Solve for distance from origin.

$$\kappa(s) = \frac{A}{|A^3|} s$$

$$\frac{1}{300} = \frac{-207.0196678}{|-207.0196678^3|} s$$

$$s = -142.8571429$$

Place the parent curve at (0,0) with a tangent direction of (1,0)

```
#45 = IFCCLOTHOID(#46, -207.019667802706);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

Place the curve segment at (0,0) with the tangent in the direction (1,0).

```
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
```

Define the curve segment

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(-142.857142857143), IFCLENGTHMEASURE(100.), #45);
```

2.5.3 Compute Point on Curve

Compute the placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Evaluate the parent curve at the trim start

Start by computing the point and curve tangent at the start of the parent curve trim.

Recall that the clothoid equations are in terms of a unit parameterization. Compute the parametric position on the curve.

$$u = \frac{-142.8571428}{\left|-207.0196678\sqrt{\pi}\right|} = -0.3893278$$

Compute the point on curve tangent direction

$$\theta_0(-0.3893278) = \frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}(-0.3893278)^2 = -0.238095237$$

and compute the point on the curve

$$x_0(-0.3893278) = -207.0196678\sqrt{\pi}\int_0^{-0.3893278}\cos\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}t^2\right)dt = -142.04941746210602$$

$$y_0(-0.3893278) = -207.0196678\sqrt{\pi}\int_0^{-0.3893278}\sin\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}t^2\right)dt = 11.292042785713347$$

Step 2 — Form the normalization matrix M_N

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -x_0\cos\theta_0 - y_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & -x_0\sin\theta_0 - y_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.971788979 & -0.235852028 & 0 & 140.705309648 \\ 0.235852028 & 0.971788979 & 0 & 22.529160405 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix. This is not true in all cases.

$$M_{CSP} = I$$

Step 4 — Evaluate and map each point

Compute point and curve tangent at 50 m from the start,

$$s = -142.8571428 + 50 = -92.8571428$$

$$u = \frac{-92.8571428}{\left|-207.0196678\sqrt{\pi}\right|} = -0.25306307$$

$$x(-0.25306307) = -207.0196678\sqrt{\pi}\int_0^{-0.25306307}\cos\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}s^2\right)ds = -92.763220844871512$$

$$y(-0.25306307) = -207.0196678\sqrt{\pi}\int_0^{-0.25306307}\sin\left(\frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}s^2\right)ds = 3.1114126254443812$$

$$\theta(-0.25306307) = \frac{\pi}{2}\frac{-207.0196678}{|-207.0196678|}(-0.25306307)^2 = -0.100595238$$

$$dx = \cos(-0.100595238) = 0.994944564$$

$$dy = \sin(-0.100595238) = -0.100425663$$

In matrix form

$$M_{PC} = \begin{bmatrix} 0.994944564 & 0.100425663 & 0 & -92.763220844871512 \\ -0.100425663 & 0.994944564 & 0 & 3.1114126254443812 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP} M_N M_{PC} = I \begin{bmatrix} 0.971788979 & -0.235852028 & 0 & 140.705309648 \\ 0.235852028 & 0.971788979 & 0 & 22.529160405 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.994944564 & 0.100425663 & 0 & -92.763220844871512 \\ -0.100425663 & 0.994944564 & 0 & 3.1114126254443812 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = \begin{bmatrix} 0.99056175921247780 & -0.13706714116038599 & 0 & 49.825200930152405 \\ 0.13706714116038599 & 0.99056175921247780 & 0 & 3.6744032279728032 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.6 Cubic Transition Curve

`IfcPolynomalCurve` is the geometric type for cubic transition spirals.

2.6.1 Parent Curve Parametric Equations

The approach for calculating the coordinates of a horizontal cubic curve is unique compared to the other curve types. For a horizontal alignment, the coordinate point and curve tangent angle is generally defined by parametric equations where the parameter is the distance along the curve. A cubic curve is represented with the `IfcPolynomialCurve` parent curve type. When `IfcPolynomialCurve` is used as a cubic transition curve, the x coefficients are [0,1] and the y coefficients are [0,0,0, A_3] resulting in a function of the form $y(x) = A_3 x^3$. See 4.2.2.1.5 Cubic Transition Segment.

The distance along a curve is defined by

$$d = \int \sqrt{(y'(x))^2 + 1}\, dx$$

$$y'(x) = 3A_3 x^2$$

$$d = \int \sqrt{9A_3^2 x^4 + 1}\, dx$$

This equation is solved for x and then y can be computed. This can be accomplished with numerical methods.

1. For a distance u along the curve, find x for $d - u = 0$, within a tolerance consistent with the modeled elements. See Section 11 for a discussion on tolerances.
2. Compute $y(x) = A_3 x^3$
3. Compute curve tangent slope as $\frac{dy}{dx} = 3A_3 x^2$

2.6.2 Semantic Definition to Geometry Mapping

Consider a horizontal cubic transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $,.CUBIC.);
```

Compute the polynomial curve coefficients

$$A_0 = A_1 = A_2 = 0$$

$$A_3 = \frac{1}{6R_eL} - \frac{1}{6R_sL} = \frac{1}{6(300\ m)(100\ m)} - \frac{1}{6(\infty)(100\ m)} = 5.55555 \cdot 10^{-6}\ m^{-2}$$

The geometric representation is

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCPOLYNOMIALCURVE(#46, (0., 1.), (0., 0., 0., 5.55555555555556E-6), $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

—- :warning:

There is a flaw in the IFC Specification. IfcAlignmentHorizontalSegment indicates the cubic formula is $y = \frac{x^3}{6RL}$ which means the `IfcPolynomialCurve.CoefficentY[3]` attribute must have unit of $Length^{-2}$. This is a direct contradiction to IfcPolynomialCurve which clearly states the coefficent are real values (i.e. scalar values) with `IfcReal`.

Why is this a problem? The calculation of a point on the polynomal curve is different depending on how the coefficients are defined.

The parametric form of the polynomial curve is $\lambda(u) = (x(u), y(u), z(u))$. The polynomial curve represents real geometry so the resulting values, x, y, and z must have units of $Length$.

When u is parametrized as a Length measure, as required by IfcCurveSegment, the parametric terms of the polynomal equation are:

$$x(u) = \sum_{i=0}^{l} a_i u^i$$

$$y(u) = \sum_{j=0}^{m} b_j u^j$$

$$z(u) = \sum_{k=0}^{n} c_k u^k$$

For x, y, and z to have values in $Length$ measure, the implicit unit of measure of the coefficients must be:

$Length^{(1-i)}$ for a_i

$Length^{(1-j)}$ for b_j

$Length^{(1-k)}$ for c_k

When u is parametrized as a scalar, as required by IfcPolynomialCurve, the parametric terms of the polynomial equation are:

$$x(u) = L\sum_{i=0}^{l} a_i u^i$$

$$y(u) = L\sum_{j=0}^{m} b_j u^j$$

$$z(u) = L\sum_{k=0}^{n} c_k u^k$$

The parameter u and the coefficients are scalar so they must be multipled with the curve length L to result values of Length.

The equations for x, y, and z differ depending on the parameterization.

Concept Template 4.2.2.1.5 Cubic Transition Segment and IfcAlignmentHorizontalSegment provide clear direction (not withstanding typographical errors) that `IfcPolynomalCurve` is to be evaluated as $y = CoefficientsY[3]x^3$ which dictates that $CoefficientsY[3]$ must be in units of $Length^{-2}$ and must be encoded in the `IfcPolynomialCurve.CoefficientY` attribute as an `IfcReal`.

An offical Implementation Agreement doesn't seem to exist, however, this interpetation is consistent with several different discussions on GitHub.

:warning:

—-

2.6.3 Evaluate Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Evaluate the parent curve at the trim start

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Step 2 — Form the normalization matrix M_N

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = I$.

Step 3 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = I$$

Step 4 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

From the `IfcPolynomalCurve` definition,

X Coefficients = ($0m^1$, $1m^0$)

Y Coefficients = ($0m^1$, $0m^0$, $0m^{-1}$, $5.55555 \cdot 10^{-6}m^{-2}$)

$$y(x) = \left(5.55555 \cdot 10^{-6}m^{-2}\right)x^3$$

Find x such that

$$|d - u| = \left| \int_0^x \left(\sqrt{(y'(x))^2 + 1} \right) dx - u \right| < 10^{-4} \approx 0$$

$$y'(x) = 3\left(5.55555 \cdot 10^{-6}m^{-2}\right)x^2$$

Solve numerically

$$x = 99.72593255m$$

Check solution

$$d = \int_0^{99.72593255\ m} \sqrt{\left(3 \cdot (5.55555 \cdot 10^{-6}m^{-2})(x\ m)^2\right)^2 + 1} dx = 100\ m$$

Compute y

$$y(x) = (5.55555 \cdot 10^{-6}m^{-2})(99.72593255\ m)^3 = 5.5100\ m$$

The tangent vector at $u = 100\ m$ along the curve is

$$y'(99.72593255\ m) = 3\left(5.55555 \cdot 10^{-6}m^{-2}\right)(99.72593255\ m)^2 = 0.165753$$

Normalizing the tangent vector

$$dx = \frac{1}{\sqrt{1^2 + 0.615753^2}} = 0.986539$$

$$dy = \frac{0.165753}{\sqrt{1^2 + 0.615753^2}} = 0.1635219$$

The resulting matrix is

$$M_{PC} = \begin{bmatrix} 0.986539 & -0.1635219 & 0 & 99.72593255 \\ 0.1635219 & 0.986539 & 0 & 5.5100 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP} M_N M_{PC} = I\,I \begin{bmatrix} 0.986539 & -0.1635219 & 0 & 99.72593255 \\ 0.1635219 & 0.986539 & 0 & 5.5100 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.986539 & -0.1635219 & 0 & 99.72593255 \\ 0.1635219 & 0.986539 & 0 & 5.5100 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.7 Helmert Transition Curve

The Helmert curve is unique in that it is semantically defined as a single layout segment (`IfcAlignmentHorizontalSegment`) but geometrically defined with two curve segments (`IfcCurveSegment`), each representing half of the transition curve. The parent curve for a helmert transition curve is `IfcSecondOrderPolynomialSpiral`.

2.7.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(t) = \frac{t^3}{3A_2^3} + \frac{A_1}{2\left|A_1^3\right|}t^2 + \frac{t}{A_0}$$

$$\kappa(t) = \frac{1}{A_2^3}t^2 + \frac{A_1}{\left|A_1^3\right|}t + \frac{1}{A_0}$$

2.7.2 Semantic Definition to Geometry Mapping

Consider a horizontal Helmert transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $, .HELMERTCURVE.);
```

Compute the curve parameters

$$L = 100\ m$$

$$R_s = \infty$$

$$R_e = 300\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

First Half

$$a_0 = \frac{L}{R_s},\ A_0 = \frac{L}{|a_0|}\ \frac{a_0}{|a_0|}$$

$$a_1 = 0,\ A_1 = 0$$

$$a_2 = 2f,\ A_2 = \frac{L}{\sqrt[3]{|a_2|}}\ \frac{a_2}{|a_2|}$$

$$a_0 = \frac{100}{\infty} = 0,\ A_0 = 0$$

$$A_1 = 0$$

$$a_2 = 2(0.33333) = 0.66667,\ A_2 = \frac{100\ m}{\sqrt[3]{|0.66667|}}\ \frac{0.66667}{|0.66667|} = 114.4714255\ m$$

Second Half

$$a_0 = -f + \frac{L}{R_s},\ A_0 = \frac{L}{|a_0|}\ \frac{a_0}{|a_0|}$$

$$a_1 = 4f,\ A_1 = \frac{L}{\sqrt{|a_1|}}\ \frac{a_1}{|a_1|}$$

$$a_2 = -2f,\ A_2 = \frac{L}{\sqrt[3]{|a_2|}}\ \frac{a_2}{|a_2|}$$

$$a_0 = -0.33333 + \frac{100}{\infty} = -0.33333,\ A_0 = \frac{100\ m}{|-0.33333|}\frac{-0.33333}{|-0.33333|} = -300\ m$$

$$a_1 = 4(0.33333) = 1.33333,\ A_1 = \frac{100\ m}{\sqrt{|1.33333|}}\frac{1.33333}{|1.33333|} = 86.602540378\ m$$

$$a_2 = -2(0.33333) = -0.66667,\ A_2 = \frac{100\ m}{\sqrt[3]{|-0.66667|}}\frac{-0.66667}{|-0.66667|} = -114.4714255\ m$$

End point and end slope of first half is the placement of the second half.

$$\theta(t) = \frac{t^3}{3A_2^3} + \frac{A_1}{2\left|A_1^3\right|}t^2 + \frac{t}{A_0} = \frac{t^3}{3(114.4714255)^3}$$

$$x_1 = \int_0^{L/2} \cos\theta(t)\ dt = 49.99724\ m$$

$$y_1 = \int_0^{L/2} \sin\theta(t)\ dt = 0.347204\ m$$

$$\theta_1\left(\frac{L}{2}\right) = \frac{(50\ m)^3}{3(114.4714255\ m)^3} = 0.0277777777$$

$$dx = \cos(0.027777777) = 0.999614$$

$$dy = \sin(0.027777777) = 0.0277742$$

Start point of second parent curve

$$\theta(t) = \frac{t^3}{3A_2^3} + \frac{A_1}{2\left|A_1^3\right|}t^2 + \frac{t}{A_0} = \frac{t^3}{3(-114.4714255\ m)^3} + \frac{86.602540378\ m}{2\left|(86.602540378\ m)^3\right|}t^2 + \frac{t}{-300\ m}$$

$$x_2 = \int_0^{L/2} \cos\theta(t)\ dt = 49.9664\ m$$

$$y_2 = \int_0^{L/2} \sin\theta(t)\ dt = -1.73556\ m$$

$$\theta_2\left(\frac{L}{2} = 50\ m\right) = \frac{(50\ m)^3}{3(-114.4714255\ m)^3} + \frac{86.602540378\ m}{2\left|(86.602540378\ m)^3\right|}(50\ m)^2 + \frac{50\ m}{-300\ m} = -0.0277777$$

The end point of the first parent curve and the start point of the second parent curve are offset so a rotation and translation for the placement of the second parent curve is needed.

$$\theta_p = \theta_1 - \theta_2 = 0.0277777777 - (-0.0277777777) = 0.05555555$$

The `RefDirection` of the parent curve `Position` (`#55`) is $(\cos\theta_p,\ \sin\theta_p)$:

$$\cos\theta_p = \cos(0.05555555) = 0.998457$$

$$\sin\theta_p = \sin(0.05555555) = 0.055526$$

$$x_p = x_1 - x_2\cos\theta_p + y_2\sin\theta_p$$

$$y_p = y_1 - x_2\sin\theta_p - y_2\cos\theta_p$$

The `Location` of the parent curve `Position` (`#54`) is $(x_p,\ y_p)$:

$$x_p = 49.9972 - 49.9664\cos(0.0555555) - 1.73556\sin(0.0555555) = 0.011503\ m$$

$$y_p = 0.347204 - 49.9664\sin(0.055555) + 1.73556\cos(0.0555555) = -0.6943693\ m$$

The geometric representation of the first half is

```
#36 = IFCCURVESEGMENT(.CONTSAMEGRADIENTSAMECURVATURE., #42, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(50.), #45);
#37 = IFCLOCALPLACEMENT($, #38);
#38 = IFCAXIS2PLACEMENT3D(#39, $, $);
#39 = IFCCARTESIANPOINT((0., 0., 0.));
#45 = IFCSECONDORDERPOLYNOMIALSPIRAL(#46, 114.471424255333, $, $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

> Note: when a polynomial coefficient is zero, the correspond term is unused and coded as `$` in the IFC entity

Two-level placement for the second half

`IfcCurveSegment` evaluation normalizes the parent curve at `SegmentStart`, then applies the curve segment `Placement`. For the second half, `SegmentStart = L/2`, so the IFC machinery uses the parent curve's value at $t = L/2$ as the normalization origin. The second half parent curve's own `Position` attribute is therefore set so that

$$\mathrm{ParentCurve}_2\left(\frac{L}{2}\right) = (x_1,\ y_1,\ \theta_1)$$

— exactly the endpoint of the first half. With this alignment the normalization cancels the offset, and the curve segment `Placement` supplies only the world-space join position.

Level	Attribute	Purpose
`IfcSecondOrderPolynomialSpiral.Position`	$(x_p,\ y_p,\ \theta_p)$	Shift/rotate the raw spiral so its value at $t = L/2$ equals (x_1, y_1, θ_1)
`IfcCurveSegment.Placement`	World-space join point	Place the normalized second-half curve at the correct position in the composite curve

Table 2.7.2-1 — Two-level placement for the second Helmert half

Finding the parent curve Position

Evaluate the raw second half spiral (with `Position` at origin) at $t = L/2$ to obtain (x_2, y_2, θ_2). The rigid-body transform that maps this to (x_1, y_1, θ_1) is

$$\theta_p = \theta_1 - \theta_2$$
$$x_p = x_1 - x_2 \cos\theta_p + y_2 \sin\theta_p$$
$$y_p = y_1 - x_2 \sin\theta_p - y_2 \cos\theta_p$$

Finding the curve segment Placement

The `Placement` is the world-space position and direction at the join point, obtained by rotating (x_1, y_1) by the overall segment start direction θ_s and translating by the start point (x_s, y_s):

$$x_{join} = x_s + x_1 \cos\theta_s - y_1 \sin\theta_s$$
$$y_{join} = y_s + x_1 \sin\theta_s + y_1 \cos\theta_s$$
$$\theta_{join} = \theta_s + \theta_1$$

For the example with $\theta_s = 0$:

$$x_{join} = 0.0 + 49.99724 \cos(0.0) - 0.347204 \sin(0.0) = 49.99724 \text{ m}$$
$$y_{join} = 0.0 + 49.99724 \sin(0.0) + 0.347204 \cos(0.0) = 0.347204 \text{ m}$$
$$\theta_{join} = 0.0 + 0.02777777 = 0.02777777 \text{ rad}$$

The `RefDirection` of the curve segment `Placement` (`#51`) is $(\cos\theta_{join},\ \sin\theta_{join})$:

$$\cos\theta_{join} = \cos(0.02777777) = 0.999614$$
$$\sin\theta_{join} = \sin(0.02777777) = 0.027774$$

The geometric representation of the second half

```
#48 = IFCCURVESEGMENT(.CONTINUOUS., #49, IFCLENGTHMEASURE(50.), IFCLENGTHMEASURE(50.), #52);
#49 = IFCAXIS2PLACEMENT2D(#50, #51);
#50 = IFCCARTESIANPOINT((49.9972443634885, 0.347204361427475));
#51 = IFCDIRECTION((0.999614222337484, 0.027774205670508 8));
#52 = IFCSECONDORDERPOLYNOMIALSPIRAL(#53, -114.471424255333, 86.6025403784439, -300.);
#53 = IFCAXIS2PLACEMENT2D(#54, #55);
#54 = IFCCARTESIANPOINT((0.0115725277555399, -0.694297741112199));
#55 = IFCDIRECTION((0.998457186998745, 0.055526982004733 9));
```

2.7.3 Compute Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$ (the end of the full Helmert segment).

The point falls in the second half ($50 < u \leq 100$). The parent curve parameter is $t = \text{SegmentStart} + (u - 50) = 50 + 50 = 100$.

Step 1 — Evaluate the second half parent curve at SegmentStart

The second half parent curve has `Position` at $(x_p, y_p) = (0.011503,\ -0.694370)$ with direction $\theta_p = 0.055556$ rad. This was chosen so that the parent curve at $t = L/2 = 50$ yields $(x_1, y_1, \theta_1) = (49.9972,\ 0.347204,\ 0.027778)$ — the endpoint of the first half.

Step 2 — Form the normalization matrix M_N

M_N maps (x_1, y_1, θ_1) to the origin with the x-axis tangent direction:

$$M_N = \begin{bmatrix} \cos\theta_1 & \sin\theta_1 & 0 & -x_1\cos\theta_1 - y_1\sin\theta_1 \\ -\sin\theta_1 & \cos\theta_1 & 0 & x_1\sin\theta_1 - y_1\cos\theta_1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_N = \begin{bmatrix} 0.999614 & 0.027774 & 0 & -50.0 \\ -0.027774 & 0.999614 & 0 & 1.734 \times 10^{-4} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

The curve segment `Placement` for the second half is at $(x_{join}, y_{join}) = (49.9972,\ 0.347204)$ with direction $\theta_{join} = 0.027778$ rad:

$$M_{CSP} = \begin{bmatrix} 0.999614 & -0.027774 & 0 & 49.9972 \\ 0.027774 & 0.999614 & 0 & 0.347204 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Evaluate and map the point at $t = 100$

Evaluate the second half parent curve at $t = 100$. The raw spiral (before applying `Position`) uses $(A_{0,2},\ A_{1,2},\ A_{2,2}) = (-300,\ 86.6025,\ -114.4714)$:

$$\theta_{raw}(t) = \frac{t^3}{3(-114.4714)^3} + \frac{86.6025}{2\left|86.6025^3\right|}t^2 + \frac{t}{-300}$$

$$\theta_{raw}(100) = -0.22222 + 0.66666 - 0.33333 = 0.11111 \text{ rad}$$

$$x_{raw} = \int_0^{100} \cos\theta_{raw}(t)\,dt = 97.9866 \text{ m}$$

$$y_{raw} = \int_0^{100} \sin\theta_{raw}(t)\,dt = 6.0948 \text{ m}$$

Apply the parent curve `Position` (x_p, y_p, θ_p):

$$x_{pos} = x_p + x_{raw}\cos\theta_p - y_{raw}\sin\theta_p = 0.011503 + 97.9866(0.998457) - 6.0948(0.055527) = 97.9201 \text{ m}$$

$$y_{pos} = y_p + x_{raw}\sin\theta_p + y_{raw}\cos\theta_p = -0.694370 + 97.9866(0.055527) + 6.0948(0.998457) = 10.7408 \text{ m}$$

$$\theta_{pos}(100) = \theta_p + \theta_{raw}(100) = 0.055556 + 0.11111 = 0.16\overline{6}\ \text{rad}$$

Note: $\theta_{pos}(100) = \frac{1}{6} = \frac{L}{2R_e} = \frac{100}{2\times 300}$, confirming the total tangent angle change matches a Clothoid with the same endpoints.

Form M_{PC} from the parent curve point at $t = 100$:

$$M_{PC} = \begin{bmatrix} \cos\theta_{pos} & -\sin\theta_{pos} & 0 & x_{pos} \\ \sin\theta_{pos} & \cos\theta_{pos} & 0 & y_{pos} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.98615 & -0.16590 & 0 & 97.9201 \\ 0.16590 & 0.98615 & 0 & 10.7408 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply normalization and curve segment placement:

$$M_h = M_{CSP}\, M_N\, M_{PC}$$

$$M_N M_{PC} = \begin{bmatrix} 0.98615 & -0.16590 & 0 & 47.6227 \\ 0.16590 & 0.98615 & 0 & 10.3938 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_h = M_{CSP}(M_N M_{PC}) = \begin{bmatrix} 0.97260 & -0.23256 & 0 & 99.2496 \\ 0.23256 & 0.97260 & 0 & 9.7486 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The end point of the Helmert segment at $u = 100$:

$$x = 99.2496\ \text{m}, \quad y = 9.7486\ \text{m}, \quad \theta = \frac{1}{6}\ \text{rad}$$

2.8 Bloss Transition Curve

`IfcThirdOrderPolynomialSpiral` is the geometric type for Bloss transition spirals.

2.8.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(t) = \frac{A_3}{4\left|A_3^5\right|}t^4 + \frac{1}{3A_2^3}t^3 + \frac{A_1}{2\left|A_1^3\right|}t^2 + \frac{t}{A_0}$$

$$\kappa(t) = \frac{A_3}{\left|A_3^5\right|}t^3 + \frac{t^2}{A_2^3} + \frac{A_1}{2\left|A_1^3\right|}t + \frac{1}{A_0}$$

2.8.2 Semantic Definition to Geometry Mapping

Consider a horizontal Bloss transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $, .BLOSSCURVE.);
```

Compute the polynomial spiral parameters

$$L = 100\ m$$

$$R_s = \infty$$

$$R_e = 300\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

$$a_0 = \frac{L}{R_s} = \frac{100}{\infty} = 0$$

$$A_0 = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = 0$$

$$A_1 = 0$$

$$a_2 = 3f = 3(0.33333) = 1$$

$$A_2 = \frac{L}{\sqrt[3]{|a_2|}}\frac{a_2}{|a_2|} = \frac{100}{\sqrt[3]{|1|}}\frac{1}{|1|} = 100\ m$$

$$a_3 = -2f = -2(0.33333) = -0.66667$$

$$A_3 = \frac{L}{\sqrt[4]{|a_3|}}\frac{a_3}{|a_3|} = \frac{100\ m}{\sqrt[4]{|-0.66667|}}\frac{-0.66667}{|-0.66667|} = -110.668192\ m$$

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCTHIRDORDERPOLYNOMIALSPIRAL(#46, -110.668191970032, 100., $, $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

2.8.3 Evaluate Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Evaluate the parent curve at the trim start

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Step 2 — Form the normalization matrix M_N

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = I$.

Step 3 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = I$$

Step 4 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$\theta(s) = \frac{-110.668192}{4\left|-110.668192^5\right|}s^4 + \frac{1}{3(100)^3}s^3$$

$$x = \int_0^{100} \cos\theta(s)\ ds = 99.7486$$

$$y = \int_0^{100} \sin\theta(s)\ ds = 4.98981$$

$$\theta(100) = \frac{-110.668192}{4\left|-110.668192^5\right|}(100)^4 + \frac{1}{3(100)^3}(100)^3 = \frac{1}{6} = 0.1666667$$

$$dx = \cos\left(\frac{1}{6}\right) = 0.98614$$

$$dy = \sin\left(\frac{1}{6}\right) = 0.16589$$

In matrix form

$$M_{PC} = \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.7486 \\ 0.16589 & 0.98614 & 0 & 4.98981 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP} M_N M_{PC} = I\,I \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.7486 \\ 0.16589 & 0.98614 & 0 & 4.98981 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.7486 \\ 0.16589 & 0.98614 & 0 & 4.98981 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.9 Cosine Transition Curve

`IfcCosineSpiral` is the geometric type for cosine transition spirals.

2.9.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\kappa(t) = \frac{1}{A_0} + \frac{1}{A_1}\cos\left(\frac{\pi}{L}t\right)$$

$$\theta(t) = \frac{1}{A_0}t + \frac{L}{\pi A_1}\sin\left(\frac{\pi}{L}t\right)$$

2.9.2 Semantic Definition to Geometry Mapping

Consider a horizontal cosine transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $, .COSINECURVE.);
```

Compute the cosine spiral parameters

$$R_s = \infty,\ R_e = 300\ m,\ L = 100\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

Constant Term:

$$a_0 = 0.5f + \frac{L}{R_s} = 0.5(0.33333) + \frac{100}{\infty} = 0.16667$$

$$A_0 = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = \frac{100\ m}{|0.16667|}\ \frac{0.16667}{|0.16667|} = 600\ m$$

Cosine Term:

$$a_1 = -0.5f = -0.5(0.33333) = -0.16667$$

$$A_1 = \frac{L}{|a_1|}\frac{a_1}{|a_1|} = \frac{100}{|-0.16667|}\ \frac{-0.16667}{|-0.16667|} = -600\ m$$

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCCOSINESPIRAL(#46, -600., 600.);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

2.9.3 Evaluate Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Evaluate the parent curve at the trim start

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Step 2 — Form the normalization matrix M_N

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = I$.

Step 3 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = I$$

Step 4 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$\theta(t) = \frac{1}{600}t - \frac{100}{600\pi}\sin\left(\frac{\pi}{100}t\right)$$

$$x = \int_0^{100} \cos\theta(t)\, dt = 99.7485\ m$$

$$y = \int_0^{100} \sin\theta(t)\, dt = 4.9458\ m$$

$$\theta(100) = \frac{1}{600}100 - \frac{100}{600\pi}\sin\left(\frac{\pi}{100}100\right) = \frac{1}{6} = 0.1666667$$

$$dx = \cos\left(\frac{1}{6}\right) = 0.98614$$

$$dy = \sin\left(\frac{1}{6}\right) = 0.16589$$

In matrix form

$$M_{PC} = \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.7485 \\ 0.16589 & 0.98614 & 0 & 4.9458 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP}M_N M_{PC} = I\, I \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.7485 \\ 0.16589 & 0.98614 & 0 & 4.9458 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.7485 \\ 0.16589 & 0.98614 & 0 & 4.9458 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.10 Sine Transition Curve

`IfcSineSpiral` is the geometric type for sine transition spirals.

2.10.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\kappa(t) = \frac{1}{A_0} + \frac{A_1}{|A_1|}\left(\frac{1}{A_1}\right)^2 t + \frac{1}{A_2}\sin\left(\frac{2\pi}{L}t\right)$$

$$\theta(t) = \frac{1}{A_0}t + \frac{1}{2}\left(\frac{A_1}{|A_1|}\right)\left(\frac{t}{A_1}\right)^2 - \frac{L}{2\pi A_2}\left(\cos\left(\frac{2\pi}{L}t\right) - 1\right)$$

$$x = \int_0^L \cos\theta(t)\, dt$$

$$y = \int_0^L \sin\theta(t)\, dt$$

2.10.2 Semantic Definition to Geometry Mapping

Consider a horizontal sine transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length of the segment is 100. The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., $, .SINECURVE.);
```

Compute the sine spiral parameters

$$R_s = \infty,\ R_e = 300\ m,\ L = 100\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

Constant Term:

$$a_0 = \frac{L}{R_s} = \frac{100}{\infty} = 0$$

$$A_o = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = 0$$

Linear Term:

$$a_1 = f = 0.33333$$

$$A_1 = \frac{L}{\sqrt{|a_1|}}\frac{a_1}{|a_1|} = \frac{100}{\sqrt{|0.33333|}}\frac{0.33333}{|0.33333|} = 173.2050808\ m$$

Sine Term:

$$a_2 = -\frac{1}{2\pi}f = -\frac{1}{2\pi}(0.33333) = -0.053051647$$

$$A_2 = \frac{L}{|a_2|}\frac{a_2}{|a_2|} = \frac{100}{|-0.053051647|}\frac{-0.053051647}{|-0.053051647|} = -1884.955592\ m$$

```
#36 = IFCCURVESEGMENT(.CONTINUOUS., #42, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #45);
#42 = IFCAXIS2PLACEMENT2D(#43, #44);
#43 = IFCCARTESIANPOINT((0., 0.));
#44 = IFCDIRECTION((1., 0.));
#45 = IFCSINESPIRAL(#46, -1884.95559215388, 173.205080756888, $);
#46 = IFCAXIS2PLACEMENT2D(#47, $);
#47 = IFCCARTESIANPOINT((0., 0.));
```

2.10.3 Evaluate Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Evaluate the parent curve at the trim start

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Step 2 — Form the normalization matrix M_N

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = I$.

Step 3 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = I$$

Step 4 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$\theta(t) = \frac{1}{2}\left(\frac{t}{173.2050808}\right)^2 + \frac{100}{2\pi(1884.955592)}\left(\cos\left(\frac{2\pi}{100}t\right) - 1\right)$$

$$x = \int_0^{100} \cos\theta(t)\, dt = 99.757\ m$$

$$y = \int_0^{100} \sin\theta(t)\, dt = 4.70132\ m$$

$$\theta(100) = \frac{1}{2}\left(\frac{100}{173.2050808}\right)^2 + \frac{100}{2\pi(1884.955592)}\left(\cos\left(\frac{2\pi}{100}100\right) - 1\right) = \frac{1}{6} = 0.1666667$$

$$dx = \cos\left(\frac{1}{6}\right) = 0.98614$$

$$dy = \sin\left(\frac{1}{6}\right) = 0.16589$$

In matrix form

$$M_{PC} = \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.757 \\ 0.16589 & 0.98614 & 0 & 4.70132 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP} M_N M_{PC} = I\, I \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.757 \\ 0.16589 & 0.98614 & 0 & 4.70132 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.98614 & -0.16589 & 0 & 99.757 \\ 0.16589 & 0.98614 & 0 & 4.70132 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.11 Viennese Transition Curve

[todo: need a description of a viennese bend and its application in railroad. also need to note the unique feature that the cant parameters factor into the horizontal geometry. state the parent curve type in this description]

Parent curve type: `IfcSeventhOrderPolynomialSpiral`

2.11.1 Parent Curve Parametric Equations

The curve tangent angle and curvature are given by the following equations

$$\theta(s) = \frac{A_7}{8\left|A_7^9\right|}s^8 + \frac{1}{7A_6^7}s^7 + \frac{A_5}{6\left|A_5^7\right|}s^6 + \frac{1}{5A_4^5}s^5 + \frac{A_3}{4\left|A_3^5\right|}s^4 + \frac{1}{3A_2^3}s^3 + \frac{A_1}{2\left|A_1^3\right|}s^2 + \frac{1}{A_0}s$$

$$\kappa(s) = \frac{A_7}{\left|A_7^9\right|}s^7 + \frac{1}{A_6^7}s^6 + \frac{A_5}{\left|A_5^7\right|}s^5 + \frac{1}{A_4^5}s^4 + \frac{A_3}{\left|A_3^5\right|}s^3 + \frac{1}{A_2^3}s^2 + \frac{A_1}{\left|A_1^3\right|}s + \frac{1}{A_0}$$

2.11.2 Semantic Definition to Geometry Mapping

Consider a horizontal Viennese Bend transition curve segment that starts at (0,0) with a start direction of 0.0. The radius at the start is infinite and the radius at the end is 300. The arc length is 100. The Gravity Center Line Height is 1.8 (this optional parameter is required for the Viennese Bend). The semantic definition is

```
#28 = IFCCARTESIANPOINT((0., 0.));
#29 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #28, 0., 0., 300., 100., 1.8, .VIENNESEBEND.);
```

Viennese Bend transition curves are unique in that the horizontal geometry of the curve depends on the cant. For the example, the cant segment horizontal length is 100, the start cant is 0.0 and the end cant is 0 at the left rail and 0.1 at the right rail. The semantic definition is

```
#64 = IFCALIGNMENTCANTSEGMENT($, $, 0., 100., 0., 0., 0., 0.1, .VIENNESEBEND.);
```

h_{cg} = Gravity Center Line Height = `IfcAlignmentHorizontalSegment.GravityCenterLineHeight`

d_{rh} = Rail Head Distance = `IfcAlignmentCant.RaliHeadDistance`

D_{sl} = `IfcAlignmentCantSegment.StartCantLeft`

D_{el} = `IfcAlignmentCantSegment.EndCantLeft`

D_{sr} = `IfcAlignmentCantSegment.StartCantRight`

D_{er} = `IfcAlignmentCantSegment.EndCantRight`

$\theta_s = \frac{(D_{sr}-D_{sl})}{d_{rh}}$, Start Cant Angle

$\theta_e = \frac{(D_{er}-D_{el})}{d_{rh}}$, End Cant Angle

$cf = -420.\left(\frac{h_{cg}}{L}\right)(\theta_e - \theta_s)$, Cant Factor

Compute the polynomial spiral parameters

$$R_s = \infty,\ R_e = 300\ m,\ L = 100\ m$$

$$f = \frac{L}{R_e} - \frac{L}{R_s} = \frac{100}{300} - \frac{100}{\infty} = 0.33333$$

$$\theta_s = \frac{(0. - 0.)}{1.5} = 0.0$$

$$\theta_e = \frac{0.1 - 0.0}{1.5} = 0.066667$$

$$cf = -420\left(\frac{1.8}{100}\right)(0.066667 - 0.) = -0.504$$

Constant term

$$a_0 = \frac{L}{R_s} = \frac{100}{\infty} = 0$$

$$A_0 = \frac{L}{|a_0|}\frac{a_0}{|a_0|} = 0$$

Linear term

$$a_1 = 0$$

$$A_1 = 0$$

Quadratic term

$$a_2 = cf = -0.504$$

$$A_2 = \frac{L}{\sqrt[3]{|a_2|}}\frac{a_2}{|a_2|} = \frac{100}{\sqrt[3]{|-0.504|}}\frac{-0.504}{|-0.504|} = -125.6579069\ m$$

Cubic term

$$a_3 = -4cf = -4(-0.504) = 2.016$$

$$A_3 = \frac{L}{\sqrt[4]{|a_3|}}\frac{a_3}{|a_3|} = \frac{100}{\sqrt[4]{|2.016|}}\frac{2.016}{|2.016|} = 83.92229813\ m$$

Quartic term

$$a_4 = 5cf + 35f = 5(-0.504) + 35(0.33333) = 9.1466655$$

$$A_4 = \frac{L}{\sqrt[5]{|a_4|}} \frac{a_4}{|a_4|} = \frac{100}{\sqrt[5]{|9.1466655|}} \frac{9.1466655}{|9.1466655|} = 64.231406\ m$$

Quintic term

$$a_5 = -2cf - 84f = -2(-0.504) - 84(0.33333) = -26.9919999$$

$$A_5 = \frac{L}{\sqrt[6]{|a_5|}} \frac{a_5}{|a_5|} = \frac{100}{\sqrt[6]{|-26.9919999|}} \frac{-26.9919999}{|-26.9919999|} = -57.7378785\ m$$

Sextic term

$$a_6 = 70f = 70(0.33333) = 23.33333333$$

$$A_6 = \frac{L}{\sqrt[7]{|a_6|}} \frac{a_6}{|a_6|} = \frac{100}{\sqrt[7]{|23.33333333|}} \frac{23.33333333}{|23.33333333|} = 63.76388134\ m$$

Septic term

$$a_7 = -20f = -20(0.33333) = -6.66666666$$

$$A_7 = \frac{L}{\sqrt[8]{|a_7|}} \frac{a_7}{|a_7|} = \frac{100}{\sqrt[8]{|-6.66666666|}} \frac{-6.66666666}{|-6.66666666|} = -78.8880838\ m$$

```
#66 = IFCCURVESEGMENT(.CONTINUOUS., #72, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #75);
#72 = IFCAXIS2PLACEMENT2D(#73, #74);
#73 = IFCCARTESIANPOINT((0., 0.));
#74 = IFCDIRECTION((1., 0.));
#75 = IFCSEVENTHORDERPOLYNOMIALSPIRAL(#76, -78.8880838459446, 63.7638813456506, -57.7378785242934, 64.2314061308743, 83.922298125931, -125.657906854859, $, $);
#76 = IFCAXIS2PLACEMENT2D(#77, $);
#77 = IFCCARTESIANPOINT((0., 0.));
```

2.11.3 Evaluate Point on Curve

Compute the curve coordinates at a distance along the curve, $u = 100$

Step 1 — Evaluate the parent curve at the trim start

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Step 2 — Form the normalization matrix M_N

Since $x_0 = 0$, $y_0 = 0$, and $\theta_0 = 0$, $M_N = I$.

Step 3 — Form the curve segment placement matrix M_{CSP}

Represent `IfcCurveSegment.Placement` in matrix form. In this example, the placement is at (0,0) with RefDirection (1,0) which results in an identity matrix.

$$M_{CSP} = I$$

Step 4 — Evaluate and map each point

Compute point and curve tangent at 100 m from the start.

$$\theta(s) = \frac{A_7}{8\,|A_7^9|} s^8 + \frac{1}{7A_6^7} s^7 + \frac{A_5}{6\,|A_5^7|} s^6 + \frac{1}{5A_4^5} s^5 + \frac{A_3}{4\,|A_3^5|} s^4 + \frac{1}{3A_2^3} s^3 + \frac{A_1}{2\,|A_1^3|} s^2 + \frac{1}{A_0} s$$

$$x = \int_0^{100} \cos\theta(s)\ ds = 99.76319823871657$$

$$y = \int_0^{100} \sin\theta(s)\ ds = 4.499916129045404$$

$$\theta(100) = \frac{-78.8880838}{8\,|(-78.8880838)^9|} 100^8 + \frac{1}{7(63.76388134)^7} 100^7 + \frac{-57.7378785}{6\,|(-57.7378785)^7|} 100^6 + \frac{1}{5(64.231406)^5} 100^5 + \frac{83.92229813}{4\,|(83.92229813)^5|} 100^4 + \frac{}{3(-1}$$

$$dx = \cos(0.1666666667) = 0.9861432315629198$$

$$dy = \sin(0.1666666667) = 0.16589613269344608$$

In matrix form

$$M_{PC} = \begin{bmatrix} 0.9861432315629198 & -0.16589613269344608 & 0 & 99.76319823871657 \\ 0.16589613269344608 & 0.9861432315629198 & 0 & 4.499916129045404 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Apply the normalization and curve segment placement to the parent curve point

$$M_h = M_{CSP} M_N M_{PC} = I\,I \begin{bmatrix} 0.9861432315629198 & -0.16589613269344608 & 0 & 99.76319823871657 \\ 0.16589613269344608 & 0.9861432315629198 & 0 & 4.499916129045404 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.9861432315629198 & -0.16589 \\ 0.16589613269344608 & 0.986143 \\ 0 & \\ 0 & \end{bmatrix}$$

Section 3.0 - Vertical Alignment

A vertical alignment defines the elevation profile of a route as a function of distance along its horizontal path. Combined with the horizontal alignment, it establishes the complete three-dimensional geometry of a road, railway, or other linear infrastructure element.

3.1 General

The geometric representation of a vertical alignment is accomplished with `IfcGradientCurve`. This defines the vertical alignment in a "distance along, elevation" coordinate system. Distance along is measured along `IfcGradientCurve.BaseCurve` = `IfcCompositeCurve`.

Table 3.1-1 maps each `IfcAlignmentVertical.PredefinedType` to its corresponding parent curve type.

Business Logic (`IfcAlignmentVertical.PredefinedType`)	**Geometric Representation (`IfcCurveSegment.ParentCurve`)**
CONSTANTGRADIENT	`IfcLine`
CIRCULARARC	`IfcCircle`
CLOTHOID	`IfcClothoid`
PARABOLICARC	`IfcPolynomialCurve`

Table 3.1-1 — Mapping of business logic to geometric representation for vertical alignment

The following parameters are used throughout this section:

$h_l =$ Horizontal Length

$g_s =$ Start Gradient

$g_e =$ End Gradient

3.2 Curve Segment Evaluation Algorithm

Vertical segments are evaluated in a two-dimensional "distance along, elevation" coordinate system. In this system $x(s)$ is the distance measured along the horizontal `IfcCompositeCurve` and $y(s)$ is the elevation. The grade angle is $\theta(s) = \tan^{-1}(g(s))$ where $g(s)$ is the gradient.

Steps 1–4 follow the identical procedure described in Section 2.2 for horizontal segments, substituting distance along for the horizontal x-coordinate and elevation for the horizontal y-coordinate. Let s_0 = `IfcAlignmentVerticalSegment.StartDistAlong`.

Step 1 — Evaluate the parent curve at the trim start

Compute the distance along $d_0 = x(s_0)$, elevation $z_0 = y(s_0) = \text{IfcAlignmentVerticalSegment.StartHeight}$, and grade angle $\theta_0 = \theta(s_0)$.

Step 2 — Form the normalization matrix M_N

M_N simultaneously translates the trim-start point to the origin and rotates so that the tangent at s_0 aligns with the positive x-direction.

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -d_0\cos\theta_0 - z_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & -d_0\sin\theta_0 - z_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

M_{CSP} is constructed from `IfcCurveSegment.Placement`, where (d_p, z_p) is the `Location` (distance along, elevation) and θ_p is the grade angle of the `RefDirection`.

$$M_{CSP} = \begin{bmatrix} \cos\theta_p & -\sin\theta_p & 0 & d_p \\ \sin\theta_p & \cos\theta_p & 0 & z_p \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Evaluate and map each point in the vertical plane

For the point at distance along s, compute $x(s)$, $y(s)$, and $\theta(s)$:

$$M_{PC} = \begin{bmatrix} \cos\theta(s) & -\sin\theta(s) & 0 & x(s) \\ \sin\theta(s) & \cos\theta(s) & 0 & y(s) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_v = M_{CSP} M_N M_{PC}$$

Column 4 of M_v contains the distance along d and elevation z of the evaluated point. Column 1 contains $(dx_v, dy_v) = (\cos\theta_v, \sin\theta_v)$, the grade direction at that point. Step 6 is performed immediately for this point before moving to the next point.

Step 6 — Combine with the horizontal alignment point to produce the 3D placement matrix

The vertical result lives in the (distance along, elevation) plane and must be merged with the horizontal alignment to form a 3D position and orientation. The "distance along" axis corresponds to the curve tangent of the horizontal alignment and the vertical y is elevation (Z in 3D).

Evaluate the horizontal placement matrix M_h at distance d along the `IfcCompositeCurve`. This yields the horizontal 2D position (x_h, y_h) and the horizontal alignment tangent direction $(dx_h, dy_h) = (\cos\theta_h, \sin\theta_h)$.

M_v is a two-dimensional matrix whose rows index the vertical frame: row 1 = distance-along, row 2 = elevation, row 3 = out-of-plane. Three modifications produce M'_v, the form required for multiplication with M_h.

Column 4 — The distance-along value d is replaced with zero. The horizontal position already comes from M_h; retaining d would displace the point a second time along the horizontal tangent.

Rows 2 and 3 — M_h's third row $(0,\ 0,\ 1,\ 0)$ passes row 3 of the right operand through unchanged into row 3 of the product. Because row 3 of M_{3D} must carry the elevation (Z) component of every vector, elevation must occupy row 3 of M'_v — not row 2 as in M_v. The row semantics are therefore swapped: in M'_v, row 2 is the cross-track (lateral) component and row 3 is the elevation (Z) component.

Columns 2 and 3 — The row reordering reindexes the column values and exchanges the columns. M_v column 2, the in-plane normal $(-dy_v,\ dx_v,\ 0)$ in [d-along, elevation, out-of-plane] ordering, becomes $(-dy_v,\ 0,\ dx_v)$ in [d-along, cross-track, elevation] ordering — the 3D up direction — and moves to column 3. M_v column 3, the out-of-plane unit direction $(0,\ 0,\ 1)$, becomes $(0,\ 1,\ 0)$ — the cross-track direction — and moves to column 2.

$$M'_v = \begin{bmatrix} dx_v & 0 & -dy_v & 0 \\ 0 & 1 & 0 & 0 \\ dy_v & 0 & dx_v & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The columns of M'_v define directions in the 3D frame local to the alignment:

- **Column 1** $(dx_v,\ 0,\ dy_v)$: the grade direction — dx_v scales the contribution along the horizontal tangent; dy_v is the elevation rate of change (Z component of the 3D tangent).
- **Column 2** $(0,\ 1,\ 0)$: the cross-track direction, always lateral to the alignment and always horizontal.
- **Column 3** $(-dy_v,\ 0,\ dx_v)$: the "up" direction in the vertical plane of the alignment, orthogonal to the grade direction.
- **Column 4** $(0,\ 0,\ z)$: the elevation offset with no horizontal displacement.

The 3D placement matrix is:

$$M_{3D} = M_h \cdot M'_v$$

Multiplying out, the columns of M_{3D} are:

- **Column 1** (RefDirection / 3D tangent): $(dx_h \cdot dx_v,\ dy_h \cdot dx_v,\ dy_v)$ — the horizontal tangent scaled by $\cos\theta_v$, with $\sin\theta_v$ as the Z component.
- **Column 2** (Y / cross-track): $(-dy_h,\ dx_h,\ 0)$ — the horizontal normal direction, unchanged from M_h.
- **Column 3** (Axis / up): $(-dx_h \cdot dy_v,\ -dy_h \cdot dy_v,\ dx_v)$ — orthogonal to both the 3D tangent and the cross-track direction.

- **Column 4** (Location): $(x_h,\ y_h,\ z)$ — the full 3D position.

3.3 Constant Gradient

A constant gradient is geometrically represented with a segment trimmed from an `IfcLine` parent curve.

3.3.1 Parent Curve Parametric Equations

$$x(s) = p_x + s$$
$$y(s) = p_y + s(dy/dx)$$

Note that these equations are different from the equations for horizontal.

[todo: can this be reformulated to be the same as horizontal?]

3.3.2 Semantic Definition to Geometry Mapping

Mapping of the semantic definition of the linear segment to the geometric definition is described with the following example.

Consider a vertical gradient at an uphill slope of 0.5 starting at point (0,10). The horizontal projection of the segment length is 100.

```
#44 = IFCALIGNMENTVERTICALSEGMENT($, $, 0., 100., 10., 0.5, 0., $, .CONSTANTGRADIENT.);
```

Define the direction of the `IfcLine` so it matches the vertical segment

$$dx = \cos\left(\tan^{-1} 0.5\right) = 0.894427191$$
$$dy = \sin\left(\tan^{-1} 0.5\right) = 0.44713595$$

Define the `IfcLine` as passing through point (0,0)

```
#80 = IFCLINE(#81, #82);
#81 = IFCCARTESIANPOINT((0., 0.));
#82 = IFCVECTOR(#83, 1.);
#83 = IFCDIRECTION((0.894427190999916, 0.447213595499958));
```

Place the curve segment at (0,10) with a tangent direction (0.894427190999916, 0.447213595499958).

```
#77 = IFCAXIS2PLACEMENT2D(#78, #79);
#78 = IFCCARTESIANPOINT((0., 10.));
#79 = IFCDIRECTION((0.894427190999916, 0.447213595499958));
```

`IfcCurveSegment.SegmentLength` is the length measured along the trimmed curve. For a horizontal length of 100, the length along the curve is 100/0.894427190999916 = 111.803398874989

The curve segment is defined as

```
#71 = IFCCURVESEGMENT(.CONTINUOUS., #77, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(111.803398874989), #80);
```

3.3.3 Compute Point on Curve

Compute the vertical placement matrix for the point at horizontal distance $u = 50$ m from the start of the curve segment.

Step 1 — Evaluate the parent curve at the trim start

The trim begins at $s_0 = \text{SegmentStart} = 0$. For the `IfcLine` through the origin with direction $(dx,\ dy) = (0.894427191,\ 0.447213595)$:

$$d_0 = x(0) = 0 \quad z_0 = y(0) = 0 \quad \theta_0 = tan^{-1}\left(\frac{dy}{dx}\right) = tan^{-1}\left(\frac{0.894427191}{0.447213595}\right) = 0.463647609 \text{ rad}$$

Step 2 — Form the normalization matrix M_N

$$M_N = \begin{bmatrix} 0.894427191 & 0.447213595 & 0 & 0 \\ -0.447213595 & 0.894427191 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

From `IfcCurveSegment.Placement`: $(d_p,\ z_p) = (0,\ 10), \theta_p = 0.463647609$ rad:

$cos\theta_p = cos(0.463647609) = 0.894427191 \quad sin\theta_p = sin(0.463647609) = 0.447213595$

$$M_{CSP} = \begin{bmatrix} 0.894427191 & -0.447213595 & 0 & 0 \\ 0.447213595 & 0.894427191 & 0 & 10 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Evaluate and map the point

The horizontal distance $u = 50$ m is not the arc-length parameter. Convert to arc-length s along the `IfcLine`:

$$s = s_0 + \frac{u}{dx} = 0 + \frac{50}{0.894427191} = 55.9016994 \text{ m}$$

Evaluate the parent curve at $s = 55.9016994$:

$$d = x(55.9016994) = 55.9016994 \times 0.894427191 = 50.000 \text{ m}$$

$$z = y(55.9016994) = 55.9016994 \times 0.447213595 = 25.000 \text{ m}$$

$$\theta = 0.463647609 \text{ rad} \quad \text{(constant for a line)}$$

$$M_{PC} = \begin{bmatrix} 0.894427191 & -0.447213595 & 0 & 50.000 \\ 0.447213595 & 0.894427191 & 0 & 25.000 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The vertical placement matrix is:

$$M_v = M_{CSP}\ M_N\ M_{PC} = \begin{bmatrix} 0.894427191 & -0.447213595 & 0 & 0 \\ 0.447213595 & 0.894427191 & 0 & 10 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.894427191 & 0.447213595 & 0 & 0 \\ -0.447213595 & 0.894427191 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.894427191 & -0.447213595 & 0 & 5 \\ 0.447213595 & 0.894427191 & 0 & 2 \\ 0 & 0 & 1 & \\ 0 & 0 & 0 & \end{bmatrix}$$

$$M_v = \begin{bmatrix} 0.894427191 & -0.447213595 & 0 & 50.000 \\ 0.447213595 & 0.894427191 & 0 & 35.000 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.4 Circular Arc

A circular vertical curve is geometrically represented with a segment trimmed from an `IfcCircle` parent curve.

3.4.1 Parent Curve Parametric Equations

3.4.2 Semantic Definition to Geometry Mapping

Vertical circular arcs are tricky. The "distance along" dimension is horizontal while the `IfcCurveSegment` trimming parameters `SegmentStart` and `SegmentLength` are measured along the `IfcCircle`.

The following procedure maps the semantic parameters of a vertical circular arc to its geometric definition.

Given the following semantic definition of a vertical circular arc, create the geometric definition.

```
#320 = IFCALIGNMENTVERTICALSEGMENT($, $, 144.917656958471, 239.704902937655, 25.3780433292418, -8.17722122076371E-4, -1.28040164299203E-2, -20000., .CIRCULARARC.
```

Determine the tangent slope angle at the start and end of the segment.

$g_{start} = -8.17722122076371 \cdot 10^{-4}$

$\theta_{start} = tan^{-1}(g_{start})$

$\theta_{start} = tan^{-1}(-8.17722122076371 \cdot 10^{-4}) = -8.177219398 \cdot 10^{-4}$

$g_{end} = -1.28040164299203 \cdot 10^{-2}$

$\theta_{end} = tan^{-1}(g_{end})$

$\theta_{end} = tan^{-1}(-1.28040164299203 \cdot 10^{-2}) = -1.2803316789 \cdot 10^{-2}$

Compute the radius of the circle.

`IfcAlignmentVerticalSegment.RadiusOfCurvature` is optional. If provided, it should be consistent with the `HorizontalLength`, `StartGradient` and `EndGradient`, but is not guarenteed. For this reason, the radius is computed from the required `HorizontalLength`, `StartGradient` and `EndGradient` attributes. Note that in computing the radius, it is taken to be an absolute value because `IfcCircle.Radius` is a `IfcPositiveLengthMeasure`.

$R = \left| \frac{h_l}{sin(\theta_{start}) - sin(\theta_{end})} \right|$

$h_l = \text{IfcAlignmentVerticalSegment.HorizontalLength} = 239.704902937655$

$R = \left| \frac{239.704902937655}{sin(-1.2803316789 \cdot 10^{-4}) - sin(-8.177219398 \cdot 10^{-4})} \right| = 20000.0$

Comptue the direction of vectors that are perpendicular to the circle, directed away from the center point

if $\theta_{start} < \theta_{end}$

Curve is sagging (curve is on the bottom half of the circle)

$\Delta_{start} = \theta_{start} + \frac{3}{2}\pi$

$\Delta_{end} = \theta_{end} + \frac{3}{2}\pi$

else

Curve is cresting (curve is on top half of the circle)

$\Delta_{start} = \theta_{start} + \frac{1}{2}\pi = 1.56997860486$

$\Delta_{end} = \theta_{end} + \frac{1}{2}\pi = 1.55799301001$

Compute the curve trimming parameters `SegmentStart` and `SegmentLength`

$\text{Segment Start} = R\Delta_{start} = (20000.0)(1.56997860486) = 31399.5720971$

$\text{Segment Length} = R(\Delta_{end} - \Delta_{start}) = (20000.0)(1.55799301001 - 1.56997860486) = -239.711897$

For a cresting curve, the segment length is negative which means the trim occurs in clock-wise direction.

Determine the placement of the trimmed curve

X = `IfcAlignmentVerticalSegment.StartDistAlong` = 144.917656958471

Y = `IfcAlignmentVerticalSegment.StartHeight` = 25.3780433292418

$dx = cos(\theta_{start}) = cos(-8.177219398x10^{-4} = 0.9999966566$

$dy = sin(\theta_{start}) = sin(-8.177219398x10^{-4}) = -8.1772184868x10^{-4}$

The geometric representation is

```
#1974 = IFCCURVESEGMENT(.CONTSAMEGRADIENT., #1980, IFCLENGTHMEASURE(31399.5720971016), IFCLENGTHMEASURE(-239.711897000001), #1983);
#1980 = IFCAXIS2PLACEMENT2D(#1981, #1982);
#1981 = IFCCARTESIANPOINT((144.917656958471, 25.3780433292418));
#1982 = IFCDIRECTION((9.99999665665433E-1, -8.177218486836E-4));
#1983 = IFCCIRCLE(#1984, 20000.);
#1984 = IFCAXIS2PLACEMENT2D(#1985, #1986);
#1985 = IFCCARTESIANPOINT((0., 0.));
#1986 = IFCDIRECTION((1., 0.));
```

3.4.3 Compute Point on Curve

[todo: provide example calcs for steps 1 - 5. compute at u = 150, be clear u is horizontal]

3.5 Clothoid

:warning: **[Finish this]** :warning:

Start angle = atan(startGradient)

End angle = atan(endGradient)

Start radius = start radius of curvature

End radius = end radius of curvature

Set equal to horizontal length, adjust curve length until computed value is equal to the specified horizontal length. Numerically solve

$$A\sqrt{\pi}(\int_0^u \cos(\pi\frac{At^2}{2|A|})dt\ x$$

Figure 3.5-1 Vertical clothoid

3.6 Parabolic Arc

The most common transition curve in a vertical profile is a parabola. The geometric representation is `IfcPolynomialCurve`. Mapping of the semantic definition to the geometric definition can be a bit tricky.

3.6.1 Parent Curve Parametric Equations

The general form of a parabola is

$$y(x) = A_2x^2 + A_1x + A_0$$

where:

A_2 = (end gradient - start gradient)/(2 * horizontal length)

A_1 = start gradient

A_0 = start height

The gradient of the curve is

$$y'(x) = 2A_2x + A_1$$

3.6.2 Semantic Definition to Geometry Mapping

Consider a 1600 m parabolic vertical cover that starts 1200 m along the horizontal alignment. The entry grade is 1.75% and the exit grade is -1%. The elevation at the start of the curve is 121 m.

The semantic definition is

```
#289=IFCALIGNMENTVERTICALSEGMENT($,$,1200.,1600.,121.,0.017500000000000002,-0.01,$,.PARABOLICARC.);
```

Compute the polynomai curve coefficients

$A_0 = 121.$

$A_1 = 0.0175$

$A_2 = \frac{-0.01-0.0175}{2 \cdot 1600.} = -8.59375 \cdot 10^{-6}$

It is easiest to place the parent curve at the origin and orient it with the global coordinate system. The parent curve is defined as

```
#291=IFCCARTESIANPOINT((0.,0.));
#292=IFCDIRECTION((1.,0.));
#293=IFCAXIS2PLACEMENT2D(#291,#292);
#294=IFCPOLYNOMIALCURVE(#293,(0.,1.),(121.,0.017500000000000002,-8.5937500000000005E-06),$);
```

> Note 1: Even though vertical is typically z, alignment is 2.5D geometry and the coordinate system of gradient curve is "Distance along Horizontal", "Elevation" which is a 2D curve in the plane of the horizontal curve. When the `IfcGradientCurve` and `IfcCompositeCurve` are combined to get a 3D point, the elevation is then mapped to Z. See example in Section 3.7 below.

> Note 2: The coefficients A_0, A_1, and A_2 must have the following unit of measure, consistent with the project units:
>
> $A_0 = Length^1$
>
> $A_1 = Length^0$
>
> $A_2 = Length^{-1}$
>
> The coefficients of `IfcPolynomialCurve` expect real numbers without explictit unit of measure. This is a problem with the IFC Specification. See the discussion of `IfcAlignmentHorizontalSegment` and `IfcPolynomialCurve` for Cubic Transition Curve in Section 2.0 - Horizontal Alignment. Implicit units of measure are required for the polynomial coefficients.

The polynomial curve is trimmed using `IfcCurveSegment.SegmentStart` and `IfcCurveSegment.SegmentLength`. These parameters are measured along the length of the curve. The horizontal projection of the segment length, from `IfcAlignmentVerticalSegment.HorizontalLength` is $h_l = 1600$. `SegmentLength` will be slightly longer because it is the distance along the curve.

The distance along a curve is

$s(x) = \int (\sqrt{(y')^2 + 1})dx$

The length along the parabolic curve is then:

$s(x) = \int \sqrt{4A_2^2x^2 + 4A_2A_1x + A_1^2 + 1}dx$

The solution to this integral is:

$s(x) = \int \left(\sqrt{ax^2 + bx + c}\right)dx = \frac{b+2ax}{4a}\sqrt{ax^2 + bx + c} + \frac{4ac-b^2}{8a^{\frac{3}{2}}}ln\left|2ax + b + 2\sqrt{a(ax^2 + bx + c)}\right|$

The length along the curve is $L_c = s(h_l) - s(0.0)$.

Let $\quad a = 4A_2^2 \quad b = 4A_2A_1 \quad c = A_1^2 + 1$

Compute the coefficients a, b, c, substitute curve length equation and solve. The curve length is

$L_c = 1600.0616641340894$

The placement of the trimmed curve segment is noteworthy. From the `IfcAlignmentVerticalSegment` attributes, the parabolic segment of the vertical alignment starts at 1200 from the beginning of the horizontal alignment and is at an elevation of 121. The RefDirection vector at the start of

the curve is needed as well.

The gradient is $y'(x = 0) = 0.0175$

$\theta_p = tan^{-1}(0.0175) = 0.017498213869856595$

$dx = cos(0.017498213869856595) = 0.017497320927833689$

$dy = sin(0.017498213869856595) = 0.99984691016192495$

The geometric representation is

```
#291=IFCCARTESIANPOINT((0.,0.));
#292=IFCDIRECTION((1.,0.));
#293=IFCAXIS2PLACEMENT2D(#291,#292);
#294=IFCPOLYNOMIALCURVE(#293,(0.,1.),(121.,0.017500000000000002,-8.5937500000000005E-06),$);
#295=IFCCARTESIANPOINT((1200.,121.));
#296=IFCDIRECTION((0.99984691016192495,0.017497320927833689));
#297=IFCAXIS2PLACEMENT2D(#295,#296);
#298=IFCCURVESEGMENT(.CONTSAMEGRADIENT.,#297,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(1600.0616641340894),#294);
```

3.6.3 Compute Point on Curve

[todo: provide example calcs for steps 1 - 5. compute at d = 1500, u = 1500-1200 = 300, be clear u is horizontal from the start of the parabola]

Compute the vertical placement matrix for the point at horizontal distance 1500 m from the start of the alignment. This vertical alignment segments starts at 1200 from the start of the alignment. The evaluation point is $u = 1500 - 1200 = 300$ from the start of the segment.

Step 1 — Evaluate the parent curve at the trim start

Because the parent curve is located at (0,0) in the direction (1,0), $x_0 = 0, y_0 = 0, \theta_0 = 0$.

Step 2 — Form the normalization matrix M_N

Since $x_0 = 0, y_0 = 0, \theta_0 = 0$,M_N = I$

Step 3 — Form the curve segment placement matrix M_{CSP}

From `IfcCurveSegment.Placement`: $(d_p,\ z_p) = (1200,\ 121)$, $dx_p = 0.99984691016192495$, $dy_p = 0.017497320927833689$

$$M_{CSP} = \begin{bmatrix} 0.99984691016192495 & -0.017497320927833689 & 0 & 1200 \\ 0.017497320927833689 & 0.99984691016192495 & 0 & 121 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Evaluate and map the point

Evaluate the parent curve at $x = 300$

$y(300) = -8.59375 \cdot 10^{-6} 300^2 + 0.0175(300) + 121 = 125.4765625$

$y'(300_2(-8.59375 \cdot 10^{-6})300 + 0.0175 = 0.01234375$

$dx = \frac{1}{\sqrt{(0.01234375)^2+1}} = 0.999923825$

$dy = \frac{0.01234375}{\sqrt{(0.01234375)^2+1}} = 0.01234281$

$$M_{PC} = \begin{bmatrix} 0.999923825 & -0.01234281 & 0 & 300 \\ 0.01234281 & 0.999923825 & 0 & 125.4765625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The vertical placement matrix is:

$$M_v = M_{CSP}\, M_N\, M_{PC} = \begin{bmatrix} 0.99984691016192495 & -0.017497320927833689 & 0 & 1200 \\ 0.017497320927833689 & 0.99984691016192495 & 0 & 121 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} [I] \begin{bmatrix} 0.999923825 & -0.01234281 & 0 & 300 \\ 0.01234281 & 0.999923825 & 0 & 125.4765625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_v = \begin{bmatrix} 0.9999998299568322 & -0.0005831691921746294 & 0.0 & 1500.0 \\ 0.0005831691921746294 & 0.9999998299568322 & 0.0 & 125.4765625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.7 Combined 3D

To complete evaluation of a point on the alignment, the 2D vertical is combined with the 2D horizontal resulting in a point in 3D space.

From Section 2.3.3, the horizontal alignment point at 1600 along a line is

$$M_h = \begin{bmatrix} 0.83925279 & 0.54374114 & 0 & 1758.879185 \\ -0.54374114 & 0.83925279 & 0 & 1684.3878387 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

From the parabolic arc example

$$M_v = \begin{bmatrix} 0.9999998299568322 & -0.0005831691921746294 & 0.0 & 1500.0 \\ 0.0005831691921746294 & 0.9999998299568322 & 0.0 & 125.4765625 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 6 — Combine with the horizontal alignment to produce the 3D placement matrix

$$M'_v = \begin{bmatrix} dx_v & 0 & -dy_v & 0 \\ 0 & 1 & 0 & 0 \\ dy_v & 0 & dx_v & z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.9999998299568322 & 0.0 & -0.0005831691921746294 & 0.0 \\ 0 & 1 & 0 & 0 \\ 0.0005831691921746294 & 0 & 0.9999998299568322 & 125.4765625 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The 3D placement matrix is:

$$M_{3D} = M_h \cdot M'_v$$

$$M_{3D} = \begin{bmatrix} 0.83925279 & 0.54374114 & 0 & 1758.879185 \\ -0.54374114 & 0.83925279 & 0 & 1684.3878387 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.9999998299568322 & 0.0 & -0.0005831691921746294 & 0.0 \\ 0 & 1 & 0 & 0 \\ 0.0005831691921746294 & 0 & 0.9999998299568322 & 125.4765625 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{3D} = \begin{bmatrix} 0.8391888595725846 & 0.5437414408769801 & -0.010358737485349092 & 1758.8791849555332 \\ -0.5437000211676682 & 0.8392527899703555 & 0.006711297136288405 & 1684.38783868453 \\ 0.012342809710188737 & 0.0 & 0.999923824622885 & 125.4765625 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

outline:

- describe cant in terms of it being a deviation to railroad alignment vertical in curves. it provides for track banking
- need figure the show cross slope and coordinate system 3D and 2-2D elevation and section.
- need figure that shows centerline and railhead deviation
- reference british standard (BS EN 13803:2017 Railway applications - Track - Track alignment design parameters - Track gauges 1 435 mm and wider, Table C.1 "C:\Users\BriceR\OneDrive - Washington State Department of Transportation\BIM for Bridges\Rail Spec\bs-en-13803-2017.pdf") and describe the equivalent functional form of equations used for horizontal. The form of the horizontal transition spirals equations is the same form as the cant equation.
- Equivalance of functional form - state that the derivative of $\theta(t)$ equation, which is the curvature $\kappa(t)$, for horizontal has the same form as the $\frac{D(t)}{L^2}$ equation for cant.
- Need to add a general discussion about the spiral coefficients. This is the text used in some sections - it doesn't need to be repeated everywhere. "The polynomial coefficients must have units of length. The basic form of the coefficient of the ith term is $A_i = \frac{L^{\frac{i+2}{i+1}}}{\sqrt[(i+1)]{|a_i|}} \frac{a_i}{|a_i|}$" "Perform a dimensional analysis with l representing term with length units. We see that the resulting coefficient A_i has units of length. $\frac{l^{\frac{i+2}{i+1}}}{\sqrt[(i+1)]{|l|}} \frac{l}{|l|} = \frac{l^{\frac{i+2}{i+1}}}{l^{\frac{1}{i+1}}} = l^{\frac{i+2}{i+1}} l^{\frac{-1}{(i+1)}} = l^{\frac{i+1}{i+1}} = l$," - maybe this needs to go in the last segment (or maybe just the dimensionality analysis in the last section)
- horizontal and cant always use the same curve type, though technically ifc would permit mixing them
- provide a bit more discussion in the examples. reference the example files
- need to discuss cross slope, Axis vector, and their orientation and relation to one another. Axis is normal to cross slope line. $\phi(s) = \phi_s + \left(\frac{\phi_e - \phi_s}{D_e - D_s}\right)\left(D(s) - D_s\right)$. Should this equation use ΔD?
- state up front that the railhead distance for all examples is 1.5 m, so we don't have to keep restating this

Section 4.0 - Cant Alignment

Cant (also called superelevation) is the transverse inclination of a railway track cross-section: the elevation difference between the two railheads when one rail is raised above the other. By tilting the track into a curve, cant directs a component of gravitational force inward, partially offsetting the centrifugal acceleration experienced by vehicles negotiating the curve. The result is improved ride quality, reduced wheel–rail lateral forces, and the ability to operate at higher speeds through curves without exceeding comfort or safety limits.

Cant is measured as a length — the height difference between railheads — rather than as an angle. The conversion between the two depends on track gauge. `IfcAlignmentCant.RailHeadDistance` supplies the centre-to-centre distance between railheads, which is the lever arm for this conversion.

4.1 General

An `IfcSegmentedReferenceCurve` describes the cross slope of superelevated rail lines as a track banks through curves. When the point of rotation is about one of the railheads, the alignment elevation must deviate to accomodate the cross slope.

An `IfcSegmentedReferenceCurve` also describes this deviation in elevation. As a subtype of `IfcCompositeCurve`, an `IfcSegmentedReferenceCurve` consists of an end to start collection of `IfcCurveSegment`. An `IfcSegmentedReferenceCurve` also has a `BasisCurve` which is typically an `IfcGradientCurve`.

The `IfcCurveSegment.ParentCurve` defines the change in cross slope between rail heads over the length of the segment. When the `IfcCurveSegment.Placement.Location` differs from the `IfcCurveSegment.Placement.Location` of the next segment, the `IfcCurveSegment.ParentCurve` also defines variation in the deviating elevation. If the `IfcCurveSegment.Placement.Location` is the same as for the start of the next segment, the deviating elevation along the length of the segment is constant.

Table 4.1-1 maps each `IfcAlignmentCant.PredefinedType` to its corresponding parent curve type.

Business Logic (`IfcAlignmentCant.PredefinedType`)	**Geometric Representation (`IfcCurveSegment.ParentCurve`)**
CONSTANTCANT	`IfcLine`
LINEARTRANSITION	`IfcClothoid`
HELMERTCURVE	`IfcSecondOrderPolynomialSpiral`
BLOSSCURVE	`IfcThirdOrderPolynomialSpiral`
COSINECURVE	`IfcCosineSpiral`
SINECURVE	`IfcSineSpiral`
VIENNESEBEND	`IfcSeventhOrderPolynomialSpiral`

Table 4.1-1 — Mapping of business logic to geometric representation for cant alignment

The transition functions used to shape cant variation have the same functional form as functions used for horizontal transition curves: clothoid (linear cant), Helmert, Bloss, cosine, sine, and Viennese bend. In practice, the cant transition type always matches the horizontal transition type, though IFC does not enforce this constraint.

As an example, the radius of curvature for a Helmert curve is a second order polynomial of the form

$$\theta(t) = \frac{t^3}{3A_2^3} + \frac{A_1}{2\left|A_1^3\right|}t^2 + \frac{t}{A_0}$$

$$\kappa(t) = \frac{\theta(t)}{dt} = \frac{1}{A_2^3}t^2 + \frac{A_1}{\left|A_1^3\right|}t + \frac{1}{A_0}$$

The cant deviating elevation is of the form

$$\frac{D(s)}{L^2} = \frac{1}{A_2^3}s^2 + \frac{A_1}{\left|A_1^3\right|}s + \frac{1}{A_0}$$

The deviating elevation has the same functional form as the radius of curvature of the horizontal alignment segment.

In IFC, the semantic cant profile is encoded in `IfcAlignmentCant` and its child `IfcAlignmentCantSegment` entities. The geometric representation is an `IfcSegmentedReferenceCurve`, which evaluates the cant at every point along the alignment and, in conjunction with the horizontal alignment `IfcCompositeCurve` and the vertical alignment `IfcGradientCurve`, produces the full 3D track centerline geometry.

[todo: add a figure to illustrate the cross slope and centerline elevation deviation]

Each `IfcCurveSegment` in an `IfcSegmentedReferenceCurve` is evaluated in a two-dimensional coordinate system whose axes are *distance along the horizontal alignment* d and *deviating elevation* D. The deviating elevation is the vertical offset applied to the track centerline, away from the gradient curve, to produce the correct cross-slope angle at every point along the segment.

The `StartCantLeft` D_{sl}, `EndCantLeft` D_{el}, `StartCantRight` D_{sr}, and `EndCantRight` D_{el} attributes of `IfcAlignmentCantSegment` determine D_s and D_e as the averages of their respective left and right values:

$$D_s = \frac{D_{sl} + D_{sr}}{2}, \quad D_e = \frac{D_{el} + D_{er}}{2}$$

The cross-slope angle ϕ at a given point is not obtained directly from the parent curve equation; instead it is interpolated between the start and end cross-slope angles encoded in the `Axis` vectors of the `IfcCurveSegment.Placement`, using the deviating elevation as the interpolation parameter:

$$\phi(s) = \phi_s + \left(\frac{\phi_e - \phi_s}{\Delta D}\right)\big(D(s) - D_s\big)$$

where ϕ_s and ϕ_e are the cross-slope angles at the start and end of the segment and D_s and D_e are the corresponding deviating elevations. $\Delta D = D_e - D_s$, and $D(s)$ is the deviation elevation at s. The `Axis` vector at any point is $(0,\ \cos\phi(s),\ \sin\phi(s))$. The `Axis` vector is perpendicular to a line connecting the railheads in an upwards direction. The cross-slope angle ϕ is the angle from the transverse y axis to the `Axis` vector. In sections without cant, `Axis` is (0,0,1) and $\phi = \frac{\pi}{2}$.

Together, the distance along, the deviating elevation $D(s)$, and the cross-slope angle $\phi(s)$ fully specify a 3D placement frame for the track centerline at each arc-length s. Section 4.2 describes an algorithm for constructing that frame and composing it with the horizontal and vertical matrices to produce a 3D position.

4.2 Curve Segment Evaluation Algorithm

Cant segments are evaluated in a two-dimensional "distance along, deviating elevation" coordinate system in which $x(s) = d$ is the distance measured along the horizontal `IfcCompositeCurve` and $y(s) = D(s)$ is the deviating elevation: the vertical offset applied to the track centerline to accommodate the cross slope. Unlike horizontal and vertical segments, each cant point also carries a cross slope angle ϕ, making the local frame inherently three-dimensional.

Steps 1—4 follow the identical procedure describe in Section 2.2 for horizontal segments, substituting distance along for the horizontal x-coordinate and deviating elevation for the horizontal y-coordinate. Let $s_0 =$ `IfcAlignmentCantSegment.StartDistAlong`.

Step 1 — Evaluate the parent curve at the trim start

Compute the deviating elevation $z_0 = D_0 = D(s_0)$ and the slope angle $\theta_0 = \tan^{-1}(D'(s_0))$. $d_0 = x(s_0)$ is the distance along the parent curve at the trim start.

Step 2 — Form the normalization matrix M_N

M_N simultaneously translates the trim-start point to the origin and rotates so that the tangent at s_0 aligns with the positive x-direction.

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -d_0\cos\theta_0 - z_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & -d_0\sin\theta_0 - z_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

M_{CSP} is constructed from `IfcCurveSegment.Placement`: (d_p, D_p) is the `Location` (distance along, deviating elevation), the `RefDirection` gives slope angle θ_p, and the `Axis` gives cross slope angle ϕ_p.

$$\mathbf{RefDir}_p = (\cos\theta_p,\ \sin\theta_p,\ 0), \quad \mathbf{Axis}_p = (0,\ \cos\phi_p,\ \sin\phi_p)$$
$$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p, \quad \mathbf{X}_p = \mathbf{Axis}_p \times \mathbf{Y}_p$$

The placement in matrix form is

$$M_{CSP} = \begin{bmatrix} X_p.x & Y_p.x & Axis_p.x & d_p \\ X_p.y & Y_p.y & Axis_p.y & 0 \\ X_p.z & Y_p.z & Axis_p.z & D_p \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Evaluate and map each point

For the point at a distance along the horizontal alignment s, compute $D(s)$ and $\theta(s) = \tan^{-1}(D'(s))$. Interpolate the cross slope angle between the segment start and end values:

$$\phi(s) = \phi_s + \left(\frac{\phi_e - \phi_s}{\Delta D}\right)(D(s) - D_s)$$

Form the local frame:

$$\mathbf{X} = (\cos\theta(s),\ \sin\theta(s),\ 0), \quad \mathbf{Z} = (0,\ \cos\phi,\ \sin\phi)$$
$$\mathbf{Y} = \mathbf{Z} \times \mathbf{X}, \quad \mathbf{Axis} = \mathbf{X} \times \mathbf{Y}$$
$$M_{PC} = \begin{bmatrix} X.x & Y.x & Axis.x & d \\ X.y & Y.y & Axis.y & 0 \\ X.z & Y.z & Axis.z & D(s) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where $d = x(s)$ is the distance along the horizontal alignment at arc-length s.

Apply the normalization and placement in sequence:

$$M_c = M_{CSP}\ M_N\ M_{PC}$$

Column 4 of M_c is $(d,\ 0,\ D)$ — the distance along and deviating elevation. Column 3 (Axis) is the cross-slope direction at that point. Step 5 is performed immediately for this point before moving to the next arc-length s.

Step 5 — Combine with horizontal and vertical to produce the 3D placement matrix

The cant result must be combined with the horizontal matrix M_h (evaluated at distance s along the `IfcCompositeCurve`) and the vertical matrix M_v (evaluated at the same s). All three coordinate systems share the distance-along axis, so the positional components of M_v and M_c must be extracted before multiplication and added back afterward to avoid incorrectly rotating the position offsets.

Construct M_v' as described in Section 3.2.

Construct M_c' by zeroing the distance-along component in column 4 of M_c:

$$M'_c = \begin{bmatrix} X.x & Y.x & Axis.x & 0 \\ X.y & Y.y & Axis.y & 0 \\ X.z & Y.z & Axis.z & D \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Extract position vectors from each modified matrix (setting row 4 to zero), then zero column 4 before multiplying:

$$P_v = M_v' \text{ column 4, row 4 set to } 0 = \begin{bmatrix} 0 \\ 0 \\ z \\ 0 \end{bmatrix}, \qquad P_c = M_c' \text{ column 4, row 4 set to } 0 = \begin{bmatrix} 0 \\ 0 \\ D \\ 0 \end{bmatrix}$$

$$M_v'' = M_v' \text{ with column 4 set to } (0,0,0,1)^T, \qquad M_c'' = M_c' \text{ with column 4 set to } (0,0,0,1)^T$$

Multiply the three orientation matrices, then add back both position offsets:

$$M' = M_h \cdot M_v'' \cdot M_c''$$

$$M_{3Dcant} = M' + \begin{bmatrix} 0 & 0 & 0 & P_{v.x} \\ 0 & 0 & 0 & P_{v.y} \\ 0 & 0 & 0 & P_{v.z} \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & P_{c.x} \\ 0 & 0 & 0 & P_{c.y} \\ 0 & 0 & 0 & P_{c.z} \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Column 4 of the final matrix M_{3Dcant} is the full 3D position of the track centerline. Column 3 is the cross-slope direction, encoding the railhead superelevation at that point.

4.3 Constant Cant

Cant has a constant deviating elevation and cross-slope angle in rail segments between curves or in the circular portion of curves where the tracks are banked at a constant rate.

The parent curve is `IfcLine`. The slope-intercept form of a line is $y = mx + b$, where m is the slope and b is the y-intercept. The derivative is $y' = m$, and the second derivative is $y'' = 0$.

In the context of cant, $D(s) = y'(x)$, so the deviating elevation is a constant value, m, and the rate of change of the deviating elevation $D'(s) = y''(x) = 0$.

4.3.3 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$D(s) = D_s = D_e$$

$$\frac{d}{ds} D(s) = 0.0$$

4.3.2 Semanic Definition to Geometry Mapping

Consider 100 m long segment of a railway that is turning towards the left. The right rail is raised 0.16 m through the circular portion of the curve. The sementic definition is

```
#64 = IFCALIGNMENTCANTSEGMENT($, $, 0., 100., 0., 0., 1.6E-1, 1.6E-1, .CONSTANTCANT.);
```

$D_s = \frac{0.0\ m + 0.16\ m}{2} = 0.08\ m \quad D_e = \frac{0.0\ m + 0.16\ m}{2} = 0.08\ m$

$\Delta D = D_e - D_s = 0.08\ m - 0.08\ m = 0.0\ m$

The parent curve has a constant elevation at $0.08\ m$ and a slope of 0.0.

Define the parent curve as an `IfcLine` passing through (0,0.08) in the direction (1,0).

```
#123 = IFCLINE(#124, #125);
#124 = IFCCARTESIANPOINT((0., 0.08));
#125 = IFCVECTOR(#126, 1.);
#126 = IFCDIRECTION((1., 0.));
```

The `IfcCurveSegment` trims a segment from the parent curve. The curve segment placement must capture the cross-slope rotation between the rail heads. For this reason the placement is an `IfcAxis2Placement3D` (instead of `IfcAxis2Placement2D` used for horizontal and vertical representations).

The cross-slope angle is $\phi = tan^{-1}\left(\frac{\Delta D}{D_{rh}}\right)$, where D_{rh} is the rail head distance, $1.5\ m$.

$\phi = tan^{-1}\left(\frac{0.08}{1.5}\right) = 0.05328285$

$dz_y = sin\phi = sin(0.05328285) = 0.0532576$

$dz_z = cos\phi = cos(0.05328285) = 0.9985808$

```
#113 = IFCCURVESEGMENT(.CONTINUOUS., #119, IFCLENGTHMEASURE(0.), IFCLENGTHMEASURE(100.), #123);
#119 = IFCAXIS2PLACEMENT3D(#120, #121, #122);
#120 = IFCCARTESIANPOINT((0., 0.08, 0.));
#121 = IFCDIRECTION((0.,  0.0532576, 0.9985808));
#122 = IFCDIRECTION((1., 0., 0.));
```

4.3.2 Compute Point on Curve

[todo - look at this example carefully, the cant terms are in the Z position - should they be in the Y position?]

Compute the placement matrix for a point $s = 50\ m$ from the start of the curve segment.

Step 1 - Evaluate the parent curve at the trim start

$s_0 = 0, D(0) = 0.08\ m, D'(0) = 0.$

$\phi(0) = tan^{-1}\left(\frac{0.9985808}{0.0532576}\right) = 1.517513518$

Step 2 - Form the normalization matrix M_N

$$\mathbf{X}_0 = (1,\ 0,\ 0), \quad \mathbf{Z}_0 = (0,\ 0.0532576,\ 0.9985808)$$

$$\mathbf{Y}_0 = \mathbf{Z}_0 \times \mathbf{X}_0, \quad \mathbf{Axis}_0 = \mathbf{X}_0 \times \mathbf{Y}_0$$

$$\mathbf{Y}_0 = (0,\ 0.9985808,\ -0.0532576)$$

$$\mathbf{Axis}_0 = (0,\ 0.0532576,\ 0.9985808)$$

$$M_N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.9985808 & -0.0532576 & 0 \\ 0 & 0.0532576 & 0.9985808 & -0.08 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 - Form the curve segment placement matrix M_{CSP}

$$\mathbf{RefDir}_p = (1,\ 0,\ 0), \quad \mathbf{Axis}_p = (0,\ 0.0532576,\ 0.9985808)$$

$$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0,\ 0.0532576,\ 0.9985808), \quad \mathbf{X}_p = \mathbf{Axis}_p \times \mathbf{Y}_p = (1,\ 0,\ 0)$$

$$M_{CSP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.9985808 & 0.0532576 & 0 \\ 0 & 0.0532576 & 0.9985808 & 0.08 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 - Evaluate and map each point

$$M_{PC} = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 0.9985808 & 0.0532576 & 0 \\ 0 & 0.0532576 & 0.9985808 & 0.08 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_c = M_{CSP}\, M_N\, M_{PC}$$

$$M_c = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.9985808 & 0.0532576 & 0 \\ 0 & 0.0532576 & 0.9985808 & 0.08 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.9985808 & -0.0532576 & 0 \\ 0 & 0.0532576 & 0.9985808 & -0.08 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 0.9985808 & 0.0532576 & 0 \\ 0 & 0.0532576 & 0.9985808 & 0.08 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_c = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 0.9985808 & 0.0532576 & 0 \\ 0 & 0.0532576 & 0.9985808 & 0.08 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.4 Linear Transition

A linear transition in cant is represented with an `IfcClothoid`.

4.4.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{1}{A_0} + \frac{A_1}{|A_1^3|}s$$

$$\frac{d}{ds}D(s) = L^2\frac{A_1}{|A_1^3|}$$

$$a_0 = D_s,\ A_0 = \frac{L^{2/1}}{\sqrt[1]{a_0}}\frac{a_0}{|a_0|},\ A_0 = \frac{L^2}{a_0}$$

$$a_1 = \Delta D,\ A_1 = \frac{L^{\frac{3}{2}}}{\sqrt{|a_1|}}\frac{a_1}{|a_1|}$$

A_1 is the clothoid constant.

4.4.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a clothoid transition curve towards the left. The track banks at a constant rate resulting in a linearly deviating elevation between the rail heads. The rail head separation is 1.5 m and the cant is 160 mm.

The semantic representation of the cant alignment is

```
#64 = IFCALIGNMENTCANTSEGMENT($, $, 0., 100., 0., 0., 0.16, 0., .LINEARTRANSITION.);
```

Compute the parent curve parameters.

$D_{sl} = 0\ m \quad D_{el} = 0\ m$

$D_{sr} = 0.16\ m \quad D_{er} = 0\ m$

$D_s = \frac{(0.16+0\ m)}{2} = 0.08\ m$

$D_e = \frac{(0+0)}{2} = 0\ m$

$\Delta D = D_e - D_s = 0.0 - 0.08 = -0.08\ m$

$a_0 = D_s = 0.08\ m,\ A_0 = \frac{(100\ m)^2}{0.08\ m} = 125000\ m$

$a_1 = \Delta D = -0.08\ m \quad A_1 = \frac{(100\ m)^{\frac{3}{2}}}{\sqrt{|-0.08\ m|}}\frac{-0.08\ m}{|-0.08\ m|} = -3535.533906\ m$

The clothoid parent curve is

```
#95=IFCCARTESIANPOINT((0.,0.));
#96=IFCDIRECTION((1.,0.));
#97=IFCAXIS2PLACEMENT2D(#95,#96);
#98=IFCCLOTHOID(#97,-3535.53390593274);
```

The cant segment begins with a deviating elevation of $D_s = 0.08\ m$. $y = d = 0.08$

The slope of the cant curve is $\frac{\Delta D}{L} = \frac{-0.08}{100} = -0.0008$.

$\theta = tan^{-1}(-0.0008) = -0.0007999998$

$dx_x = cos(-0.0007999998) = 0.9999996800$

$dy_x = sin(-0.0007999998) = -0.0007999997$

The cross-slope at the start of the segment is

$\phi_s = tan^{-1}\left(\frac{D_{rh}}{D_{sr}-D_{sl}}\right) = tan^{-1}\left(\frac{1.5}{0.16-0.0}\right) = 1.464531464$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.464531464) = 0.106064981$

$dz_z = sin(\phi_s) = sin(1.464531464) = 0.994359201$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.106064981,0.994359201));
#102=IFCDIRECTION((0.9999996800,-0.0007999997,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.4.3 Compute Point on Curve

Compute the cant placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Evaluate the parent curve at the trim start

From the parent curve

$s_0 = 0,\ x(s_0) = 0$

$y(s_0) = D(0\ m) = (100\ m)^2 \left(\frac{1}{125000\ m} + \frac{(-3535.533906\ m)}{|(-3535.533906\ m)^3|}(0\ m)\right) = 0.08\ m$

$y'(0) = D'(0) = 0,\ \theta_0 = 0$

Step 2 — Form the normalization matrix M_N

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -d_0\cos\theta_0 - z_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & -d_0\sin\theta_0 - z_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.08 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (0.9999996800, -0.0007999997, 0.),\ \mathbf{Axis}_p = (0., 0.106064981, 0.994359201)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0.0007954871, 0.9943588828, -0.1060649471)$

$$M_{CSP} = \begin{bmatrix} 0.9999996800 & 0.0007954871 & 0 & 0 \\ -0.0007999997 & 0.9943588828 & 0.106064981 & 0.08 \\ 0 & -0.1060649471 & 0.994359201 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The **Axis** vector is perpendicular to the railhead cross slope line.

$\mathbf{Axis} = (0.0,\ 0.106064981,\ 0.994359201)$

$\phi_p = tan^{-1}\left(\frac{0.994359201}{0.106064981}\right) = 1.464531464$

With y to the left and z upwards, the vector is nearly vertical, pointing slightly to the left. This is consistent with a curve to the left and the right railhead being superelevated.

Step 4 — Evaluate and map each point

Evaluate the parent curve at $s = 50\ m$

$D(50\ m) = (100\ m)^2 \left(\frac{1}{125000\ m} + \frac{(-3535.533906\ m)}{|(-3535.533906\ m)^3|}(50\ m)\right) = 0.04\ m$

$$M_{PC} = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0.04 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$M_c = M_{CSP} M_N M_{PC}$

$$M_c = \begin{bmatrix} 0.999999680000 & 0.0007988711 & 0.0000424859 & 0 \\ -0.000800000085 & 0.9985884845 & 0.053107402 & 0.08 \\ 0 & -0.053107419 & 0.9985882289 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.08 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0.04 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_c = \begin{bmatrix} 0.9999996836000798 & 0.0007955209455802008 & 0.0 & 50.0 \\ -0.0007909999255054242 & 0.9985884897861665 & 0.05310743567896648 & 0.04 \\ 0.0 & -0.05310740211832019 & 0.9985888044014736 & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

$\mathbf{Axis} = (0.0,\ 0.05310743567896648,\ 0.9985888044014736)$

$\phi(50\ m) = tan^{-1}\left(\frac{0.9985888044014736}{0.05310743567896648}\right) = 1.517663895$

The **Axis** vector is closer to vertical half way through the cant segment. At the end of the cant segment, **Axis** will be $\frac{\pi}{2}$.

As a quick check, the **Axis** direction vector half way through the cant segment should be the average value. $\frac{1.464531464+\frac{\pi}{2}}{2} = 1.517663895$

4.5 Helmert Curve

Like the Helmert transition spiral, the Helmert cant sementic definition maps to two `IfcCurveSegment` instances for the geometric representation. The parent curve is `IfcSecondOrderPolynomialSpiral`.

4.5.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{1}{A_2^3} s^2 + \frac{A_1}{|A_1^3|} s + \frac{1}{A_0}$$

$$\frac{d}{ds} D(s) = L^2 \left(\frac{2s}{A_2^3} + \frac{A_1}{|A_1^3|} \right)$$

The polynomial coefficietions carry a second subscript to indicate first half 1, and second half 2. For example, A_{21} is coefficient A_2 for the first half and A_{32} is coefficient A_3 for the second half. **[todo: add this description to 2_Horizonta.md]**

In the first half of the cant transition

Constant Term:

$a_{01} = 4D_s$, $A_{01} = \frac{L^2}{|a_{o1}|} \frac{a_{01}}{|a_{o1}|}$

Linear Term:

$a_{11} = 0$, $A_{11} = \frac{L^{\frac{3}{2}}}{\sqrt{|a_{11}|}} \frac{a_{11}}{|a_{11}|} = 0$

Quadratic Term:

$a_{21} = 8f$, $A_{21} = \frac{L^{4/3}}{\sqrt[3]{|a_{21}|}} \frac{a_{21}}{|a_{21}|}$

In the second half

Constant Term:

$a_{02} = -4f + 4D_s$, $A_{02} = \frac{L^2}{|a_{02}|} \frac{a_{01}}{|a_{02}|}$

Linear Term:

$a_{12} = 16f,\ A_{12} = \frac{L^{\frac{3}{2}}}{\sqrt{|a_{12}|}} \frac{a_{12}}{|a_{12}|}$

Quadratic Term:

$a_{22} = -8f,\ A_{22} = \frac{L^{\frac{4}{3}}}{\sqrt[3]{|a_{22}|}} \frac{a_{22}}{|a_{22}|}$

4.5.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Helmert transition curve towards the left. The start cant is $0.08\ m$ and transitions to zero over $100\ m$.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.HELMERTCURVE.);
```

Compute the parent curve parameters

$$L = 100\ m$$
$$D_{sl} = 0\ m,\ D_{el} = 0\ m$$
$$D_{sr} = 0.16\ m,\ D_{er} = 0\ m$$
$$D_s = \frac{0 + 0.16\ m}{2} = 0.08\ m$$
$$D_e = \frac{0 + 0}{2} = 0.\ m$$
$$\Delta D = 0.0 - 0.08 = -0.08\ m$$
$$f = \Delta D = -0.08\ m$$

First half:

$$a_{01} = 4D_s = 4(0.08\ m) = 0.32\ m$$
$$A_{01} = \frac{L^2}{|a_{o1}|} \frac{a_{01}}{|a_{o1}|} = \frac{(100\ m)^2}{|0.32\ m|} \frac{0.32\ m}{|0.32\ m|} = 31250\ m$$
$$a_{11} = 0,\ A_{11} = 0$$
$$a_{21} = 8f = 8(-0.08\ m) = -0.64\ m$$
$$A_{21} = \frac{L^{4/3}}{\sqrt[3]{|a_{21}|}} \frac{a_{21}}{|a_{21}|} = \frac{(100\ m)^{\frac{4}{3}}}{\sqrt[3]{|-0.64\ m|}} \frac{-0.64\ m}{|-0.64\ m|} = -538.6086725\ m$$

The first half parent curve `IfcSecondOrderPolynomialSpiral` is

```
#104=IFCCARTESIANPOINT((0.,0.));
#105=IFCDIRECTION((1.,0.));
#106=IFCAXIS2PLACEMENT2D(#104,#105);
#107=IFCSECONDORDERPOLYNOMIALSPIRAL(#106,-538.60867250797071,$,31250.);
```

Determine the first half placement and trimming.

Note that the length of the first half curve is $L_1 = 50\ m$, half the length of the full Helmert transition. **[Need to do a better job explaining why L/2 when A is computed on full L]**

The deviation elevation at the start of the first half transition is

$D(0\ m) = (50\ m)^2 \left(\frac{1}{31250\ m} + \frac{1}{(-538.6086725\ m)^3}(0\ m)^2\right) = 0.08\ m$

The slope at $0\ m$ is 0.0. $dx_x = 1.0,\ dx_y = 0.0$

The cross slope is

$\phi_s = tan^{-1}\left(\frac{D_{rh}}{D_{sr} - D_{sl}}\right) = tan^{-1}\left(\frac{1.5}{0.16 - 0.0}\right) = 1.464531464$

$dy_z = cos(\phi_s) = cos(1.464531464) = 0.106064981$

$dz_z = sin(\phi_s) = sin(1.464531464) = 0.994359201$

```
#108=IFCCARTESIANPOINT((0.,0.08,0.));
#109=IFCDIRECTION((0.,0.10606498139220574,0.9943592005519288 3));
#110=IFCDIRECTION((1.,0.,0.));
#111=IFCAXIS2PLACEMENT3D(#108,#109,#110);
#112=IFCCURVESEGMENT(.CONTINUOUS.,#111,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(50.),#107);
```

Second half:

$$a_{02} = -4f + 4D_s = -4(-0.08\ m) + 4(0.08)\ m = 0.64\ m$$

$$A_{02} = \frac{L^2}{|a_{02}|}\frac{a_{01}}{|a_{02}|} = \frac{(100\ m)^2}{|0.64\ m|}\frac{0.64\ m}{|0.64\ m|} = 15625\ m$$

$$a_{12} = 16f = 16(-0.08\ m) = -1.28\ m$$

$$A_{12} = \frac{L^{\frac{3}{2}}}{\sqrt{|a_{12}|}}\frac{a_{12}}{|a_{12}|} = \frac{(100\ m)^{\frac{3}{2}}}{\sqrt{|-1.28\ m|}}\frac{-1.28\ m}{|-1.28\ m|} = -883.8834765\ m$$

$$a_{22} = -8f = -8(-0.08\ m) = 0.64\ m$$

$$A_{22} = \frac{L^{\frac{4}{3}}}{\sqrt[3]{|a_{22}|}}\frac{a_{22}}{|a_{22}|} = \frac{(100\ m)^{\frac{4}{3}}}{\sqrt[3]{|0.64\ m|}}\frac{0.64\ m}{|0.64\ m|} = 538.6086725\ m$$

The second half parent curve `IfcSecondOrderPolynomialSpiral` is

```
#113=IFCCARTESIANPOINT((0.,0.));
#114=IFCDIRECTION((1.,0.));
#115=IFCAXIS2PLACEMENT2D(#113,#114);
#116=IFCSECONDORDERPOLYNOMIALSPIRAL(#115,538.60867250797071,-883.8834764831845,15625.);
```

Determine the second half placement and trimming.

[todo: consider adding a figure that plots both parent curves. discuss that they both start at 0,0 and don't align at the mid-point. could either adjust origin of second half, or just trim and place - it's far easier to trim and place so we get the placement information from the end of the first half curve]

The starting elevation of the second half is the end elevation of the first half

$D(50\ m) = (50\ m)^2\left(\frac{1}{31250\ m} + \frac{1}{(-538.6086725\ m)^3}(50\ m)^2\right) = 0.04\ m$

The slope at $50\ m$ is $\frac{dD'}{ds} = L_1^2\left(\frac{2s}{A_2^3} + \frac{A_1}{|A_1^3|}\right) = (50\ m)^2\left(\frac{2{\cdot}50\ m}{(-538.6086725\ m)^3} + \frac{31250\ m}{|(31250\ m)^3|}\right) = -0.00159744$.

$dx_x = \frac{1}{\sqrt{(-0.00159744)^2+1}} = 0.999998724,\ dx_y = \frac{-0.00159744}{\sqrt{(-0.00159744)^2+1}} = -0.001597438$

The cross slope at the end of the first half is

$\phi(50\ m) = 1.464531464 + \left(\frac{\frac{\pi}{2}-1.464531464}{0.08\ m}\right)(0.04\ m - 0.08\ m) = 1.517663895$

$dy_z = cos(\phi_s) = cos(1.517663895) = 0.053107436$

$dz_z = sin(\phi_s) = sin(1.517663895) = 0.998588804$

The trimming begins at $50\ m$ and progresses for a length of $50\ m$ for the second half segment.

```
#117=IFCCARTESIANPOINT((50.,0.04,0.));
#118=IFCDIRECTION((0.,0.05325764291615075 3,0.99858080467782662));
#119=IFCDIRECTION((0.9999987200024576,-0.0015999979520039342,0.));
#120=IFCAXIS2PLACEMENT3D(#117,#118,#119);
#121=IFCCURVESEGMENT(.CONTINUOUS.,#120,IFCLENGTHMEASURE(50.),IFCLENGTHMEASURE(50.),#116);
```

4.5.3 Compute Point on Curve

Compute the cant placement matrix for a point 75 m from the start of the curve segment.

From the parent curve

$s_0 = 50\ m,\ x(s_0) = 50\ m$

$y(s_0) = D(50\ m) = (100\ m)^2 \left(\frac{1}{(538.6086725\ m)^3}(50\ m)^2 + \frac{-883.8834765}{|(-883.8834765\ m)^3|}(50\ m) + \frac{1}{15625\ m} \right) = 0.16\ m$

$y'(s_0) = D'(s_0) = (100\ m)^2 \left(\frac{2(50\ m)}{(538.6086725\ m)^3} + \frac{-883.8834765\ m}{|(-883.8834765\ m)^3|} \right) = -0.0064$

$\theta_0 = tan^{-1}(-0.0064) = -0.006399913$

$cos\theta_0 = cos(-0.006399913) = 0.999979521$

$sin\theta_0 = sin(-0.006399913) = -0.006399869$

Step 2 — Form the normalization matrix M_N

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -d_0\cos\theta_0 - z_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & -d_0\sin\theta_0 - z_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

[todo - look at this carefully, the MN matrix doesn't look right - this will trickle down through all the calcs]

$$M_N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.08 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir_p} = (0.9999987200024576, -0.0015999979520039342, 0.),\ \mathbf{Axis_p} = (0., 0.053257642916150753, 0.99858080467782662)$

$\mathbf{Y_p} = \mathbf{Axis_p} \times \mathbf{RefDir_p} = (0.001597727253, 0.998579530, -0.0532575749)$

$$M_{CSP} = \begin{bmatrix} 0.999998720 & 0.001597727253 & 0 & 50.0 \\ -0.001599997953 & 0.998579530 & 0.0532576429 & 0.04 \\ 0 & -0.0532575749 & 0.998580805 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Evaluate and map each point

A point 75 m from the start of the segment is in the second half of the helmert curve. Evaluate the second half parent curve at $s = 75\ m - 50\ m = 25\ m$

$D(25\ m) = (100\ m)^2 \left(\frac{1}{(538.6086725\ m)^3}(25\ m)^2 + \frac{-883.8834765}{|(-883.8834765\ m)^3|}(25\ m) + \frac{1}{15625\ m} \right) = 0.36\ m$

$$M_{PC} = \begin{bmatrix} 1 & 0 & 0 & 25 \\ 0 & 1 & 0 & 0.36 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$M_c = M_{CSP} M_N M_{PC}$

$$M_c = \begin{bmatrix} 0.999998720 & 0.001597727253 & 0 & 50.0 \\ -0.001599997953 & 0.998579530 & 0.0532576429 & 0.04 \\ 0 & -0.0532575749 & 0.998580805 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} [I = NOT\ CORRECT] \begin{bmatrix} 1 & 0 & 0 & 25 \\ 0 & 1 & 0 & 0.36 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

[todo - verify matrix multiplication - final result is correct from IfcOpenShell]

$$M_c = \begin{bmatrix} 0.999999684 & 0.000799928774 & 0 & 75.0 \\ -0.000795461553 & 0.999910964 & 0.0133204468 & 0.01 \\ 0 & -0.0133203149 & 0.999911281 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$\mathbf{Axis} = (0.0,\ 0.0133204468,\ 0.999911281)$

$\phi(75\ m) = tan^{-1}\left(\frac{0.999911281}{0.0133204468}\right) = 1.557475486$

The **Axis** vector is closer to vertical 75% way through the cant segment. At the end of the cant segment, **Axis** will be $\frac{\pi}{2}$.

4.6 Bloss Curve

A Bloss transition in cant is represented with an `IfcThirdOrderPolynomialSpiral`.

4.6.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{A_3}{\left|A_3^5\right|}s^3 + \frac{1}{A_2^3}s^2 + \frac{A_1}{2\left|A_1^3\right|}s + \frac{1}{A_0}$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{3A_3}{\left|A_3^5\right|}s^2 + \frac{2}{A_2^3}s + \frac{A_1}{2\left|A_1^3\right|}\right)$$

Constant Term

$a_0 = D_s,\ A_0 = \frac{L^{\frac{2}{1}}}{\sqrt[0]{|a_0|}}\frac{a_0}{|a_0|} = \frac{L^2}{|a_0|}\frac{a_0}{|a_0|}$

Linear Term

$a_1 = 0,\ A_1$

Quadratic Term

$a_2 = 3f,\ A_2 = \frac{L^{\frac{4}{3}}}{\sqrt[3]{|a_2|}}\frac{a_2}{|a_2|}$

Cubic Term

$a_3 = -2f\ A_3 = \frac{L^{\frac{5}{4}}}{\sqrt[4]{|a_3|}}\frac{a_3}{|a_3|}$

4.6.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Bloss transition curve towards the left. The start cant is $0.08\ m$ and transitions to zero over $100\ m$.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.BLOSSCURVE.);
```

Compute the parent curve parameters

$D_s = \frac{0+0.16}{2} = 0.08\ m,\ D_e = \frac{0+0\ m}{2} = 0.\ m,\ \Delta D = 0.0 - 0.16 = -0.08\ m$

$f = \Delta D = -0.08m$

Constant Term

$a_0 = -0.08\ m,\ A_0 = \frac{(100\ m)^2}{|-0.08\ m|}\frac{-0.08\ m}{|-0.08\ m|} = 125000\ m$

Linear Term

$A_1 = 0\ m$

Quadratic Term

$a_2 = 3(-0.08\ m) = -0.24\ m,\ A_2 = \frac{(100\ m)^{\frac{4}{3}}}{\sqrt[3]{|-0.24\ m|}}\left(\frac{-0.24\ m}{|-0.24\ m|}\right) = -746.9007911\ m$

Cubic Term

$a_3 = -2f = -2(-0.08\ m) = 0.16\ m,\ A_3 = \frac{(100\ m)^{\frac{5}{4}}}{\sqrt[4]{|0.16\ m|}}\left(\frac{0.16\ m}{|0.16\ m|}\right) = 500\ m$

The parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCTHIRDORDERPOLYNOMIALSPIRAL(#98,500.00000000000006,-746.90079109286057,$,125000.);
```

The cant segment begins with a deviating elevation of $D_s = 0.08\ m$. $y = d = 0.08\ m$

The slope of the cant curve is $\frac{\Delta D}{L} = \frac{-0.08}{100} = -0.0008$.

$\theta = tan^{-1}(-0.0008) = -0.0007999998$

$dx_x = cos(-0.0007999998) = 0.9999996800$

$dy_x = sin(-0.0007999998) = -0.0007999997$

The cross-slope at the start of the segment is

$\phi_s = tan^{-1}\left(\frac{D_{rh}}{D_{sr}-D_{sl}}\right) = tan^{-1}\left(\frac{1.5}{0.16-0.0}\right) = 1.464531464$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.464531464) = 0.106064981$

$dz_z = sin(\phi_s) = sin(1.464531464) = 0.994359201$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.10606498139220574,0.99435920055192883));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.6.3 Compute Point on Curve

Compute the cant placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Evaluate the parent curve at the trim start

From the parent curve

$s_0 = 0,\ x(s_0) = 0$

$y(s_0) = D(0\ m) = (100\ m)^2\left(\frac{500\ m}{|(500\ m)^5|}(0\ m)^3 + \frac{1}{(-746.9007911\ m)^3}(0\ m)^2 + \frac{1}{125000\ m}\right) = 0.08\ m$

$y'(0) = D'(0) = 0,\ \theta_0 = 0$

Step 2 — Form the normalization matrix M_N

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -d_0\cos\theta_0 - z_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & -d_0\sin\theta_0 - z_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.08 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (1, 0, 0.),\ \mathbf{Axis}_p = (0., 0.106064981, 0.994359201)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0, 0.9943592, -0.10606498)$

$$M_{CSP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.9943592 & 0.106064981 & 0.08 \\ 0 & -0.10606498 & 0.9943592 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The **Axis** vector is perpendicular to the railhead cross slope line.

$\mathbf{Axis} = (0.0,\ 0.106064981,\ 0.9943592)$

$\phi_p = tan^{-1}\left(\frac{0.9943592}{0.106064981}\right) = 1.464531464$

With y to the left and z upwards, the vector is nearly vertical, pointing slightly to the left. This is consistent with a curve to the left and the right railhead being superelevated.

Step 4 — Evaluate and map each point

Evaluate the parent curve at $s = 50\ m$

$D(50\ m) = (100\ m)^2 \left(\frac{500\ m}{|(500\ m)^5|}(50\ m)^3 + \frac{1}{(-746.9007911\ m)^3}(50\ m)^2 + \frac{1}{125000\ m}\right) = 0.04\ m$

$$M_{PC} = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0.04 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$M_c = M_{CSP} M_N M_{PC}$

$$M_c = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.9943592 & 0.106064981 & 0.08 \\ 0 & -0.10606498 & 0.9943592 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.08 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0.04 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_c = \begin{bmatrix} 0.999999280 & 0.00119830570 & 0.0 & 50.0 \\ -0.00119999914 & 0.998588085 & 0.0531073592 & 0.04 \\ 0 & 0 & 0.9985888041 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.7 Cosine Curve

A Cosine transition in cant is represented with an `IfcCosineSpiral`.

4.7.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$\frac{D(s)}{L^2} = \frac{1}{A_0} + \frac{1}{A_1}\cos\left(\pi\frac{s}{L}\right)$

$\frac{d}{ds}D(s) = L^2\left(-\frac{\pi}{L}\sin\left(\pi\frac{s}{L}\right)\right)$

Constant term,

$A_0 = \frac{L^2}{D_s + \frac{1}{2}\Delta D}$

Cosine term,

$A_1 = \frac{L^2}{-\frac{1}{2}\Delta D}$

4.7.2 Semantic Definition to Geometry Mapping

Consider and alignment segment that has a Cosine transition curve towards the left. The start cant is $0.08\ m$ and transitions to zero over $100\ m$.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.COSINECURVE.);
```

Compute the parent curve parameters

$D_s = \frac{0+0.16}{2} = 0.08\ m,\ D_e = \frac{0+0\ m}{2} = 0.\ m,\ \Delta D = 0.0\ m - 0.08\ m = -0.08\ m$

$A_0 = \frac{(100\ m)^2}{0.08\ m + \frac{1}{2}(-0.08\ m)} = 250000\ m$

$A_1 = \frac{(100\ m)^2}{-\frac{1}{2}(-0.08\ m)} = 250000\ m$

The parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCCOSINESPIRAL(#98,250000.,250000.);
```

The cant segment begins with a deviating elevation of $D_s = 0.08\ m$. $y = d = 0.08\ m$

The slope of the cant curve is $\frac{\Delta D}{L} = \frac{-0.08}{100} = -0.0008$.

$\theta = tan^{-1}(-0.0008) = -0.0007999998$

$dx_x = cos(-0.0007999998) = 0.9999996800$

$dy_x = sin(-0.0007999998) = -0.0007999997$

The cross-slope at the start of the segment is

$\phi_s = tan^{-1}\left(\frac{D_{rh}}{D_{sr} - D_{sl}}\right) = tan^{-1}\left(\frac{1.5}{0.16 - 0.0}\right) = 1.464531464$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.464531464) = 0.106064981$

$dz_z = sin(\phi_s) = sin(1.464531464) = 0.994359201$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.10606498139220574,0.99435920055192883));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.7.3 Compute Point on Curve

Compute the cant placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Evaluate the parent curve at the trim start

From the parent curve

$s_0 = 0,\ x(s_0) = 0$

$y(s_0) = D(0\ m) = (100\ m)^2\left(\frac{1}{250000\ m} + \frac{1}{250000\ m}\cos\left(\pi\frac{0\ m}{100\ m}\right)\right) = 0.08\ m$

$y'(0) = D'(0) = (100\ m)^2\left(-\frac{\pi}{100\ m}\sin\left(\pi\frac{0\ m}{100\ m}\right)\right) = 0$

$\theta_0 = 0$

Step 2 — Form the normalization matrix M_N

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -d_0\cos\theta_0 - z_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & -d_0\sin\theta_0 - z_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.08 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (1, 0, 0.),\ \mathbf{Axis}_p = (0., 0.106064981, 0.994359201)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0, 0.9943592, -0.10606498)$

$$M_{CSP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.9943592 & 0.106064981 & 0.08 \\ 0 & -0.10606498 & 0.9943592 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The **Axis** vector is perpendicular to the railhead cross slope line.

$\mathbf{Axis} = (0.0,\ 0.106064981,\ 0.9943592)$

$\phi_p = tan^{-1}\left(\frac{0.9943592}{0.106064981}\right) = 1.464531464$

With y to the left and z upwards, the vector is nearly vertical, pointing slightly to the left. This is consistent with a curve to the left and the right railhead being superelevated.

Step 4 — Evaluate and map each point

Evaluate the parent curve at $s = 50\ m$

$D(50\ m) = (100\ m)^2\left(\frac{1}{250000\ m} + \frac{1}{250000\ m}\cos\left(\pi\frac{50\ m}{100\ m}\right)\right) = 0.04\ m$

$$M_{PC} = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0.04 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$M_c = M_{CSP}M_N M_{PC}$

$$M_c = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.9943592 & 0.106064981 & 0.08 \\ 0 & -0.10606498 & 0.9943592 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.08 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0.04 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_c = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 0.9985888041 & 0.0531074357 & 0.04 \\ 0 & -0.05310743572 & 0.998588804 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

[todo - check the matrix multiplication the Y and Z columns can]

4.8 Sine Curve

A Sine transition in cant is represented with an `IfcSineSpiral`

4.8.1 Parent Curve Parametric Equations

The deviation elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{1}{A_0} + \frac{A_1}{|A_1|}\left(\frac{1}{A_1}\right)^2 s + \frac{1}{A_2}\sin\left(2\pi\frac{s}{L}\right)$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{A_1}{|A_1|}\left(\frac{1}{A_1}\right)^2 + \frac{2\pi}{LA_2}\cos\left(2\pi\frac{s}{L}\right)\right)$$

Constant term: $A_0 = \frac{L^2}{D_s}$

Linear term: $A_1 = \frac{L^{\frac{3}{2}}}{\sqrt{\Delta D}}$

Sine term: $A_2 = \frac{L^2}{-\frac{1}{2\pi}\Delta D}$

4.8.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Sine transition curve towards the left. The start cant is $0.08\ m$ and transitions to zero over $100\ m$.

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.16,0.,.SINECURVE.);
```

Compute the paraent curve parameters

$D_s = \frac{0+0.16}{2} = 0.08\ m,\ D_e = \frac{0+0\ m}{2} = 0.\ m,\ \Delta D = 0.0\ m - 0.08\ m = -0.08\ m$

Constant term: $A_0 = \frac{L^2}{D_s} = \frac{(100\ m)^2}{0.08\ m} = 125000\ m$

Linear term: $A_1 = \frac{L^{\frac{3}{2}}}{\sqrt{\Delta D}} = \frac{(100\ m)^{\frac{3}{2}}}{\sqrt{|-0.08\ m|}}\frac{-0.08\ m}{|-0.08\ m|} = -3535.533906\ m$

Sine term: $A_2 = \frac{L^2}{-\frac{1}{2\pi}\Delta D} = \frac{(100\ m)^2}{-\frac{1}{2\pi}(-0.08\ m)} = 78539.81634\ m$

The parent curve is

```
#96=IFCCARTESIANPOINT((0.,0.));
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCSINESPIRAL(#98,785398.16339744814,-3535.533905932738,125000.);
```

The cant segment begins with a deviating elevation of $D_s = 0.08\ m$. $y = d = 0.08\ m$

The slope of the cant curve is $\frac{\Delta D}{L} = \frac{-0.08}{100} = -0.0008.$

$\theta = tan^{-1}(-0.0008) = -0.0007999998$

$dx_x = cos(-0.0007999998) = 0.9999996800$

$dy_x = sin(-0.0007999998) = -0.0007999997$

The cross-slope at the start of the segment is

$\phi_s = tan^{-1}\left(\frac{D_{rh}}{D_{sr}-D_{sl}}\right) = tan^{-1}\left(\frac{1.5}{0.16-0.0}\right) = 1.464531464$

The cross slope orientation is

$dy_z = cos(\phi_s) = cos(1.464531464) = 0.106064981$

$dz_z = sin(\phi_s) = sin(1.464531464) = 0.994359201$

```
#100=IFCCARTESIANPOINT((0.,0.08,0.));
#101=IFCDIRECTION((0.,0.10606498139220574,0.9943592005519288 3));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.8.3 Compute Point on Curve

Compute the cant placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Evaluate the parent curve at the trim start

From the parent curve

$s_0 = 0,\ x(s_0) = 0$

$y(s_0) = D(0\ m) = \frac{(100\ m)^2}{125000\ m} + \left(\frac{-3535.533906\ m}{|-3535.533906\ m|}\right)\left(\frac{1}{-3535.533906\ m}\right)^2(0\ m)(100\ m)^2 + \frac{(100\ m)^2}{78539.81634\ m}\sin\left(2\pi\frac{0\ m}{100\ m}\right) = 0.08\ m$

$y'(0) = D'(0) = (100\ m)^2\left(\frac{-3535.533906}{|-3535.533906|}\left(\frac{1}{-3535.533906}\right)^2 + \frac{2\pi}{(100\ m)(78539.81634\ m)}\cos\left(2\pi\frac{0\ m}{100\ m}\right)\right) = 0$

$\theta_0 = 0$

Step 2 — Form the normalization matrix M_N

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -d_0\cos\theta_0 - z_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & -d_0\sin\theta_0 - z_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.08 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (1, 0, 0.),\ \mathbf{Axis}_p = (0., 0.106064981, 0.994359201)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0, 0.9943592, -0.10606498)$

$$M_{CSP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.9943592 & 0.106064981 & 0.08 \\ 0 & -0.10606498 & 0.9943592 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The **Axis** vector is perpendicular to the railhead cross slope line.

$\mathbf{Axis} = (0.0,\ 0.106064981,\ 0.9943592)$

$\phi_p = tan^{-1}\left(\frac{0.9943592}{0.106064981}\right) = 1.464531464$

With y to the left and z upwards, the vector is nearly vertical, pointing slightly to the left. This is consistent with a curve to the left and the right railhead being superelevated.

Step 4 — Evaluate and map each point

Evaluate the parent curve at $s = 50\ m$

$D(50\ m) = \frac{(100\ m)^2}{125000\ m} + \left(\frac{-3535.533906\ m}{|-3535.533906\ m|}\right)\left(\frac{1}{-3535.533906\ m}\right)^2(50\ m)(100\ m)^2 + \frac{(100\ m)^2}{78539.81634\ m}\sin\left(2\pi\frac{50\ m}{100\ m}\right) = 0.04\ m$

$$M_{PC} = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0.04 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$M_c = M_{CSP}M_NM_{PC}$

$$M_c = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.9943592 & 0.106064981 & 0.08 \\ 0 & -0.10606498 & 0.9943592 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.08 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0.04 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_c = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 0.9985888041 & 0.0531074357 & 0.04 \\ 0 & -0.05310743572 & 0.998588804 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

[todo - check the matrix multiplication the Y and Z columns can]

4.9 Viennese Bend

A Viennese Bend transition in cant is represented with an `IfcSeventhOrderPolynomialSpiral`.

4.9.1 Parent Curve Parametric Equations

The deviating elevation and its rate of change are given by the following equations.

$$\frac{D(s)}{L^2} = \frac{A_7}{|A_7^9|}s^7 + \frac{1}{A_6^7}s^6 + \frac{A_5}{|A_5^7|}s^5 + \frac{1}{A_4^5}s^4 + \frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2 + \frac{A_1}{2\,|A_1^3|}s + \frac{1}{A_0}$$

$$\frac{d}{ds}D(s) = L^2\left(\frac{7A_7}{|A_7^9|}s^6 + \frac{6}{A_6^7}s^5 + \frac{5A_5}{|A_5^7|}s^4 + \frac{4}{A_4^5}s^3 + \frac{3A_3}{|A_3^5|}s^2 + \frac{2}{A_2^3}s + \frac{A_1}{2\,|A_1^3|}\right)$$

$$D_s = \frac{D_{sl} + D_{sr}}{2},\ D_e = \frac{D_{el} + D_{er}}{2},\ \Delta D = D_e - D_s$$

$$f = \Delta D$$

Constant Term

$$a_0 = D_1, A_0 = \frac{L^2}{|a_0|}\frac{a_0}{|a_0|}$$

Linear Term

$$a_1 = 0, A_1 = \frac{L^{\frac{3}{2}}}{\sqrt{|a_1|}}\frac{a_1}{|a_1|}$$

Quadratic Term

$$a_2 = 0, A_2 = \frac{L^{\frac{4}{3}}}{\sqrt[3]{|a_2|}}\frac{a_2}{|a_2|}$$

Cubic Term

$$a_3 = 0, A_3 = \frac{L^{\frac{5}{4}}}{\sqrt[4]{|a_3|}}\frac{a_3}{|a_3|}$$

Quartic Term

$$a_4 = 35f, A_4 = \frac{L^{\frac{6}{5}}}{\sqrt[5]{|a_4|}}\frac{a_4}{|a_4|}$$

Quintic Term

$$a_5 = -84f, A_5 = \frac{L^{\frac{7}{6}}}{\sqrt[6]{|a_5|}}\frac{a_5}{|a_5|}$$

Sextic Term

$$a_6 = 70f, A_6 = \frac{L^{\frac{8}{7}}}{\sqrt[7]{|a_6|}}\frac{a_6}{|a_6|}$$

Septic Term

$$a_7 = -20f, A_7 = \frac{L^{\frac{9}{8}}}{\sqrt[8]{|a_7|}} \frac{a_7}{|a_7|}$$

4.9.2 Semantic Definition to Geometry Mapping

Consider an alignment segment that has a Viennese Bend transition curve **???**. **[todo - finish the description]**

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.10000000000000001,0.029999999999999999,.VIENNESEBEND.);
```

Compute the parent curve parameters.

$$D_{sl} = 0.0\ m$$
$$D_{el} = 0.0\ m$$
$$D_{sr} = 0.1\ m$$
$$D_{er} = 0.03\ m$$
$$D_s = \frac{0\ m + 0.1\ m}{2} = 0.05\ m, D_e = \frac{0\ m + 0.03\ m}{2} = 0.015\ m,\ \Delta D = 0.015 - 0.05 = -0.035\ m$$
$$f = \Delta D = -0.035m$$

Constant Term

$$a_0 = 0.05\ m, A_0 = \frac{(100\ m)^2}{|0.05\ m|} \frac{0.05\ m}{|0.05\ m|} = 20000\ m$$

Linear Term

$$a_1 = 0, A_1 = 0\ m$$

Quadratic Term

$$a_2 = 0, A_2 = 0\ m$$

Cubic Term

$$a_3 = 0, A_3 = 0\ m$$

Quartic Term

$$a_4 = 35(-0.035\ m) = -1.225\ m, A_4 = \frac{(100\ m)^{\frac{6}{5}}}{\sqrt[5]{|-1.225\ m|}} \frac{-1.225\ m}{|-1.225\ m|} = -241.1974890085123\ m$$

Quintic Term

$$a_5 = -84(-0.035\ m) = 2.94\ m, A_5 = \frac{(100\ m)^{\frac{7}{6}}}{\sqrt[6]{|2.94\ m|}} \frac{2.94\ m}{|2.94\ m|} = 180.0012184608678\ m$$

Sextic Term

$$a_6 = 70(-0.035\ m) = -2.45\ m, A_6 = \frac{(100\ m)^{\frac{8}{7}}}{\sqrt[7]{|-2.45\ m|}} \frac{-2.45\ m}{|-2.45\ m|} = -169.87095595653895\ m$$

Septic Term

$$a_7 = -20(-0.035\ m) = 0.7\ m, A_7 = \frac{(100\ m)^{\frac{9}{8}}}{\sqrt[8]{|0.7\ m|}} \frac{0.7\ m}{|0.7\ m|} = 185.93568367635672\ m$$

The parent curve is

```
#97=IFCDIRECTION((1.,0.));
#98=IFCAXIS2PLACEMENT2D(#96,#97);
#99=IFCSEVENTHORDERPOLYNOMIALSPIRAL(#98,185.93568367635649,-169.87095595653892,180.00121846086768,-241.19748900851218,$,$,$,200000.);
```

The cant segment begins with a deviating elevation of $D_s = 0.05\ m$.

[todo - fix this everywhere.... the slope is the slope equation evaluated at 0.0. for this particular example the RefDir is (1,0,0), not what is calculated here - the DeltaD/L only applies to the ifcline case]

The slope of the cant curve is $\frac{\Delta D}{L} = \frac{-0.035}{100} = -0.00035$

$\theta = tan^{-1}(-0.00035) = -0.00035$

$dx_x = cos(\theta) = 0.999999939$

$dx_y = sin(\theta) = -0.00035$

The cross-slope at the start of the segment is

$\phi_s = tan^{-1}\left(\frac{D_{rh}}{D_{sr}-Dsl}\right) = tan^{-1}\left(\frac{1.5}{0.1-0.0}\right) = 1.504228163$

The cross slope orientation is

$dy_z = cos(\theta_s) = 0.066519011$

$dz_z = sin(\theta_s) = 0.997785158$

```
#100=IFCCARTESIANPOINT((0.,0.050000000000000003,0.));
#101=IFCDIRECTION((0.,0.066519010523773944,0.99778515785660904));
#102=IFCDIRECTION((1.,0.,0.));
#103=IFCAXIS2PLACEMENT3D(#100,#101,#102);
#104=IFCCURVESEGMENT(.DISCONTINUOUS.,#103,IFCLENGTHMEASURE(0.),IFCLENGTHMEASURE(100.),#99);
```

4.9.3 Compute Point on Curve

Compute the cant placement matrix for a point 50 m from the start of the curve segment.

Step 1 — Evaluate the parent curve at the trim start

From the parent curve

$s_0 = 0,\ x(s_0) = 0$

$y(s_0) = D(0\ m) = (100\ m)^2\left(\frac{185.93568367635672\ m}{|(185.93568367635672\ m)^9|}(0\ m)^7 + \frac{1}{(-169.87095595653895\ m)^7}(0\ m)^6 + \frac{180.0012184608678\ m}{|(180.0012184608678\ m)^7|}(0\ m)^5 + \frac{1}{(-241.1974890085123\ m)^5}(0\ n$

$y'(0) = D'(0) = 0,\ \theta_0 = 0$

Step 2 — Form the normalization matrix M_N

$$M_N = \begin{bmatrix} \cos\theta_0 & \sin\theta_0 & 0 & -d_0\cos\theta_0 - z_0\sin\theta_0 \\ -\sin\theta_0 & \cos\theta_0 & 0 & -d_0\sin\theta_0 - z_0\cos\theta_0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.5 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3 — Form the curve segment placement matrix M_{CSP}

$\mathbf{RefDir}_p = (1, 0, 0.),\ \mathbf{Axis}_p = (0., 0.06651901, 0.99778516)$

$\mathbf{Y}_p = \mathbf{Axis}_p \times \mathbf{RefDir}_p = (0, 0.99778516, -0.06651901)$

$$M_{CSP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.99778516 & 0.06651901 & 0.05 \\ 0 & -0.06651901 & 0.99778516 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 4 — Evaluate and map each point

Evaluate the parent curve at $s = 50\ m$

$$D(50\ m) = (100\ m)^2 \left(\frac{185.93568367635672\ m}{|(185.93568367635672\ m)^9|}(50\ m)^7 + \frac{1}{(-169.87095595653895\ m)^7}(50m)^6 + \frac{180.0012184608678\ m}{|(180.0012184608678\ m)^7|}(50\ m)^5 + \frac{1}{(-241.1974890085123\ m)^5}(50\ m)^4 \right.$$

$$M_{PC} = \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0.4825 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$M_c = M_{CSP} M_N M_{PC}$

$$M_c = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.99778516 & 0.06651901 & 0.05 \\ 0 & -0.06651901 & 0.99778516 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.5 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 50 \\ 0 & 1 & 0 & 0.4825 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_c = \begin{bmatrix} 1 & 0 & 0 & 50.0 \\ 0 & 0.999445843 & 0.0332779172 & 0.0325 \\ 0 & -0.0332779270 & 0.999446136 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4.10 Combined 3D

[do a full cant example]

4.11 Deviation from EnrichIfc4x3 Reference Implementation

The cant calculations in this section deviate from the ones performed by the EnrichIfc4x3 reference implementation. The calculations in the reference implementation, published at [IFC-Rail-Unit-Test-Reference-Code/EnrichIFC4x3/EnrichIFC4x3/business2geometry at master · bSI-RailwayRoom/IFC-Rail-Unit-Test-Reference-Code (github.com)](), yield coefficients for sine, cosine, and polynomial spirals that are not in units of length. The IFC specification is clear that the coefficients have units of length and are represented by `IfcLengthMeasure`.

The error in the EnrichIfc4x3 reference implementation is illustrated by way of an example. Consider the Bloss Curve example [GENERATED__CantAlignment_BlossCurve_100.0_1000_300_1_Meter.ifc]().

The semantic definition of the cant transition segment is

```
#64=IFCALIGNMENTCANTSEGMENT($,$,0.,100.,0.,0.,0.,0.16,.BLOSSCURVE.);
```

The EnrichIfc4x3 reference implementation maps the semantic definition to the geometric representation as follows:

$D_s = \frac{0+0}{2} = 0\ m,\ D_e = \frac{0+0.16}{2} = 0.08\ m,\ \Delta D = 0.08 - 0 = 0.08\ m,\ f = \Delta D = 0.08\ m$

Quadratic Term:

$a_2 = 3f = 3(0.08\ m) = 0.24\ m$

$A_2 = \frac{(100\ m)}{\sqrt[3]{|0.24\ m|}} \left(\frac{0.24\ m}{|0.24\ m|} \right) = 160.917897\ m^{2/3}$

Cubic Term:

$a_3 = -2f = -0.016\ m$

$A_3 = \frac{100\ m}{\sqrt[4]{|-0.16\ m|}} \frac{-0.16\ m}{|-0.16\ m|} = -158.113883\ m^{3/4}$

Notice that the polynomial coefficents do not have whole length units (e.g. the units aren't m).

The mapped geometric representation is

```
#127 = IFCTHIRDORDERPOLYNOMIALSPIRAL(#128, -158.113883008419, 160.914897434272, $, $);
```

Recalling from Bloss Curve example above,

Quadratic Term

$$A_2 = \frac{(100m)^{\frac{4}{3}}}{\sqrt[3]{|0.24m|}}\left(\frac{0.24m}{|0.24m|}\right) = 746.9007911\ m$$

Cubic Term

$$A_3 = \frac{(100\ m)^{\frac{5}{4}}}{\sqrt[4]{|-0.16\ m|}}\left(\frac{-0.16\ m}{|-0.16\ m|}\right) = -500\ m$$

The polynomial coefficients have units of length as required by IFC.

The resulting geometric representation is

```
#127=IFCTHIRDORDERPOLYNOMIALSPIRAL(#98,-500.,746.900791092861,$,$);
```

Table 4.10-1 compares the third order polynomal terms.

Polynomial Term	**EnrichIfc4x3**	**This Guide**
Constant	0.0 (unitless)	$0\ m$
Linear	$0\ m^{1/2}$	$0\ m$
Quadratic	$160.91789\ m^{2/3}$	$746.90078\ m$
Cubic	$-158.113883\ m^{3/4}$	$-500\ m$

Table 4.10-1 — Comparison of polynomial coefficients

The EnrichIfc4x3 reference implementation computes the correct value for the deviating elevation. This is because there is a compensating error in the implementation. The deviating elevation is computed as

$$D(s) = L\left(\frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2\right)$$

Evaluated at $s = 50\ m$

$$D(50\ m) = (100\ m)\left(\frac{-158.113883\ m^{3/4}}{|(-158.113883\ m^{3/4})^5|}(100\ m)^3 + \frac{1}{(160.914897434272\ m^{2/3})^3}(100\ m)^2\right) = (100\ m)(-0.0002 + 0.0006) = 0.04\ m$$

The deviating elevation as computed in this guide is

$$D(s) = L^2\left(\frac{A_3}{|A_3^5|}s^3 + \frac{1}{A_2^3}s^2\right)$$

Notice that the first term is L^2, not L as in the reference implementation.

$$D(50\ m) = (100\ m)^2\left(\frac{-500\ m}{|(-500\ m)^5|}(100\ m)^3 + \frac{1}{(746.90079\ m)^3}(100\ m)^2\right) = (100\ m)^2(-0.000002\ m^{-1} + 0.000006\ m^{-1}) = 0.04\ m$$

Both approaches give the same result, however, a serious problem emerges if the EnrichIfc4x3 semantic to geometry mapping is mixed with the deviating elevation equation from this guide and visa-versa. Table 4.10-2 compares the results for all the combinations of the sematic to geometry mapping and deviation elevation computations for the Bloss curve evaluated at $s = 50\ m$. The deviating elevation can be incorrect proportional (or inversely proportional) to the length of the segment (100 or 1/100 in this example).

Mapping	$D(s)$ **Equation**	$D(50\ m)$
EnrichIfc4x3	EnrichIfc4x3	$0.04\ m$
This Guide	This Guide	$0.04\ m$
EnrichIfc4x3	This Guide	$4.0\ m$
This Guide	EnrichIfc4x3	$0.0004\ m$

Table 4.10-2 — Comparison of deviation elevation results

Section 5.0 - Linear Placement

outline:

- describe the concept of object placement in ifc.
- introduce the idea of linear placement for horizontal construction projects. describe locating a bridge pier on an alignment using 2D like fron traditional plans and then a drain inlet in 3d with station, offset and elevation. add a figure to illustrate.
- discuss how linear placement is accomplished with ifc. this will include distance along diretrix with explaination of stationing which is covered in a different section, lateral and vertical offset, longitudinal offset for otherwise unreachable point (include a figure to illustrate), and the placement coordinate system with discussion of uncertaintly frim the spec on default axis and refdirection.
- discuss linear placement along ifcoffsetcurvebydistances
- review the iso 19148 on linear placement to see if there is other important concepts to discuss. one could be milepost system compared to stationing. see https://ifc43-docs.standards.buildingsmart.org/IFC/RELEASE/IFC4x3/HTML/lexical/Pset_LinearReferencingMethod.htm.

Many elements in the built environment for infrastructure works are located relative to an alignment. The IFC class `IfcLinearPlacement` accomplishes this type of object placement.

Generally alignments are defined in the project coordinate system. To indicate that a linear placement is relative to the alignment, the `PlacementRelTo` attribute is omitted indicating that the placement is relative to the start of the basis curve of the alignment.

The `IfcLinearPlacement.RelativePlacement` attribute places an object. RelativePlacement is an `IfcAxis2PlacementLinear`.

`IfcAxis2PlacementLinear.Location` is an `IfcPoint`, but is constrained to be an `IfcPointByDistanceExpression` by a where rule. This means that the placement location is measured along the basis curve. For a 3D alignment curve such as `IfcGradientCurve` or `IfcSegmentedReferenceCurve`, the basis curve is `IfcCompositeCurve` representing the projection of the 3D curve onto a 2D horizontal plane.

5.1 General

`IfcAxis2PlacementLinear` has optional `Axis` and `RefDirection` attributes. When omitted, `RefDirection` is the tangent to the 3D curve at Location.

Unfortunately, `Axis` is not clearly defined in the IFC schema documentation (See Default value for `IfcAxis2PlacementLinear.Axis` is not defined · Issue #125 · buildingSMART/IFC4.x-IF).

The most logical default `Axis` is "perpendicular" to the `RefDirection` in the general direction of global Z-axis. However, the most common interpetation is that `Axis` is up (0,0,1).

When `Axis` and `RefDirection` are not provided, the local coordinate system can be computed as follows:

1. Set Axis = Z = (0,0,1)
2. Compute the tangent vector to the 3D curve. This is the RefDirection.
3. Y = cross product of Z and RefDirection.
4. X = cross product of Y and Z.

X, Y, Z are orthoginal axes.

5.2 Linear Placement along IfcOffsetCurveByDistances

Linear placement along an `IfcOffsetCurveByDistances` is not precise. `IfcOffsetCurveByDistances` is an interpoloated curve defined by offsets from a basis curve. The offset values are linearly interpolated forming a segmented curve. The length of the offset curve is approximate and depends on the frequency of the interpoloated points along the curve. For this reason, `IfcAxis2PlacementLinear.Location.DistanceAlong` cannot be precisely determined.

todo:

- Need to discuss that for surface and horizontal section solid, the offset curves must be associated with an IfcAlignment or the IFC model gets flagged by the validation service. Even though an IfcOffsetCurveByDistance is related to a surface or solid, which relates to a rooted entity, the validation service doesn't recognize this. This leads to having dummy alignments and each of those alignments needs to ahve stationing defined otherwise more validation service warnings

Section 6.0 - Offset Curves

Offset curves are very common in infrastructure geometry. Some examples are:

- Edge of pavement
- Bridge girder centerlines
- Lane striping
- Utility centerline

`IfcOffsetCurveByDistance` defines the geometry of offset curves. It has two attributes:

- BasisCurve : The curve from which offsets are measured
- OffsetValues: Defines the offsets along the curve

A single offset value indicates a constant offset over the entire length of the curve.

If the offsets do not span the full length of the curve, the first and last offset are implicitly continued to the head and tail of the basis curve, respectively.

The `IfcOffsetCurveByDistance.OffsetValues` are of type `IfcPointByDistanceExpression`. `IfcOffsetCurveByDistance.BasisCurve` and `IfcPointByDistanceExpression.BasisCurve` must reference the same curve.

The `IfcOffsetCurveByDistance` geometry is derived by interpoloating the `OffsetValues`.

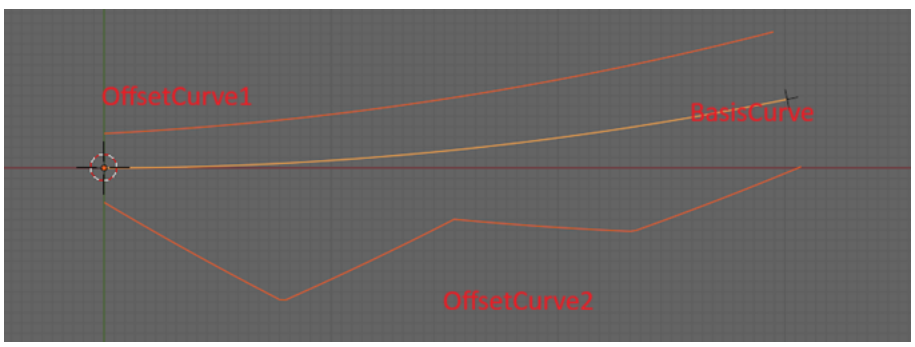

Figure 6.0-1 Offset curves

Figure 6.0-1 shows two offset curves that share a single basis curve. Offset curve 1 has different offset values defined at the start and the end of the basis curve. The start offset is 10ft and the end offset is 20ft. The basis curve is 100 ft long. The point on Offset curve 1 that corresponds to the mid-point of the Basis Curve is determined by evaluating the basis curve at 50ft, then apply the linearly interpolated offsedt of 15ft.

Offset curve 2 has several offsets. Again, linear interpolotation is used to determine the offset from the basis curve to the offset curve. Linear interpolotation is used between each pair of consequitive offset points.

Example model: IfcOffsetCurveByDistances.ifc

outline:

- drawing on the definition of stationing in chapter 5, describe how stationing is defined in ifc.
- discuss stationing breaks, both gap and overlap, icnluding why there may be breaks because of realignment of a road. discuss how this relates to and influences distance along to diretrix
- discuss what it means to inform on the position of something as semantic information
- introduce ifcreferent and give examples
- discuss uncertainty in how to nest referents from bim fit check report (own nest or same nest with alignment layouts)

Section 7.0 - IfcReferent and Stationing

The class `IfcReferent` defines a position at a particular offset along an alignment curve. Stationing (also known as chainage) is a good example.

Referents are nested to alignments, using `IfcRelNests`. `IfcRelNests.RelatedObjects` is an ordered list so the first referent defines the starting station of the associated `IfcAlignment` related through `IfcRelNests.RelatingObject`.

`IfcRelNests.PredefinedType = STATION` is not well defined in the IFC specification, however it seems most appropriate for referents that only indicate a station.

The stationing value is provided using `Pset_Stationing`. The `Station` property defines the station value at a location. Station equations (or chainage breaks) are defined by providing the `IncomingStation` and `Station` properties.

7.1 IfcRelNests Usage

Alignment layouts and stationing referents both decompose `IfcAlignment` through `IfcRelNests`. The IFC specification is not clear if alignment layouts and referents belong in the same or different nests. Since alignment layout and referent are completely different things, it is recommended that they are contained within their own nest. This is illustrated in Figure 7.1-1.

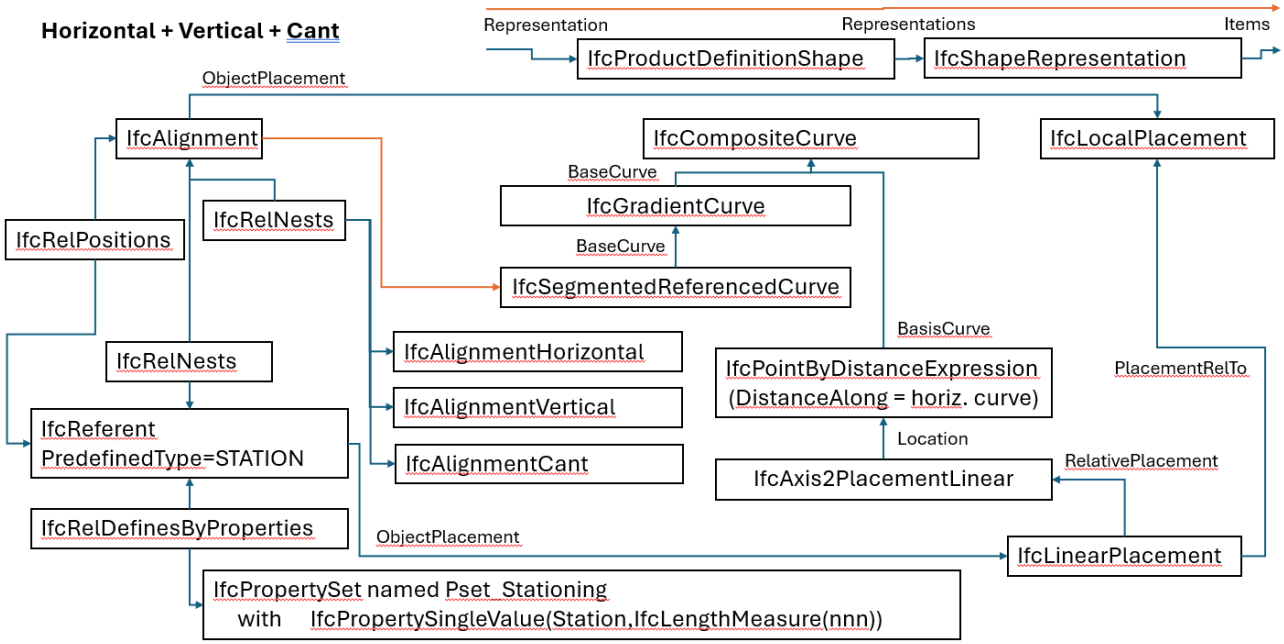

Figure 7.1-1 Recommended approach to nesting alignment layouts and referents

Figure 7.1-1 shows two `IfcRelNests`, one each for alignment layout and referent. Also note the `IfcRelPositions` relationship. The referent defines the stationing and positions the alignment. The case shown is for horizontal, vertical, and cant. Horizontal only and horizontal with vertical are similar.

7.2 Key Alignment Points

Key alignment points, such as Point Of Curvature (PC) of a horizontal curve or Point of Vertical Curve (VPC) for a vertical profile, are defined with an `IfcReferent`. These referents are contained in the `IfcRelNests.RelatedObjects` list. Based on the Object Nesting concept template, all `IfcReferent` nest with the `IfcAlignment`. For this reason the `IfcRelNests.RelatedObjects` list may contain critical points for horizontal and vertical layouts. The

`IfcReferent` must be in order of occurance. Additionally, the `IfcReferent` informs of the position of the curve segment with the `IfcRelPositions` relationship.

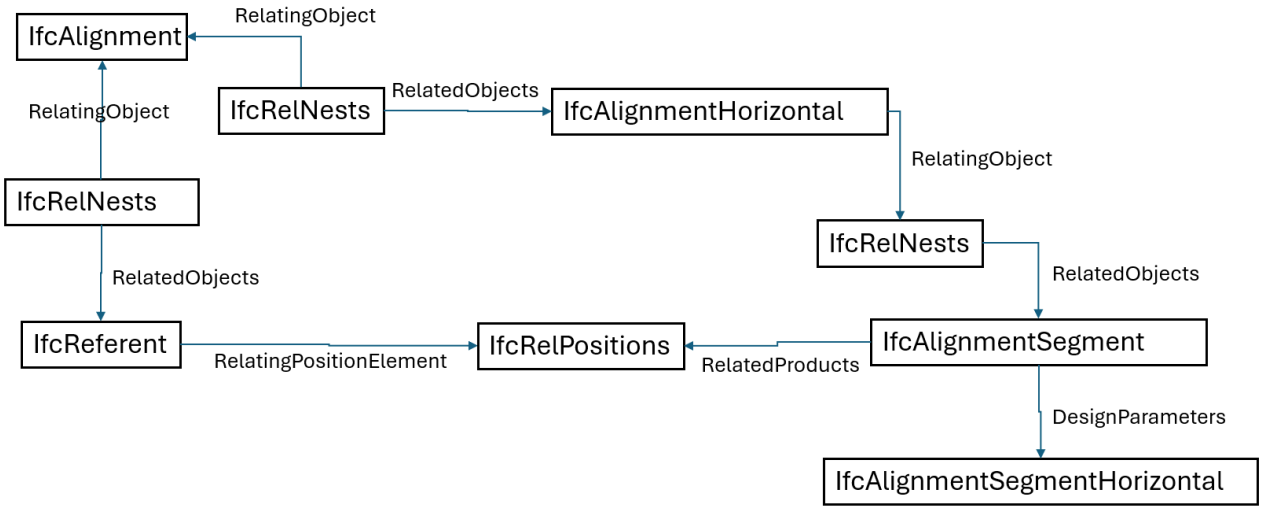

Figure 7.2-1 Example of IfcReferent informing on the position of an IfcAlignmentSegment key point

Section 8.0 - Specialized Alignment-based Infrastructure Geometry

Placeholder - in this section it is intended that there is a discussion of IfcSectionedSurface and IfcSectionedSolidHorizontal. These are two new surface and solid geometries developed specifcially for infrastructure and alignments.

8.1 IfcSectionedSurface

Describe when this surface geometry might be advantagous to use

- Roadway surface
- Visualization of superelevation transitions and widenings

Figure 8.1-1 Superelevation transition

8.1.1 Breaklines

Describe the concept of breaklines for widening sections.

Figure 8.1.1-1 Sectioned surface with breaklines

8.1.2 Stringlines

Describe string lines and how they work (don't have a good implementation yet)

8.2 IfcSectionedSolidHorizontal

- Describe when this geometry may be useful
- Give examples for different kinds of rotations as defined in the documentation
- Discuss error in image from the documentation with projecting geometry along the line
- Describe string lines (guide curves) and how they work.
- Clarify direction (CCW) for defining polygon points for the profile
- Discuss error in documentation of profile orientation and compare it to the text for IfcSectionedSurface. Show my figures from github on the issue

todo:

- Review this section - it is completely generated

Section 9.0 - LandXML to IFC Conversion

9.1 Introduction

LandXML 1.x is the dominant legacy format for civil alignment data; IFC 4x3 is the modern target. The goal of this chapter is to guide implementers through the non-obvious decisions and pitfalls that arise during conversion, drawn from practical experience.

9.2 Schema Conceptual Comparison

The following table provides a high-level mapping of the two data models.

LandXML	IFC 4x3
`<Alignments>` / `<Alignment>`	`IfcAlignment` nested under `IfcProject` via `IfcRelAggregates`
`<CoordGeom>`	`IfcAlignmentHorizontal` nested via `IfcRelNests`
`<Profile>` / `<ProfAlign>`	`IfcAlignmentVertical` nested via `IfcRelNests`
`<Cant>`	`IfcAlignmentCant` nested via `IfcRelNests`
`<StaEquation>`	`IfcReferent` with stationing attributes
`<Units>`	`IfcUnitAssignment` in `IfcProject`

Table 9.2-1 — LandXML to IFC schema conceptual comparison

The key conceptual difference between the two schemas is that LandXML describes geometry procedurally — PI-based for vertical alignment, point-to-point for horizontal — whereas IFC describes segments declaratively as typed business objects with a start point, radii, and length.

9.3 Units

LandXML carries explicit unit declarations (`<Metric>` , `<Imperial>`) for linear and angular measure. IFC requires all geometry in SI base units (metres, radians) or with an explicit `IfcConversionBasedUnit` .

Read `<Units>` before processing any geometry — all coordinate values depend on it.

Linear units. Convert foot and US Survey foot to metres as needed. Note that the US Survey foot and the international foot are not the same:

$$1 \text{ US Survey foot} = \frac{1200}{3937} \text{ m} \approx 0.3048006 \text{ m}$$

$$1 \text{ international foot} = 0.3048 \text{ m (exact)}$$

Using the wrong conversion factor introduces a systematic error that compounds over long alignments.

Angular units. LandXML directions can be expressed in decimal degrees, grads, or radians. `IfcAlignmentHorizontalSegment.StartDirection` is always in radians. Apply the appropriate conversion before writing IFC output.

Bearing convention. LandXML directions are azimuths measured clockwise from North. IFC plane angles are measured counter-clockwise from the positive X-axis (East). The conversion is

$$\theta_{IFC} = \frac{\pi}{2} - \theta_{bearing}$$

where $\theta_{bearing}$ is in radians.

9.4 Horizontal Alignment

9.4.1 LandXML Horizontal Element Types

The LandXML `<CoordGeom>` element contains a sequence of `<Line>`, `<Curve>`, and `<Spiral>` child elements. Each maps to an `IfcAlignmentSegment` nested under `IfcAlignmentHorizontal`.

9.4.2 Direction from Geometry

LandXML lines and spirals often omit explicit direction attributes (`Dir`, `DirStart`) and rely on the geometry — start/end or start/PI points — to imply direction. `IfcAlignmentHorizontalSegment.StartDirection` is required. When the direction attribute is absent, compute it from the available geometry:

$$\theta = \arctan 2(\Delta y, \Delta x)$$

then apply the bearing-to-IFC plane angle conversion from Section 9.3.

9.4.3 Circular Curve (`<Curve>` → `CIRCULARARC`)

`<Curve>` provides center, start, end, and radius. IFC requires the start point, the start direction (the tangent direction, not the radial direction), the signed radius, and the arc length.

Compute the tangent direction from the radial line by rotating 90°:

$$\theta_{tangent} = \arctan 2(y_{start} - y_{center},\ x_{start} - x_{center}) + \text{sign} \cdot \frac{\pi}{2}$$

where sign $= +1$ for counter-clockwise (`rot="ccw"`) and sign $= -1$ for clockwise (`rot="cw"`). The signed radius passed to `IfcAlignmentHorizontalSegment` follows the same sign convention.

9.4.4 Spiral Type Mapping

The following table maps LandXML `spiType` values to `IfcAlignmentHorizontalSegmentTypeEnum`.

LandXML `spiType`	IFC Type	Notes
`clothoid`	`CLOTHOID`	Direct mapping
`bloss`	`BLOSSCURVE`	Direct mapping
`biquadratic`	`HELMERTCURVE`	Direct mapping
`biquadraticParabola`	`HELMERTCURVE`	Equivalent to biquadratic
`cubic`	`CUBIC`	Direct mapping
`cubicParabola`	`CUBIC`	Equivalent to cubic
`cosine`	`COSINECURVE`	Direct mapping
`sinusoid`	`SINECURVE`	Direct mapping
`sineHalfWave`	`COSINECURVE`	Approximation — loss of fidelity, see note below
`japaneseCubic`	`VIENNESEBEND`	Direct mapping
`revBiquadratic`	—	No IFC equivalent, log warning and skip
`revBloss`	—	No IFC equivalent, log warning and skip
`revCosine`	—	No IFC equivalent, log warning and skip
`revSinusoid`	—	No IFC equivalent, log warning and skip
`radioid`	—	No IFC equivalent, log warning and skip
`weinerBogen`	—	No IFC equivalent, log warning and skip

Table 9.4.4-1 — LandXML spiType to IFC type mapping

> **Note on `sineHalfWave`.** The LandXML specification describes the sine half-wavelength spiral as an approximation of a cosine spiral. Mapping it to `COSINECURVE` is therefore reasonable but not exact. Implementations should log a warning when this substitution is made.

9.4.5 Radius Sign Convention

LandXML uses `rot="cw"` / `rot="ccw"` to indicate curve direction. IFC encodes direction in the sign of `StartRadius` and `EndRadius`: a positive value indicates a left turn (counter-clockwise) and a negative value indicates a right turn (clockwise).

$$\text{sign} = \{-1 \quad \text{if } rot = \texttt{cw} \; +1 \quad \text{if } rot = \texttt{ccw}$$

Apply this sign to both `StartRadius` and `EndRadius`.

9.4.6 Infinite Radius

LandXML represents an infinite start or end radius by omitting the attribute or by supplying `DBL_MAX`. IFC uses `0.0` in `IfcAlignmentHorizontalSegment.StartRadius` or `EndRadius` to indicate an infinite radius. Substitute `0.0` whenever the LandXML value is absent or exceeds a practical threshold.

9.4.7 Unhandled Horizontal Element Types

`<Chain>` and `<IrregularLine>` elements have no direct equivalent in the IFC alignment semantic model. Log a warning when these elements are encountered and skip them. If approximate geometric fidelity is acceptable, a `<Chain>` (polyline) could be decomposed into a sequence of `LINE` segments.

9.5 Vertical Alignment

9.5.1 LandXML Vertical Data Model

LandXML represents vertical geometry as a sequence of PVI-based elements under `<ProfAlign>`: `<PVI>`, `<ParaCurve>`, `<UnsymParaCurve>`, and `<CircCurve>`. This is fundamentally different from IFC's segment-based model, which requires an explicit start station, start elevation, start grade, end grade, and length for each segment.

9.5.2 PVI-Based to Segment-Based Conversion

The core translation challenge is that LandXML locates vertical curves by their PVI station and curve length, while IFC needs each segment's start station explicitly. The conversion procedure is:

1. Compute the entry grade from the PVI station and the previous PVI or curve endpoint.
2. The start station of a symmetric parabolic curve is $\text{PVI station} - L/2$.
3. The start elevation is computed by projecting back from the PVI along the entry grade: $e_{start} = e_{PVI} - g_1 \cdot L/2$.
4. Gradient segments fill any gap between the end of one curve and the start of the next.

9.5.3 Symmetric Parabolic Arc (`<ParaCurve>` → `PARABOLICARC`)

This is the standard vertical curve case. Given PVI station s_{PVI}, PVI elevation e_{PVI}, curve length L, entry grade g_1, and exit grade g_2:

$$s_{start} = s_{PVI} - \frac{L}{2}$$

$$e_{start} = e_{PVI} - g_1 \frac{L}{2}$$

Map to `IfcAlignmentVerticalSegment` with `PredefinedType = PARABOLICARC`, `StartDistAlong = s_{start} - s_{alignment\_start}`, `StartHeight = e_{start}`, `StartGradient = g_1`, `EndGradient = g_2`, `HorizontalLength = L`.

9.5.4 Asymmetric Parabolic Arc (`<UnsymParaCurve>` → Two `PARABOLICARC` Segments)

IFC has no asymmetric vertical curve type. The `<UnsymParaCurve>` must be split into two abutting `PARABOLICARC` segments. Given entry length L_{in}, exit length L_{out}, entry grade g_1, and exit grade g_2:

Compute the vertical offset from the composite curve to the PVI:

$$h = \frac{L_{in} \cdot L_{out} \cdot (g_2 - g_1)}{2(L_{in} + L_{out})}$$

The transition point between the two sub-curves occurs at the PVI station with elevation:

$$e_{transition} = e_{PVI} + h$$

The intermediate grade at the transition point is:

$$g_{mid} = g_1 + \frac{2h}{L_{in}}$$

Create two `PARABOLICARC` segments: the first from the curve start to the PVI station with grades g_1 and g_{mid}, and the second from the PVI station to the curve end with grades g_{mid} and g_2.

9.5.5 Circular Vertical Curve (`<CircCurve>` → `CIRCULARARC`)

LandXML provides the radius and arc length. IFC `IfcAlignmentVerticalSegment` for `CIRCULARARC` accepts the radius directly in the `RadiusOfCurvature` attribute. Compute the start station and elevation the same way as for a symmetric parabolic arc, using $L/2$ as the half-length. Note that this is an approximation — the true tangent length of a circular vertical curve is $T = R\tan(\Delta/2)$ where $\Delta = L/R$, which differs from $L/2$ except for small deflection angles.

9.5.6 Start Station Offset

LandXML stations are absolute. `IfcAlignmentVerticalSegment.StartDistAlong` is measured from the start of the alignment. Subtract the alignment start station from every LandXML station:

$$\text{StartDistAlong} = s_{LandXML} - s_{alignment_{s}tart}$$

The alignment start station is found in `<Alignment staStart="...">` and must be read before processing any profile elements.

9.6 Cant

9.6.1 LandXML Cant Model

LandXML `<Cant>` uses `<CantStation>` points and `<CantSegment>` elements with left and right cant values at the start and end of each segment, along with a curve type designation.

9.6.2 Mapping to `IfcAlignmentCant`

Each `<CantSegment>` maps to an `IfcAlignmentCantSegment` with `StartCantLeft`, `StartCantRight`, `EndCantLeft`, `EndCantRight`, and `PredefinedType`. The cant segment type mapping follows the same spiral type table as horizontal alignment (Section 9.4.4).

9.6.3 Rail Head Distance

`IfcAlignmentCant.RailHeadDistance` is required and represents the distance between rail heads (track gauge). LandXML may carry gauge information in a `<Roadway>` or project-level element, or it may need to be supplied as an external parameter. The conversion must account for this value explicitly — it is used in the Viennese Bend cant factor calculation described in Section 2.

9.7 Stationing and Station Equations

9.7.1 Start Station

The LandXML `<Alignment staStart="...">` attribute gives the station at the beginning of the alignment. Map this to an `IfcReferent` with `PredefinedType = REFERENCEMARKER` at `DistanceAlong = 0.0`, with the station value stored as a property in an associated property set.

9.7.2 Station Equations (`<StaEquation>`)

LandXML `<StaEquation>` records station breaks where the back station and ahead station differ. Each equation maps to an additional `IfcReferent`. The relevant attributes are:

LandXML Attribute	Meaning
`staBack`	Station value before the equation (incoming)
`staAhead`	Station value after the equation (outgoing)
`staInternal`	Continuous internal station (no breaks)

Table 9.7.2-1 — LandXML station equation attributes

The `staInternal` value should equal `staAhead` in a well-formed LandXML file. Log a warning if `staInternal != staAhead`, as this indicates an unusual stationing condition that may require manual review.

9.8 IFC Project Structure

LandXML has no concept of project hierarchy. IFC requires a minimum entity graph regardless of what is in the LandXML file. The required structure is:

```
IfcProject
  └─ IfcRelAggregates
        ├─ IfcSite
        └─ IfcAlignment
              └─ IfcRelNests
                    ├─ IfcAlignmentHorizontal
                    │     └─ IfcRelNests → IfcAlignmentSegment (×n)
                    ├─ IfcAlignmentVertical
                    │     └─ IfcRelNests → IfcAlignmentSegment (×n)
                    └─ IfcAlignmentCant (if cant data present)
                          └─ IfcRelNests → IfcAlignmentSegment (×n)
```

The `IfcProject` must include an `IfcUnitAssignment` derived from the LandXML `<Units>` element (Section 9.3). The `IfcSite` provides the spatial context and local placement for the alignment.

9.9 Known Limitations and Unresolved Issues

The following issues are known limitations of the LandXML-to-IFC conversion and should be documented for users of any converter implementation.

Unhandled spiral types. The LandXML spiral types `radioid`, `weinerBogen`, `revBiquadratic`, `revBloss`, `revCosine`, and `revSinusoid` have no IFC equivalent. Segments of these types are skipped with a warning.

Unhandled horizontal element types. `<Chain>` (polyline) and `<IrregularLine>` elements are skipped. A future implementation could decompose them into sequences of `LINE` segments.

`sineHalfWave` approximation. Mapped to `COSINECURVE` as an approximation. The two curves are similar but not identical.

Circular vertical curve half-length approximation. The start station of a `<CircCurve>` is approximated as $s_{PVI} - L/2$. The exact value depends on the tangent length, which requires iteration for large deflection angles.

Asymmetric parabolic arc split. The split of `<UnsymParaCurve>` into two `PARABOLICARC` segments is geometrically correct but results in two IFC segments where LandXML had one. Downstream tools must be able to handle this.

Imperial units. Imperial linear unit conversion (foot, US Survey foot) must be applied to all coordinate values. Partial implementations that read the unit declaration but do not scale the coordinates will produce incorrect geometry.

No support for LandXML surface or roadway data. `<Roadway>`, `<Survey>`, `<Surfaces>`, and other LandXML elements beyond alignment geometry are out of scope for an alignment-focused converter.

todo:

- for the options of IfcCurveSegment.Transition in this section, put in a tabulated format and add the figure from the IFC documentation https://ifc43-docs.standards.buildingsmart.org/IFC/RELEASE/IFC4x3/HTML/figures/ifctransitioncode.gif

Section 10.0 - Precision and Tolerance

`IfcCurveSegment.Transition` defines the intended transition from the end of one segment to the start of the next. The options are:

CONTINUOUS: The segments joint but no condition on their tangents is implied.

CONTSAMEGRADIENT: The segments join and their tangent vectors or tangent planes are parallel and have the same direction at the joint; equality of derivative is not required.

CONSTSAMEGRADIENTSAMECURVATURE: For a curve, the segments join, their tangent vectors are parallel and in the same direction and their curvatures are equal at the joint: equality of derivatives is not required. For a surface this implies that the principle curvatures are the same and the principle directions are coincident along the common boundary.

DISCONTINUOUS: The segments do not join. This is permitted only at the boundary of the curve or surface to indicate that it is not closed.

For a variety of reasons, adjacent segment end and start points might not exactly line up in position, gradient, or curvature.

`Pset_Tolerance` is used to indicate the acceptable tolerance under which the calculated end point of a preceding segment would be considered geometrically continuous with the provided start point of the following segment. Similarly, `Pset_Uncertainty` is used to indicate the uncertainty inherent in historical information gleaned from plan sets and other historic sources. As a fallback if `Pset_Tolerance` is not provided, tolerance can be taken from `IfcGeometricRepresentationContext.Precision`.

todo:

- Develop this Section - the idea is to describe the content of the example folders developed by Andres
- Examples were developed by Andres (P - last name) as part of the buildingSMART Germany chapter's BIM Fit Check competition.

Section 11.0 - Examples and Validation Data